CENTRO DE INVESTIGACIÓN Y DE ESTUDIOS AVANZADOS DEL INSTITUTO POLITÉCNICO NACIONAL

UNIDAD ZACATENCO

DEPARTAMENTO DE MATEMÁTICAS

Algoritmo MAPPER: una aplicación de las técnicas de la topología algebraica al análisis de datos

Tesis que presenta

Kelly Jhoanna Botello Badel

Para Obtener el Grado de

Maestra en Ciencias

En la Especialidad de

Matemáticas

Director de la Tesis: Dr. Jesús González Espino Barros

Co-directora: Dra. Adriana Lara Lopez

Ciudad de México Noviembre, 2023

A Dios, a mi mami Lucy, a mi abuela Gladys y a mi amado Brandon.

ÍNDICE GENERAL

Resumen VIII

Abstract 1

Introducción 2

1. El algoritmo Mapper 5

1.1. Preliminares 5

1.2. El algoritmo mapper 6

1.2.1. FUNCIONES FILTRO 12

1.2.2. CUBIERTA 15

1.2.3. AGRUPACION, MAPPER Y VISUALIZACION 17

2. Aplicación del algoritmo Mapper al estudio de la diabetes 24

2.1. Normalización de los datos 26

2.2. Filtro: Estimación de densidad usando un kernel Gaussiano 27

2.3. Cubierta en $\mathbb{R}$ 27

2.4. Experimentos 29

2.4.1. LOW RESOLUTION 29

2.4.2. HIGH RESOLUTION 31

3. Aplicación del algoritmo Mapper a la base de datos del toro 33

3.1. Filtros: Proyección y Laplaciano 35

3.1.1. FILTRO DE PROYECCION 35

3.1.2. FILTRO DEL LAPLACIANO 36

3.2. Cubierta en $\mathbb{R}^2$ 43

3.3. Experimentos 47

3.3.1. BASE DE DATOS ALEATORIA: FILTRO DE PROYECCION 47
3.3.2. BASE DE DATOS CONSTRUIDA: FILTRO DEL LAPLACIANO 51

4. Base de datos calidad del aire 53
4.1. Normalización de los datos 55
4.2. Filtro: Estimación de densidad usando un kernel Gaussiano 57
4.3. Cubierta en $\mathbb{R}$ 57
4.4. Agrupación y mapper 58
4.5. Experimentos 59
4.5.1. BASE DE DATOS CRITICA 59
4.5.2. BASE DE DATOS GENERAL 69
4.6. Resultados obtenidos 75

5. Conclusiones 80

AGRADECIMIENTOS

Agradezco en primer lugar a Dios, pues es gracias a Él y a su regalo de la vida, la salud y la sabiduría que este objetivo fue alcanzado. Muchas gracias a mi mamá Lucy y a mi abuela Gladys, quienes han estado para mí y me han apoyado en mis 24 años de vida. Agradezco a mi novio Brandon, quien con su amor y compañía desde el inicio me motivó y me guió a seguir adelante y tomar el mejor camino a mis metas, que también son suyas. Muchas gracias a mis amigas por siempre creer en mi y alentarme, en especial a mi colega Valeria y su apoyo desde que comenzamos la carrera en el pregrado. Agradezco enormemente a mis amigos del posgrado, a Daniel por brindarme su amistad sincera y compartir jornadas de estudio conmigo, a David por su apoyo en los cursos de mi primer año. Muchas gracias además a mis compañeros Fernando y Omar quienes jugaron un papel fundamental en este proceso, este trabajo es de los tres.

Muchas gracias al Dr. Miguel Alejandro Xicoténcatl y al Dr. Ruy Fabila, a quienes le debo mi entrada a la maestría. Agradezco enormemente a mis asesores el Dr. Jesús González y la Dra. Adriana Lara quienes estuvieron de manera constante y dispuesta a resolver mis dudas y brindarme los conocimientos necesarios para lograr este objetivo. Especialmente al profesor Jesús, quien entendió perfectamente mis intereses y me entregó el mejor tema de tesis alineado a éstos, muchísimas gracias. De igual manera agradezco enormemente a mis profesores de la maestría, especialmente al profesor Ruy por su curso de matemáticas discretas y al profesor Feliú Sagols por sus dos cursos de programación, cruciales para poder realizar mi trabajo en TDA y por su disposición a ser parte del jurado en mi examen de grado junto con el Dr. Juan Manuel Burgos.

Por último, muchas gracias al Consejo Nacional de Humanidades Ciencias y Tecnologías (CONAHCYT) por la beca que me permitió permanecer y sostenerme durante dos años sin falta, al igual que el Centro de Investigación y de Estudios Avanzados del IPN (CINVESTAV) por brindarme una oportunidad sin igual en mi vida profesional.

Resumen

En el campo de la ciencia de datos es cada vez más común recurrir al análisis topológico de datos (TDA) debido a sus herramientas para la visualización, clasificación y reducción de dimensionalidad. En este trabajo se estudia uno de los procesos más aplicados en TDA, el algoritmo Mapper [1] y se realizan tres aplicaciones de éste. En primer lugar, se abordan los conceptos y resultados de la topología algebraica que le dan sentido al algoritmo y su procedimiento. Posteriormente, se detalla y sigue una aplicación del algoritmo Mapper sobre la base de datos de la diabetes presentada en [1], realizando adaptaciones propias y utilizando una implementación original del algoritmo. De manera similar, se aplica el algoritmo a la base de datos sobre la superficie del toro, cuyas adaptaciones optimizan el proceso para obtener una mejor visualización en el mapper resultante. Por último, se aporta en este trabajo una aplicación del algoritmo Mapper a la base de datos de la calidad del aire en Ciudad de México. Todos los experimentos realizados obtuvieron resultados favorables e interesantes en términos de visualización y clasificación de los datos, además de aportes útiles para posibles futuras investigaciones derivadas de este trabajo.

Abstract

In the field of data science, topological data analysis (TDA) is becoming increasingly popular due to its tools for visualization, classification and dimensionality reduction. In this document we study one of the most widely applied processes in TDA, the Mapper algorithm [1] and three applications of this algorithm are performed. First, we discuss the concepts and results of algebraic topology that give sense to the algorithm and its procedure. Subsequently, we detail and follow an application of the Mapper algorithm on the diabetes database presented in [1], making our own adaptations and using an original implementation of the algorithm. Similarly, the algorithm is applied to the torus surface database, whose adaptations optimize the process to obtain a better visualization in the resulting mapper. Finally, we provide in this document an application of the Mapper algorithm to the air quality database in Mexico City. All the experiments performed obtained favorable and interesting results in terms of data visualization and classification, as well as useful contributions for possible future research derived from this work.

INTRODUCCIÓN

El análisis topológico de datos (TDA) es el campo que busca interpretar un espacio de entrada a partir del análisis de su forma o sus características principales. Surge de la idea de incluir herramientas computacionales y tópicos de la topología algebraica y la geometría para el análisis de datos, partiendo desde el análisis sobre espacios topológicos hasta su forma discreta, el análisis topológico de nubes de datos. Desde sus inicios, el análisis topológico de datos ha tenido como objetivo principal conservar características topológicas de la entrada (como por ejemplo su conexidad o la identificación de sus componentes conexas) y realizar una simplificación de ésta durante el proceso. Entre los primeros métodos estudiados en el campo se encuentra la homología persistente [2], [3], donde se abordan conceptos tales como complejo simplicial, filtración, cubierta, cluster y nervio. A partir de este punto, se comenzaron a introducir más métodos y algoritmos con el mismo objetivo: entender los datos principalmente a través de la visualización [4]. La persistencia general, los grafos de Reeb, la teoría de Morse discreta y el algoritmo Mapper son algunos de estos métodos y son explicados en [5]. Adicionalmente, diversos estudios en análisis topológico de datos han tenido un enfoque en la ciencia de datos, estadística y machine learning [4], por lo que este campo ha tenido múltiples aplicaciones en diferentes áreas. En este trabajo se estudiará uno de los métodos mencionados, el algoritmo Mapper.

Actualmente, en diversos procesos computacionales se manejan cantidades masivas de datos y, en la mayoría de los casos, esta información es presentada en forma de conjuntos de datos multidimensionales lo que hace menos intuitivo su análisis o visualización. Lo que busca el algoritmo Mapper es lo que busca el TDA, visualizando dicho conjunto de datos de manera simplificada en forma de un complejo simplicial, además de haber mostrado resultados favorables en términos de clasificación de características relevantes del conjunto de datos de entrada. Introducido por Singh, Mémoli y Carlsson en [1], el algoritmo Mapper tomó fuerza debido a su versatilidad en diversos problemas del análisis de datos adoptando

procesos de TDA de la siguiente manera: a partir de un conjunto finito de datos X, se aplica un filtro f para reducir su dimensión y, considerando una cubierta de los datos filtrados y sus preimágenes en f, se construye el complejo simplicial resultante que consiste en el nervio de las componentes conexas de dichas preimágenes.

Construir un grafo o un complejo simplicial a partir de un conjunto de datos o un espacio topológico no es un método propio del algoritmo Mapper, otros procesos en TDA como el complejo de Vietoris-Rips y de Cech (sección 2.2 en [5]) o el grafo de Reeb (capítulo 7 en [5]) obtienen un grafo o complejo simplicial como salida, con variaciones en cada proceso y un concepto en común: *nervio*. Un resultado importante en el análisis topológico de datos, el Teorema del Nervio (teorema 1.7) es relevante para darle sentido al algoritmo Mapper y será explicado de manera más detallada en el capítulo 1. Previos al Mapper e incluso a TDA, existen métodos con un objetivo similar de visualización de datos y reducción de dimensionalidad como el método de la búsqueda de proyección (projection pursuit method) [6], el cual fue aplicado por Miller y Reaven para el conjunto de datos del experimento que se describe en el capítulo 2 y cuyos resultados fueron comparados con el mapper resultante del mismo conjunto de datos en [1] y en este trabajo. Además, procesos como la estimación de densidad [7] o las eigenfunciones laplacianas [8] se han utilizado en múltiples estudios para reducción de dimensionalidad, incluso para la aplicación del algoritmo Mapper como filtro. Adicionalmente, herramientas como multidimensional scaling [9] o isomap [10] aportan a su vez procesos para reducción de dimensionalidad y visualización de datos en un espacio Euclidiano. Estos métodos son útiles y han sido utilizados en estudios importantes del análisis de datos, sin embargo, el algoritmo Mapper aporta además como salida un complejo simplicial, el cual aparte de ser una estructura útil para conservar y visualizar relaciones entre agrupaciones de datos (por ejemplo representadas con una arista que une dos vértices, donde cada vértice representa una agrupación de datos) es más manejable o “amigable” para su posterior análisis e interpretación, además de requerir un menor costo computacional en su construcción.

Como se mencionó anteriormente, el algoritmo Mapper ha tenido diversas aplicaciones en el análisis de datos. Entre las aplicaciones para visualización y clasificación de datos se encuentran múltiples estudios en el campo de la biología y la salud, por ejemplo estudios sobre el cáncer [11], COVID [12], plegamiento molecular [13] o diabetes [1], estudios de visualización de datos como procesamiento de imágenes [14] e incluso en el campo de los deportes [15]. Más aplicaciones del algoritmo Mapper sobre diferentes conjuntos de datos para visualización o clasificación se encuentran en [1], [16] y [17]. Así mismo, se han

desarrollado diversas implementaciones del algoritmo Mapper como `Keppler Mapper` [18], una librería que implementa el algoritmo Mapper y genera grafos mappers resultantes al igual que `Giotto-tda` [19], `Python Mapper` [20] y `Mapper Interactive` [21]. En este trabajo se utilizará la librería `Keppler Mapper` y una implementación original del algoritmo en el lenguaje de programación `Python`, adoptando librerías útiles para el proceso como `Sklearn` [22], `Matplotlib` [23] y `Networkx` [24].

Este trabajo consta de cuatro capítulos. En el primer capítulo se explica el algoritmo Mapper y sus bases matemáticas, los capítulos dos y tres muestran y siguen (no de manera exacta) dos aplicaciones del algoritmo Mapper expuestas en [1] y, por último, se aportará una nueva aplicación de éste a la base de datos de la calidad del aire en Ciudad de México en el capítulo cuatro. Sobre este último, existen estudios y análisis previos sobre los conjuntos de datos de la calidad del aire como ocurre en [25], sin embargo involucra procesos estadísticos pero no directamente al algoritmo Mapper.

1

El algoritmo Mapper

El algoritmo Mapper es un método utilizado en el análisis topológico de datos para la visualización e interpretación de las características topológicas de un conjunto de datos, donde dicha visualización se muestra en forma de un grafo o de un complejo simplicial con la finalidad de hacer más sencillo el análisis y el procesamiento computacional del conjunto de datos utilizado. Fue introducido por Singh, Mémoli y Carlsson [1] quienes buscaban analizar conjuntos de datos de altas dimensiones y reconocer objetos tridimensionales (representados también como conjuntos de datos multidimensionales), todo con una visualización de baja dimensionalidad. Por tanto, el grafo o complejo simplicial resultante debe conservar orificios, componentes conexas, ramas, entre otras características topológicas del conjunto de datos original. Las siguientes definiciones y resultados de la sección (1.1) fueron extraídos de [5] y [26].

1.1 Preliminares

Definición 1.1. (Simplejo). Un simplejo de dimensión $k \geq 0$, denotado como σ, es el casco convexo de $k+1$ puntos afinamente independientes en $\mathbb{R}^n$. Es decir, para $P = \{p_0, ..., p_k \mid p_i \in \mathbb{R}^n\}$, σ contiene a todos los puntos de la forma $x = \sum_{i=0}^{k} \alpha_i p_i$, con $\sum_{i=0}^{k} \alpha_i = 1$ y $\alpha_i \geq 0$, $i \in \{0, ..., k\}$. P es entonces el conjunto de vértices que conforman a σ, también denotado en su forma discreta como $\sigma = \{p_0, ..., p_k\}$. Así mismo, una *cara* de σ es un simplejo generado por un subconjunto de P.

Definición 1.2. (Complejo simplicial). Un complejo simplicial K, también conocido como triangulación, es un conjunto finito de simplejos que satisface:

(i) K contiene a todas las caras de cada simplejo en K.

(ii) Para $\sigma, \tau \in K$, $\sigma \cap \tau = \emptyset$ o $\sigma \cap \tau$ es una cara tanto de σ como de τ.

Definición 1.3. (Complejo de Rips). Sea (X, d) un espacio métrico finito. Para $r > 0$ con $r \in \mathbb{R}$, el complejo de Vietoris-Rips se obtiene tomando como conjunto de vértices a X, añadiendo una arista entre $x_1, x_2 \in X$ si se cumple que $d(x_1, x_2) \leq r$ e incluyendo el simplejo $\sigma = \{x_1, ..., x_n\}$ si el complejo contiene a todas las aristas que lo componen, es decir, si para cualesquiera $x_i, x_j \in \sigma$ se cumple que $d(x_i, x_j) \leq r$.

Definición 1.4. (Nervio). Sea $\mathcal{U} = \{U_\alpha\}_{\alpha \in I}$ una familia finita de conjuntos. El nervio de $\mathcal{U}$, denotado como $N(\mathcal{U})$, es el complejo simplicial que resulta de tomar como conjunto de vértices a cada $\alpha \in I$ (es decir, cada α representa a uno de los miembros de la familia) y donde $\{\alpha_0, ..., \alpha_k\} \subseteq I$ forman un simplejo de orden k en $N(\mathcal{U})$ si y sólo si $U_{\alpha_0} \cap ... \cap U_{\alpha_k} \neq \emptyset$.

Definición 1.5. (Homotopía). Sean X, U espacios topológicos y sean $g : X \to U$ y $h : X \to U$ dos mapeos continuos. Una homotopía es un mapeo $H : X \times [0,1] \to U$ tal que $H(\cdot, 0) = g$ y $H(\cdot, 1) = h$. Dos mapeos son homotópicos si existe una homotopía que los conecta.

Definición 1.6. (Equivalencia homotópica). Dos espacios topológicos X y U son homotópicamente equivalentes si existen $g : X \to U$ y $h : U \to X$, mapeos tales que $h \circ g$ es homotópica al mapeo identidad $i_X : X \to X$ y $g \circ h$ es homotópica al mapeo identidad $i_U : U \to U$.

Teorema 1.7. ***(Teorema del Nervio).*** *Dado un espacio métrico M y una cubierta finita $\mathcal{U}$ de M, $N(\mathcal{U})$ es equivalente homotópicamente a M si toda intersección no vacía $\bigcap_{i=0}^{k} U_{\alpha_i}$, donde cada U_{α_i} es un elemento de la cubierta, es equivalente homotópicamente a un punto, es decir, es contráctil.*

Definición 1.8. (Conexidad por arcos). Un espacio topológico X es conexo por arcos si cualesquiera dos puntos $x, y \in X$ pueden ser unidos por un camino, esto es, existe un mapeo continuo $f : [0,1] \to X$ tal que $f(0) = x$ y $f(1) = y$.

1.2 El algoritmo Mapper

Comenzamos definiendo una función continua $f : X \to \mathbb{R}^n$ con X un espacio topológico y una cubierta abierta finita $\mathcal{U} = \{U_\alpha\}_{\alpha \in I}$ de $f(X)$. Como f es continua, cada $f^{-1}(U_\alpha)$ es un abierto en X por lo que $\mathcal{V} = \{f^{-1}(U_\alpha)\}_{\alpha \in I}$ es una cubierta abierta de X. Además, escribimos a cada $f^{-1}(U_\alpha)$ como unión de sus componentes conexas por arcos, así, obtenemos una cubierta

abierta $\overline{\mathcal{V}}$ de X a partir de $\mathcal{U}$ vía f, donde cada miembro es conexo por arcos. Luego, el *mapper* de X es el nervio de $\overline{\mathcal{V}}$ obtenido como se describe en la definición 1.4. El objetivo de construir el nervio y obtener el mapper resultante es la representación como un complejo simplicial, esto a manera de visualizar de manera simplificada a X.

Si observamos detenidamente, logramos notar que lo que enuncia el teorema del Nervio es similar a lo que busca el algoritmo Mapper. Sin embargo, para aplicar el teorema del Nervio la cubierta del espacio debe satisfacer las propiedades específicas en las hipótesis del Teorema 1.7, en cuyo caso la cubierta se suele llamar una "buena cubierta" y no en todos los casos es fácil la búsqueda de ésta. Es entonces cuando la aplicación del algoritmo Mapper requiere propiedades menos estrictas (específicamente, que los elementos de la cubierta sean conexos por arcos o se puedan escribir como unión de conjuntos conexos por arcos) para el uso de la cubierta del espacio y aún así preservar al menos parte de la estructura del conjunto de datos original, aunque no se logre la equivalencia homotópica con el nervio obtenido. De hecho, se logra un epimorfismo de los grupos de homología en dimensión uno. En los capítulos 2 y 3 de [5] se da un tratamiento adecuado de homología y persistencia, y además se detalla lo que describiremos a continuación.

Sea X un espacio topológico conexo por arcos y $\mathcal{U} = \{U_\alpha\}_{\alpha \in A}$ una cubierta finita de X, con cada U_α conexo por arcos. Definimos entonces el espacio

$$X_{\mathcal{U}} := \bigsqcup_{\sigma \in N(\mathcal{U})} U_\sigma \times |\sigma| \big/ \sim$$

donde $\sigma = \{\alpha_0, ..., \alpha_n\} \in N(\mathcal{U})$, con cada α_i vértice en $N(\mathcal{U})$ por lo que $U_\sigma = U_{\alpha_0} \cap ... \cap U_{\alpha_n} \neq \emptyset$, y $|\sigma|$ es la realización geométrica de σ en el nervio. Además, consideremos a $\delta_i \sigma = \{\alpha_0, ..., \hat{\alpha}_i, ..., \alpha_n\}$ como la cara opuesta al vértice α_i. Luego, cada elemento $(x, p) \in \bigsqcup_{\sigma \in N(\mathcal{U})} U_\sigma \times |\sigma|$ con $x \in U_\sigma$ y $p \in |\delta_i \sigma| \subset |\sigma|$ está identificado vía $\sim$ con el correspondiente elemento (x, p) en $U_{\delta_i \sigma} \times |\delta_i \sigma|$, pues $U_\sigma \subset U_{\delta_i \sigma}$. Particularmente, si $\sigma = \{\alpha\}$ y $x \in U_\alpha = U_\sigma$, entonces

$$x_\alpha = [(x, \alpha)] \in X_{\mathcal{U}} \tag{1.1}$$

es la clase de x en el vértice α. Consideremos ahora el diagrama

$$\begin{tikzcd}
 & X_{\mathcal{U}} \arrow[dr, "\pi"] & \\
X \arrow[ur, "\xi"] \arrow[rr, "\Phi_{\mathcal{U}}"] \arrow[uu, bend left, "p"] & & |N(\mathcal{U})|
\end{tikzcd}$$

Figura 1.1: Mapeos correspondientes entre los tres espacios señalados.

donde p corresponde al mapeo proyección a la primera componente, es decir, para $[(x,p)] \in X_{\mathcal{U}}$, $p([(x,p)]) = x$ y π corresponde a la proyección a la segunda componente. Adicionalmente, supongamos que X es compacto, por lo tanto, existe una partición de la unidad $\{\varphi_\alpha\}_{\alpha \in A}$ subordinada a $\mathcal{U}$ tal que

1. $\forall \alpha : \varphi_\alpha : X \to [0,1]$ es continua y se cumple que $\overline{\varphi_\alpha^{-1}(0,1]} \subseteq U_\alpha$.
2. $\forall x \in X : \varphi_\alpha(x) = 0$ para casi todo $\alpha \in A$ y $\sum_\alpha \varphi_\alpha(x) = 1$.

Luego, definimos a ξ como

$$\xi(x) = \sum_\alpha \varphi_\alpha(x) x_\alpha = \left[\left(x, \sum_\alpha \varphi_\alpha(x)\alpha\right)\right],$$

donde los sumandos que pueden ser no nulos son los correspondientes a los α (vértices) de la realización geométrica del simplejo en $|N(\mathcal{U})|$ que contiene a x.

Hecho 1.9. X es equivalente homotópicamente a $X_{\mathcal{U}}$ vía ξ y p.

Idea general de la prueba. Por un lado, tenemos claramente que $p \circ \xi = 1$, donde 1 representa el mapeo identidad en X. Por otro lado, para $\left[\left(x, \sum_\alpha t_\alpha \alpha\right)\right] \in X_{\mathcal{U}}$ tenemos que $p\left(\left[\left(x, \sum_\alpha t_\alpha \alpha\right)\right]\right) = x$ y $\xi(x) = \left[\left(x, \sum_\alpha \varphi_\alpha(x)\alpha\right)\right]$, donde no necesariamente los sumandos de la segunda componente son iguales. Sin embargo, notemos que todo ocurre en el mismo simplejo que contiene a x (el cual no cambia), por lo que podemos realizar la identificación $\left[\left(x, \sum_\alpha \varphi_\alpha(x)\alpha\right)\right] \sim \left[\left(x, \sum_\alpha t_\alpha \alpha\right)\right]$ o, geométricamente, en $X_{\mathcal{U}}$ todo ocurre en el mismo simplejo que contiene a x (el cual es convexo), por lo tanto, es posible ir de $\sum_\alpha \varphi_\alpha(x)\alpha$ a $\sum_\alpha t_\alpha \alpha$ a través de la línea que los une en el simplejo común al que pertenecen. Luego, $\xi \circ p \simeq 1$. En conclusión, X es equivalente homotópicamente a $X_{\mathcal{U}}$. $\square$

Finalmente, definimos $\Phi_{\mathcal{U}} : X \to |N(\mathcal{U})|$ como $\Phi_{\mathcal{U}} = \pi \circ \xi$, es decir

$$\Phi_{\mathcal{U}}(x) = \sum_\alpha \varphi_\alpha(x)\alpha. \tag{1.2}$$

La demostración usual del Teorema del Nervio consiste en ver que $\Phi_{\mathcal{U}}$ es una equivalencia homotópica debido al hecho 1.9 y a que $\pi : X_{\mathcal{U}} \to |N(\mathcal{U})|$ resulta ser una equivalencia homotópica cuando $\mathcal{U}$ es una buena cubierta. Para los fines del algoritmo Mapper, en el que usualmente no se puede verificar la hipótesis de que $\mathcal{U}$ sea una buena cubierta, resultan útiles los siguientes hechos:

Lema 1.10. *Si X es un espacio topológico conexo por arcos, entonces $|N(\mathcal{U})|$ es conexo y por lo tanto, $\Phi_{\mathcal{U}}$ induce un isomorfismo en H_0*[1]

$$\Phi_{\mathcal{U}} : H_0(X) = \mathbb{Z}_2 \to \mathbb{Z}_2 = H_0(|N(\mathcal{U})|).$$

Idea general de la prueba. Sean $p, q \in |N(\mathcal{U})|$ con $p \neq q$. Consideremos la línea (o curva) que une a p con un vértice de la cara a la que pertenece en el nervio, digamos v, y de manera similar procedemos con q uniéndolo a un vértice w tal que $v \neq w$. Sean ahora $x \in U_v$ y $y \in U_w$, donde U_v es el elemento (abierto) de la cubierta cuyo representante en $N(\mathcal{U})$ es el vértice v y U_w es el elemento de la cubierta cuyo representante en $N(\mathcal{U})$ es el vértice w. Si $U_v \cap U_w \neq \emptyset$, entonces los vértices v y w están conectados por una arista y por lo tanto, podemos unir a p y q en $|N(\mathcal{U})|$. Supongamos que $U_v \cap U_w = \emptyset$. Como X es conexo por arcos, existe un mapeo continuo $\gamma : [0,1] \to X$ tal que $\gamma(0) = x$ y $\gamma(1) = y$, además como $\mathcal{U}$ es una cubierta abierta finita de X, la curva γ está a su vez cubierta por elementos de $\mathcal{U}$. Consideremos entonces la secuencia de abiertos

$$\begin{aligned}
[0, \varepsilon_2] &= \gamma^{-1}(U_{\alpha_1}), \text{ con } U_{\alpha_1} = U_v \\
[\varepsilon_1, \varepsilon_4] &= \gamma^{-1}(U_{\alpha_2}), \text{ con } U_{\alpha_1} \cap U_{\alpha_2} \neq \emptyset \\
[\varepsilon_3, \varepsilon_6] &= \gamma^{-1}(U_{\alpha_3}), \text{ con } U_{\alpha_2} \cap U_{\alpha_3} \neq \emptyset \\
&\vdots \\
[\varepsilon_{n-2}, \varepsilon_n] &= \gamma^{-1}(U_{\alpha_m}), \text{ con } U_{\alpha_{m-1}} \cap U_{\alpha_m} \neq \emptyset \text{ y } U_{\alpha_m} = U_w,
\end{aligned}$$

con $0 \leq \varepsilon_1 \leq \varepsilon_2 \leq \ldots \leq \varepsilon_n = 1$. Dicha secuencia existe pues de lo contrario, existiría una parte de $\gamma \subseteq X$ no cubierta. Notemos que cada intersección no vacía en la secuencia descrita representa una arista en $N(\mathcal{U})$, por lo tanto, los vértices v y w pueden ser unidos en $N(\mathcal{U})$ a través de una trayectoria de aristas. Luego, podemos unir a p y q en $|N(\mathcal{U})|$ por lo que $|N(\mathcal{U})|$ es conexo. En conclusión, X y $|N(\mathcal{U})|$ tienen una sola componente conexa, es decir, $H_0(X) = \mathbb{Z}_2 = H_0(|N(\mathcal{U})|)$. □

[1] En este trabajo consideraremos coeficientes en $\mathbb{Z}_2$ en homología.

Teorema 1.11. *Si X y cada elemento de $\mathcal{U}$ es conexo por arcos, entonces $\Phi_{\mathcal{U}}$ induce un epimorfismo en H_1*

$$\Phi_{\mathcal{U}} : H_1(X) \to H_1(|N(\mathcal{U})|).$$

Idea general de la prueba. Como $\Phi_{\mathcal{U}} = \pi \circ \xi$, basta con probar que π es un epimorfismo. Sea $\alpha_1, ..., \alpha_n$ un ciclo en $|N(\mathcal{U})|$. Luego, existen $U_{\alpha_1}, ..., U_{\alpha_n} \in \mathcal{U}$ tales que $U_{\alpha_1} \cap U_{\alpha_2} \neq \emptyset$, $U_{\alpha_2} \cap U_{\alpha_3} \neq \emptyset$, ..., $U_{\alpha_{n-1}} \cap U_{\alpha_n} \neq \emptyset$, $U_{\alpha_n} \cap U_{\alpha_1} \neq \emptyset$. Sean ahora $x_1 \in U_{\alpha_1} \cap U_{\alpha_2}$, $x_2 \in U_{\alpha_2} \cap U_{\alpha_3}$, ..., $x_n \in U_{\alpha_n} \cap U_{\alpha_1}$. Como X es conexo por arcos, definimos las curvas γ_1 que une a x_n con x_1, γ_2 que une a x_1 con x_2, ..., γ_n que une a x_{n-1} con x_n. Consideremos entonces el conjunto de elementos en $\bigsqcup_{\sigma \in N(\mathcal{U})} U_\sigma \times |\sigma|$

1. $x_1 \times |[\alpha_1, \alpha_2]| \in (U_{\alpha_1} \cap U_{\alpha_2}) \times |[\alpha_1, \alpha_2]|$ es un segmento de curva que va de (x_1, α_1) a (x_1, α_2).

2. $\gamma_2 \times \alpha_2 \in U_{\alpha_2} \times \alpha_2$ es un segmento de curva que va de (x_1, α_2) a (x_2, α_2).

3. $x_2 \times |[\alpha_2, \alpha_3]| \in (U_{\alpha_2} \cap U_{\alpha_3}) \times |[\alpha_2, \alpha_3]|$ es un segmento de curva que va de (x_2, α_2) a (x_2, α_3).

4. $\gamma_3 \times \alpha_3 \in U_{\alpha_3} \times \alpha_3$ es un segmento de curva que va de (x_2, α_3) a (x_3, α_3).

   $\vdots$

5. $x_n \times |[\alpha_n, \alpha_1]| \in (U_{\alpha_n} \cap U_{\alpha_1}) \times |[\alpha_n, \alpha_1]|$ es un segmento de curva que va de (x_n, α_n) a (x_n, α_1).

6. $\gamma_1 \times \alpha_1 \in U_{\alpha_1} \times \alpha_1$ es un segmento de curva que va de (x_n, α_1) a (x_1, α_1).

Notemos que, la concatenación de las curvas mencionadas anteriormente describen un ciclo en $\bigsqcup_{\sigma \in N(\mathcal{U})} U_\sigma \times |\sigma|$, por lo que al realizar las identificaciones correspondientes en $X_{\mathcal{U}}$ y evaluar π, obtenemos (módulo fronteras) el ciclo $\alpha_1, ..., \alpha_n$ en $|N(\mathcal{U})|$. En conclusión, $\Phi_{\mathcal{U}}$ induce un epimorfismo en H_1. $\square$

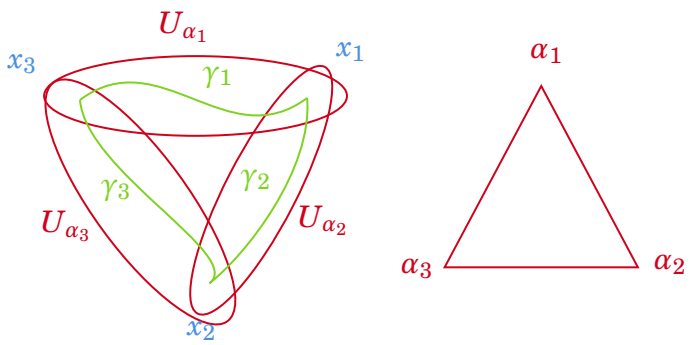


Figura 1.2: Representación gráfica del ciclo descrito en la prueba del teorema 1.11 para $n = 3$, los abiertos U_{α_i} y los γ_i mostrados en la figura, a la izquierda en X y a la derecha en $N(\mathcal{U})$.

En términos de homología y persistencia, que el morfismo del teorema 1.11 sea un epimorfismo tiene como resultado que, al construir el nervio algunas clases de homología pueden morir, sin embargo no es posible que nazcan nuevas clases. Las clases en $H_1(X)$ que sobreviven en la construcción del nervio son las que poseen un mayor tamaño. El tamaño de las clases cuando X es una nube de puntos se explicará más adelante en la sección 1.2.3.

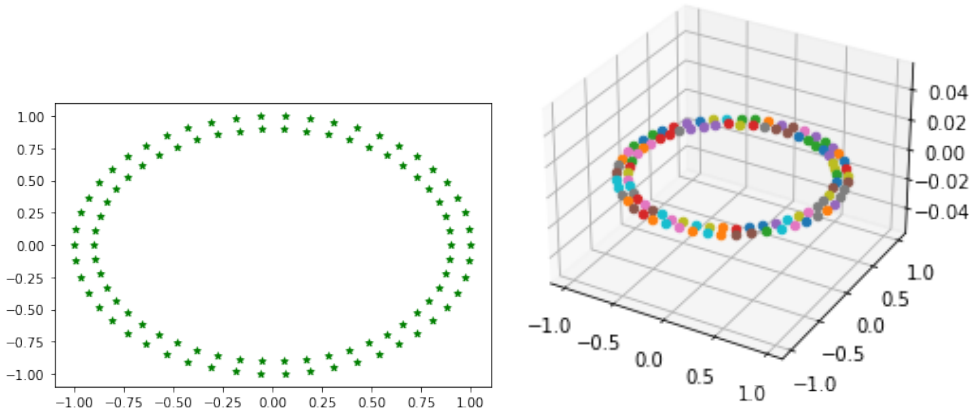
Este resultado se traduce además en una posible pérdida de una parte de la información topológica a la hora de construir el nervio (mapper resultante), por lo que diversos estudios se han llevado a cabo con el fin de optimizar el algoritmo Mapper y así recuperar con cada actualización, la mayor cantidad de información topológica del conjunto de datos original. En la presentación del *Ball Mapper* [27], se buscó implementar un algoritmo que arrojara un mapper resultante similar al que se obtendría con el procedimiento convencional, con información topológica más precisa y con un solo parámetro a escoger. Sin embargo, esto trae como consecuencia un mayor costo computacional (pues al no depender de una función filtro (sección 1.2.1), el algoritmo trabaja en su totalidad sobre el conjunto de datos original), por lo que es esperado que éste mapper resultante contenga más vértices que el convencional, además de ser más difícil de interpretar. Por otro lado, en [28] se buscó optimizar el algoritmo Mapper construyendo un "grafo mapper mejorado" para así retener más información geométrica del conjunto de datos original, cuando la función filtro está definida sobre $\mathbb{R}$.

De un modo más aplicado a los conjuntos de datos, el algoritmo Mapper se ejecuta a partir de una base de datos finita X con una noción de distancia entre los puntos pertenecientes a ésta, donde la métrica escogida para dicha distancia influye en los resultados que se puedan obtener. Sin embargo, tal como lo mencionan los autores al presentar el algoritmo en [1], el algoritmo Mapper es menos sensible a la escogencia de la métrica. Para el presente trabajo, como se trabajarán con bases de datos cuyos elementos pertenecen a $\mathbb{R}^n$, la métrica a utilizar

en todos los casos será la distancia euclidiana. Definimos la función filtro (o lente) $f : X \to \mathbb{R}^n$, donde n es menor a la dimensión de X.

1.2.1 **FUNCIONES FILTRO.** Dependiendo del conjunto de datos se escoge una función filtro adecuada, esto sabiendo en parte qué es lo que se busca al filtrar los datos. El mapper resultante depende en gran medida de la función filtro que se escoja. El objetivo principal de aplicarle un filtro al conjunto de datos X es reducir la dimensión para así simplificar el proceso, pues el algoritmo actúa sobre los datos filtrados (y posteriormente, sobre un subconjunto considerablemente menor en cardinalidad del conjunto de datos original), y reducir el costo computacional. Para conjuntos de datos multidimensionales, una de las opciones consiste en filtrar los datos usando una función de *proyección*.

Sea $f : X \to \mathbb{R}$ con $X \subset \mathbb{R}^n$ y $f(x_1, ..., x_n) = x_i$, $i \in \{1, ..., n\}$ una función de proyección, específicamente, f es una función de proyección a una coordenada. Este tipo de funciones filtro son usualmente escogidas para conjuntos de datos multidimensionales cuyas coordenadas más relevantes guardan información geométrica, como lo son las variedades y las superficies. Por lo tanto, esta función proyección puede resultar efectiva para conjuntos de datos sobre variedades o superficies encajadas en muchas dimensiones como la esfera, el toro, el toro doble, entre otras. Sin embargo, la proyección debe ser definida de la manera más conveniente y dependiendo de la base de datos, es decir, las proyecciones no necesariamente deben ser a la primera coordenada o a una sola coordenada. De igual forma, es importante tener en cuenta cuáles son las coordenadas que preservan información relevante del conjunto de datos, pues puede presentarse el caso de proyectar los puntos de X a una o varias coordenadas donde todos los valores son nulos.



(a) Conjunto de datos.

(b) Conjunto de datos 2-dimensional encajado en $\mathbb{R}^3$.

Figura 1.3: Conjunto de datos sobre una circunferencia conteniendo una circunferencia más pequeña. Si el conjunto de datos en (a) se encaja en más dimensiones (como en (b)), una función de proyección a la primera o segunda componente resulta útil para la construcción del mapper resultante, sin embargo, como las demás componentes tienen valores nulos, una función de proyección a la tercera (o mayor) componente no resultaría útil pues no guarda información geométrica del conjunto de datos.

Otra función filtro 1-dimensional intuitiva es la función filtro *altura*. Como su nombre lo indica, a cada punto de la base de datos le es asignado una altura vertical representada por un número real. La función proyección a una coordenada es en la mayoría de los casos una función altura, como se muestra en la figura 1.4. Así mismo, es el filtro más común a la hora de construir grafos mapper resultantes. En el capítulo 3 se muestran de manera gráfica más ejemplos de funciones altura para un conjunto de datos específico.

Para el caso en el que el conjunto de datos no presente una distribución normal (como ocurre en la mayoría de problemas aplicados en situaciones reales) existen otro tipo de funciones filtro como lo son los filtros de *densidad*, por ejemplo, el kernel Gaussiano. Filtrar los datos con un estimador de densidad ayuda a recuperar nociones de cercanía entre puntos y a preservar características del conjunto de datos, lo que resulta muy útil a la hora de construir el mapper resultante. Es por esto que, incluso en otras funciones filtro usadas para el algoritmo Mapper o para reducción de dimensionalidad de conjuntos de datos, es común utilizar funciones de densidad, como es el caso de [28], [8] o del filtro del Laplaciano explicado en el capítulo 3. Para un conjunto de datos de n dimensiones, la función filtro usando el kernel

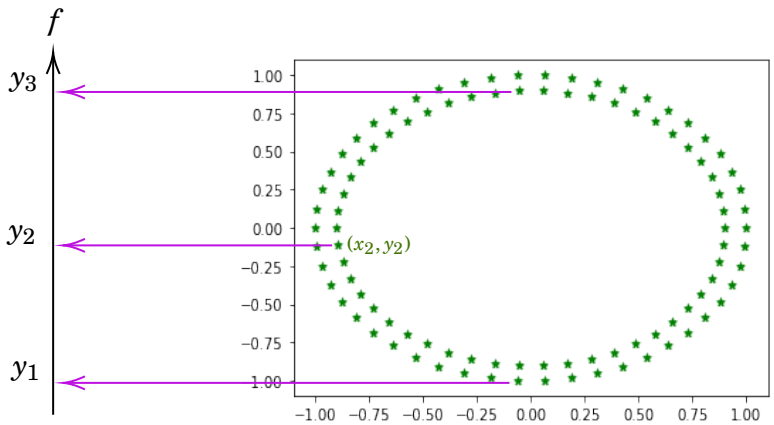


Figura 1.4: Conjunto de datos de la figura 1.3 aplicando una función filtro de altura proyectando a la segunda componente. La figura muestra tres asignaciones de la función altura f para tres puntos (x_1, y_1), (x_2, y_2) y (x_3, y_3) del conjunto de datos, con $y_1 \leq y_2 \leq y_3$.

Gaussiano se ve como

$$f(x) = \frac{1}{(\sigma\sqrt{2\pi})^n} \sum_{y \in X} e^{-\frac{d(x,y)^2}{2\sigma^2}},$$

donde X es el conjunto de datos, $x, y \in X$ y $\sigma > 0$ es el ancho del kernel Gaussiano, donde a mayor valor de σ, mayor anchura (horizontal) tendrá la gráfica de f. Además, esta función de densidad está normalizada de tal forma que se cumpla que $\int_{-\infty}^{\infty} f(x) = 1$ para cualquier σ escogido. Estas y más propiedades del kernel Gaussiano se encuentran detalladas en [29] y [7]. Para el presente trabajo, el kernel Gaussiano fue utilizado e implementado en la aplicación del algoritmo Mapper para la base de datos de la diabetes, la cual será explicada en el capítulo 2 y además en los experimentos de la aplicación del algoritmo Mapper a la base de datos de la calidad del aire en el capítulo 4.

Otra función filtro que ha arrojado resultados útiles para reducir dimensionalidad y complejidad en el análisis de conjuntos de datos es el filtro que utiliza la matriz *Laplaciana*. La matriz Laplaciana es construida a partir de valores que midan similitud entre los puntos, cuyos vectores propios contienen información geométrica de la base de datos y por lo tanto son usados como funciones filtro 1-dimensionales o multidimensionales. Adicionalmente, diferentes autores que emplearon la matriz laplaciana para el análisis de conjuntos de datos propusieron métodos que fueran robustos al ruido y además, brindaran resultados eficientes para una posterior agrupación de los datos, como ocurre en [8], [30] y [31] donde incluso se muestran métodos en el cálculo de la matriz laplaciana de un grafo, similar al método utilizado para los experimentos del capítulo 3. El filtro del Laplaciano implementado en el

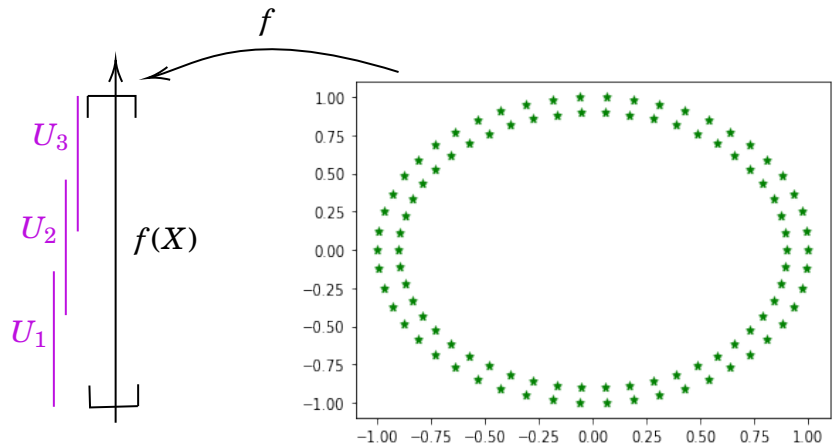


Figura 1.5: Conjunto de datos de la figura 1.3 aplicando la función filtro descrita en la figura 1.4. El rango de la función filtro $f(X)$ es cubierto por los intervalos abiertos U_1, U_2 y U_3. Un traslape de más del 50% entre los elementos de la cubierta generaría una intersección triple entre U_1,U_2 y U_3.

presente trabajo es explicado de manera más detallada en el capítulo mencionado.

Existen más funciones filtro utilizadas para la aplicación del algoritmo Mapper como la función de excentricidad (mencionada y explicada en [1] y [4]) o la profundidad de datos (mencionada y explicada en [17]). Así mismo, dependiendo del conjunto de datos se puede crear una función filtro conveniente, una combinación de varias funciones filtro (como ocurre en [32] para la eliminación de ruido en conjuntos de datos) o no usar ninguna, como ocurre en [12] donde la función filtro es la función identidad. [1], [16] y [11] muestran más ejemplos de funciones filtro aplicadas a diferentes conjuntos de datos para la aplicación del algoritmo Mapper.

1.2.2 **CUBIERTA.** Una vez es aplicada la función filtro f, el rango $f(X)$ es cubierto dependiendo de su dimensión y de los resultados que se desean obtener. La dimensión y organización de los simplejos que conformarán el mapper resultante dependen de la cubierta, que consiste en una cantidad finita de subconjuntos de $f(X)$ que se traslapan. Por lo tanto, es de alta importancia escoger una cubierta adecuada en la aplicación del algoritmo Mapper. Una cubierta abierta se define específicamente como el conjunto $\{U_\alpha\}_{\alpha\in A}$, con A un conjunto finito de índices, cada $U_\alpha \subset f(X)$ y donde se cumple que $f(X) \subseteq \bigcup_{\alpha\in A} U_\alpha$. Para el caso en el que $f(X) \subset \mathbb{R}$, los elementos de la cubierta son entonces intervalos abiertos, como se muestra en la figura 1.5.

Cualquier cubierta abierta de $f(X)$ puede ser usada. Sin embargo, las intersecciones entre los elementos de la cubierta determinarán la dimensión de los simplejos al construir el nervio (mapper resultante). Cada elemento de la cubierta generará un vértice o un conjunto de vértices (simplejos de dimensión 0), las intersecciones dobles generarán aristas (simplejos de dimensión 1), las intersecciones triples generarán triágulos rellenos (simplejps de dimensión 2), y así sucesivamente. Singh, Mémoli y Carlsson [1] mencionan que si el filtro es 1-dimensional, entonces el mapper resultante estará conformado por simplejos de dimensión a lo más 1, es decir, nodos y aristas (obteniendo así un grafo mapper resultante). Por lo tanto, si $f(X) \subset \mathbb{R}$, entonces los elementos de la cubierta (intervalos abiertos) deben tener un traslape menor al 50 %, de esta forma se forman máximo intersecciones dobles, tal como se muestra en el ejemplo de la figura 1.5. En el presente trabajo se describen dos cubiertas 1-dimensionales en los experimentos de la aplicación del algoritmo Mapper a la base de datos de la diabetes (capítulo 2) y a la base de datos de la calidad del aire (capítulo 4).

Para el caso en el que $f(X) \subset \mathbb{R}^n$ con $n > 1$, los elementos de la cubierta pueden tener la forma que resulte más conveniente. Algunas librerías de Python utilizan una cantidad m de n-cubos con cierto porcentaje de traslape para cubrir el rango de la función filtro, como es el caso de la cubierta implementada en `Keppler Mapper` [18], donde además se especifica que dicha cubierta es creada con a lo más intersecciones dobles entre sus elementos [33]. Sin embargo, naturalmente una cubierta con n-cubos puede presentar intersecciones mayores. Para el caso de los rectángulos, por ejemplo, la ubicación de estos en el rango del filtro puede generar intersecciones cuádruples, dando lugar a la posibilidad de obtener tetraedros en el mapper resultante. Por lo tanto, si se desea tener un límite en las dimensiones de los simplejos que conforman el mapper resultante, es importante entonces la implementación de una cubierta que evite las intersecciones múltiples que generan simplejos de dimensión mayor a la deseada. Un ejemplo de cubierta multidimensional se muestra en los experimentos de la aplicación del algoritmo Mapper a la base de datos del toro (capítulo 3), donde se implementa una cubierta para el caso en el que $f(X) \subset \mathbb{R}^2$.

Adicionalmente, es importante determinar el tamaño de los elementos de la cubierta, pues tamaños muy pequeños pueden significar un mapper resultante muy denso (con muchos nodos), lo que se traduce en un costo computacional mayor o en disconexidad en regiones que se esperan ser conexas. Por otro lado, tamaños muy grandes pueden implicar la pérdida de características topológicas relevantes del conjunto de datos X en la construcción del mapper resultante, como se explica más adelante en la sección 1.2.3 y se ilustra en la figura 1.8. Existen librerías en Python como `giotto-tda` [19] que implementan cubiertas 1-

dimensionales cuyos intervalos tienen un tamaño conveniente, de tal forma que las imágenes inversas de estos contengan una cantidad similar de puntos en X.

Debido a la alta sensibilidad del algoritmo Mapper a la escogencia de la cubierta, existen estudios que buscan automatizar el proceso de búsqueda de la cubierta más conveniente, como ocurre en [16]. Adicionalmente, existe una versión *multiescala* del algoritmo Mapper que adopta la posibilidad de usar un conjunto de cubiertas de $f(X)$ a través de mapeos entre éstas, de esta forma no se trabaja con una cubierta fija. En [1], [27], [17] y el capítulo 9 de [5] se aborda de manera adecuada la versión multiescala del algoritmo Mapper. Para el presente trabajo sólo se realizarán experimentos adoptando una cubierta fija del rango de la función filtro.

1.2.3 **AGRUPACION, MAPPER Y VISUALIZACION.** El siguiente paso en la construcción del mapper resultante consiste en, a partir de la cubierta de $f(X)$, obtener una cubierta del conjunto de datos X. Naturalmente, si $\mathcal{U} = \{U_\alpha\}_{\alpha \in A}$ es una cubierta de $f(X)$, entonces realizando un *pullback* de $\mathcal{U}$ a través de f, se obtiene una cubierta $\{f^{-1}(U_\alpha)\}_{\alpha \in A}$ de X. Sin embargo, esta cubierta debe ser refinada de tal forma que cada $f^{-1}(U_\alpha)$ sea descrito como una unión de sus componentes conexas. Como X es un conjunto discreto de datos, la noción de conexidad en subconjuntos de X está dada mediante procesos de *agrupación*.

El proceso de agrupación consiste en particionar el conjunto de datos X en agrupaciones (clusters) que se diferencian entre sí, es decir, los clusters son áreas de alta densidad de datos, donde la densidad se mide a partir de los parámetros escogidos y de cómo los elementos en un subconjunto de puntos de X están más próximos entre sí con respecto a los puntos en un cluster diferente. Múltiples algoritmos de agrupación han sido utilizados en el análisis topológico de datos, pues en conjunto con los procedimientos que permiten la reducción de dimensionalidad proporcionan una visualización del comportamiento de los datos. El algoritmo Mapper realiza este proceso al aplicar la función filtro (reducción de dimensionalidad) y posterior agrupación de los elementos de la cubierta de X. Singh, Mémoli y Carlsson [1] mencionan que el Mapper no depende de un método de agrupación específico, por lo tanto, cualquier algoritmo para agrupación de datos puede ser utilizado.

Uno de los algoritmos de agrupación más comunes es el DBSCAN [2] [34], un algoritmo de agrupación espacial basado en la densidad que detecta áreas de alta densidad en un conjunto de datos y las diferencia entre sí. El algoritmo DBSCAN recibe dos parámetros principales:

[2] `https://scikit-learn.org/stable/modules/generated/sklearn.cluster.DBSCAN.html`

`eps` y `min_samples`, y además se basa en la clasificación de los puntos en el conjunto de datos como “core samples”, “non-core samples” y valores atípicos (outliers) (figura 1.6). Un elemento de X es un core sample si está en un área de alta densidad, es decir, si existen mínimo `min_samples` elementos a una distancia menor a `eps` de éste. Cada vecino del core sample es entonces clasificado como core sample nuevamente si cumple con la condición, de lo contrario, será clasificado como un non-core sample. Una vez el proceso de clasificación termina (es decir, no se encontraron más core samples en los vecinos) se obtiene uno de los clusters del conjunto de datos. Cada elemento que no es un core sample y se encuentra a una distancia mayor o igual a `eps` de un core sample, es considerado un valor atípico. Por lo tanto, escoger adecuadamente estos dos parámetros es crucial para realizar una buena agrupación del conjunto de datos X, es por esto que los autores en [35] proponen un método para escoger un valor de `eps` conveniente fijando `min_samples` = 4 para conjuntos de datos 2-dimensionales. Para los experimentos en [12] y [16] de la aplicación del algoritmo Mapper a diferentes conjuntos de datos se utilizó al algoritmo DBSCAN para sus procesos de agrupación.

La librería `sklearn` de Python [22] posee una implementación del algoritmo DBSCAN donde además permite en su conjunto de parámetros la métrica que se desee usar para medir las distancias entre los puntos o el algoritmo para hallar los vecinos durante el proceso. Este método de `sklearn.cluster` en Python [36] fue usado como proceso de agrupación en los experimentos del capítulo 4.

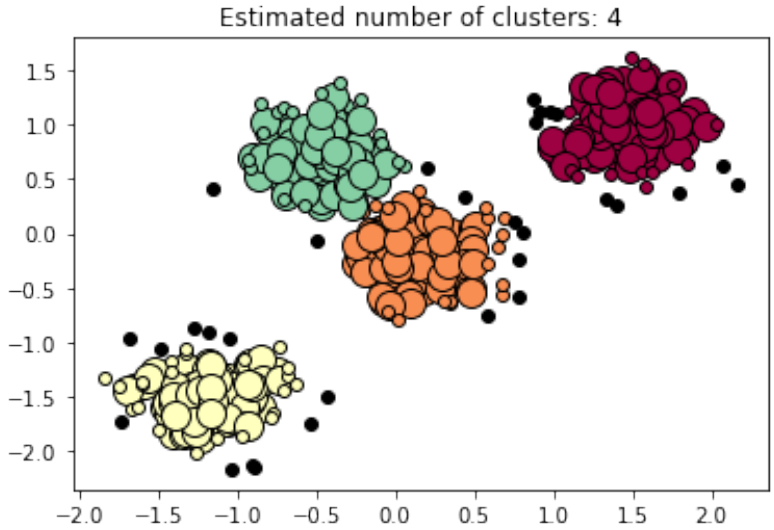
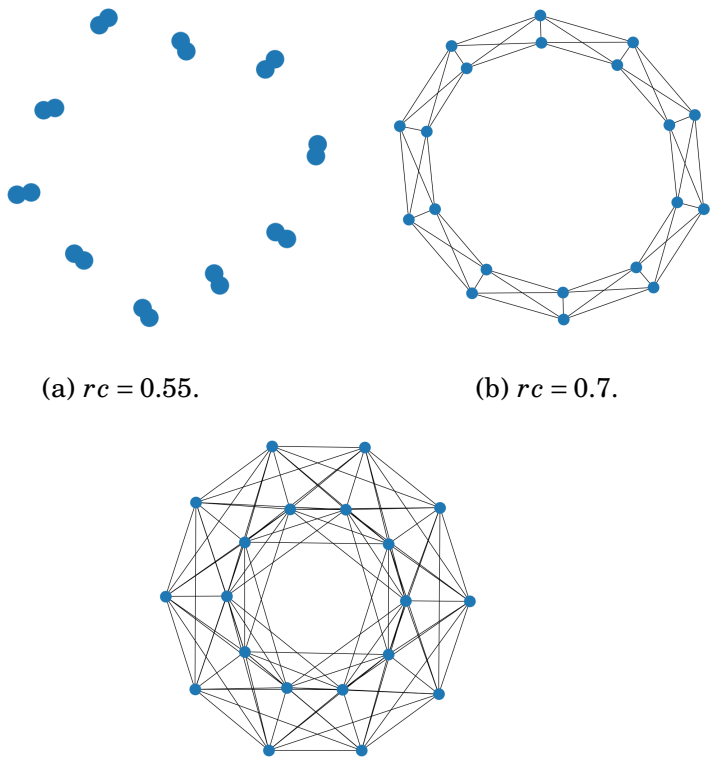


Figura 1.6: Agrupación realizada por el algoritmo DBSCAN usando la librería de Python `sklearn` para un conjunto de datos 2-dimensional de 500 elementos, con `eps` = 0.2 y `min_samples` = 9. Los círculos más grandes representan los core samples, los círculos más pequeños del color de cada cluster representan los non-core samples y los círculos negros representan los puntos aislados. Para una menor cantidad de puntos aislados es recomendable aumentar el valor de `eps` o disminuir el valor de `min_samples`. Sin embargo, como se observa en la figura, un valor mayor para `eps` o menor para `min_samples` puede significar la unión de los clusters verde y naranja como uno solo.

Otro método de agrupación muy común es la agrupación de enlace simple (*single linkage clustering*). Este algoritmo consiste en la construcción del 1-esqueleto del complejo de Rips (definición 1.3), es decir, el complejo de Rips sin incluir los simplejos de dimensión mayor a 1. Fijando un parámetro $rc > 0$, se construye un grafo cuyos vértices son los elementos del conjunto de datos X y donde se añade una arista que une a $x, y \in X$ si se cumple que $d(x, y) \leq rc$, como se observa en la figura 1.7. De esta forma, al realizar el proceso obteniendo un grafo G, para cada $f^{-1}(U_\alpha)$ en $\{f^{-1}(U_\alpha)\}_{\alpha \in A}$ consideramos el subgrafo H de G generado por los elementos de $f^{-1}(U_\alpha)$ y definimos a $\overline{f^{-1}(U_\alpha)} = \bigcup_i H_i$, donde cada H_i es una componente conexa de H. En este caso, cada cluster en X es representado por cada H_i. Notemos que como $X \subset \mathbb{R}^n$ es finito, se cumple que cada $\overline{f^{-1}(U_\alpha)}$ contiene una cantidad finita de componentes conexas. Así, obtenemos la cubierta $\overline{\mathscr{V}} = \left\{\overline{f^{-1}(U_\alpha)}\right\}_{\alpha \in A} = \{H_i\}_i$ de X. En el presente trabajo, este proceso de agrupación se implementa para los experimentos de los capítulos 2 y 3. Más ejemplos del proceso de agrupación de enlace simple en la aplicación del algoritmo Mapper se encuentran en [1] y [37].



(a) $rc = 0.55$. (b) $rc = 0.7$.

(c) $rc = 1.5$.

Figura 1.7: Agrupación obtenida para un conjunto de datos de 20 elementos con tres valores escogidos para el parámetro rc.

De esta forma, consideramos la cubierta refinada de X

$$\overline{\mathscr{V}} = \left\{\overline{f^{-1}(U_\alpha)}\right\}_{\alpha \in A} = \{C_\beta\}_{\beta \in B},$$

donde cada C_β es un cluster obtenido al aplicar el algoritmo de agrupación escogido y B es un conjunto finito de índices. El tamaño de cada cluster es determinado dependiendo de la cantidad de elementos de X que contenga, los clusters más grandes son los clusters que contienen una cantidad mayor de elementos en X con respecto a los demás.

El mapper resultante se construye entonces de la siguiente manera: dada la cubierta refinada $\overline{\mathscr{V}}$ de X, cada cluster es representado por un vértice y cada simplejo de dimensión $d > 0$ en el mapper resultante es construido e incluido si se cumple que, para $d+1$ clusters de elementos de X, $C_{\beta_1} \cap \ldots \cap C_{\beta_{d+1}} \neq \emptyset$ con $\{\beta_1, \ldots, \beta_{d+1}\} \subset B$. De esta forma, tal como se muestra en la figura 1.8 para tres ejemplos sobre el mismo conjunto de datos, se obtiene el

mapper resultante, el cual recupera información topológica del conjunto de datos escogiendo los parámetros convenientes.

Para un conjunto de datos de 20 elementos (figura 1.3 con $n = 20$) y una aplicación del filtro de proyección a la segunda componente, resta escoger la cubierta (número de intervalos y porcentaje de traslape) representada por los intervalos de la izquierda en la figura 1.8, y los parámetros del proceso de agrupación. En este caso, se escogió el proceso de agrupación de enlace simple, donde el subgrafo generado por los datos pertenecientes a la preimagen de los elementos de la cubierta (los cuales tienen un color específico) están señalados con el color correspondiente. En 1.8 (c), debido al rc escogido el mapper resultante (parte derecha en la figura) contiene regiones disconexas no deseables, pues el conjunto de datos presenta un comportamiento similar al de una circunferencia, es decir, el mapper resultante debe ser conexo y con un "orificio" (ciclo) para así recuperar las características topológicas de ésta. En 1.8 (b), con la misma cubierta escogida en (c) el parámetro rc es modificado, obteniendo un mapper resultante conveniente, pues cumple con las características topológicas mencionadas. Sin embargo, un parámetro rc conveniente no es suficiente con una cubierta que no lo es, en 1.8 (a) el parámetro de agrupación es el mismo que en (b), sin embargo el tamaño de los 3 intervalos en la cubierta es demasiado grande con intersecciones vacías entre éstos (resultando nuevamente en disconexidad en el mapper resultante) para cualquier traslape que se escoja menor al 50%.

Notemos que, el proceso de agrupación por sí solo proporciona una visualización del comportamiento del conjunto de datos, utilizado en múltiples estudios anteriores y posteriores al Mapper con este fin. En [1], [17] y [11] se describen diferentes comparaciones de los resultados obtenidos al aplicar el algoritmo Mapper vs. un proceso de reducción de dimensionalidad y agrupación escogido en cada caso para el mismo conjunto de datos estudiado. Así mismo, en el capítulo 2 se muestra explícitamente una de estas comparaciones.

Como se observó a lo largo del capítulo, el algoritmo Mapper requiere de un conjunto de parámetros para su implementación. Diferentes parámetros producen diferentes mappers resultantes. Por lo tanto, la tupla de parámetros "correcta" es entonces la que se aproxima más al conjunto de datos original en su geometría o la que realiza una mejor clasificación de los datos. Sin embargo, esto puede implicar una vasta cantidad de pruebas en la búsqueda de los parámetros que muestren los resultados esperados y que además muestren estabilidad (mínimas o nulas perturbaciones con respecto a parámetros cercanos) en su mapper resultante. Es por esto que, diferentes autores han realizado estudios que buscan proponer

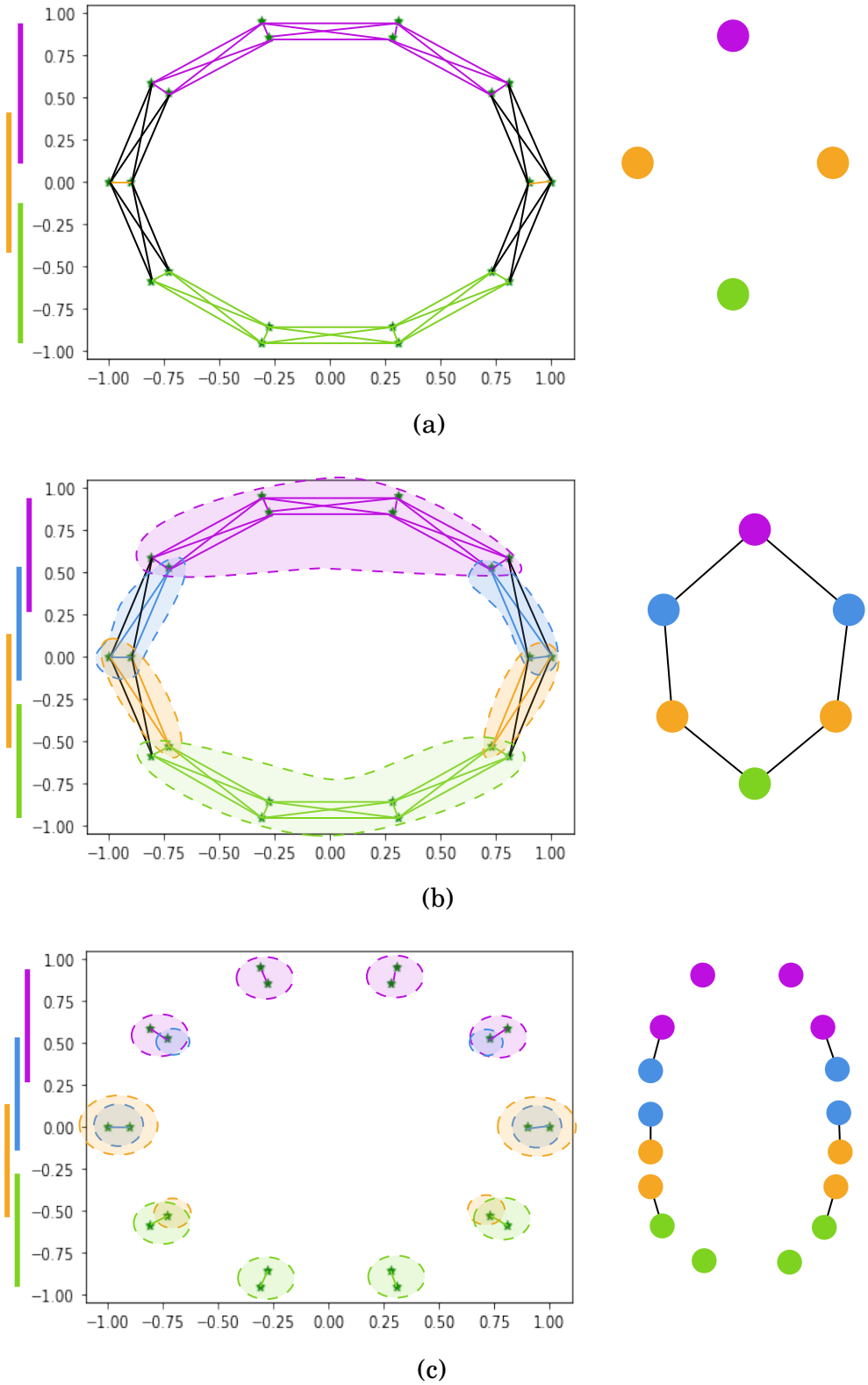


Figura 1.8: Construcción del mapper resultante para un conjunto de datos de 20 elementos, aplicando el filtro de proyección a la segunda componente y realizando el algoritmo de agrupación de enlace simple como proceso de agrupación de los datos. (a) $rc = 0.7$ y cubierta de 3 intervalos con 40 % de traslape. (b) $rc = 0.7$ y cubierta de 4 intervalos con 40 % de traslape. (c) $rc = 0.55$ y cubierta de 4 intervalos con 40 % de traslape.

métodos para una óptima selección de parámetros. En [37], se introduce un método estadístico para una aproximación en la selección de los parámetros necesarios para el algoritmo Mapper cuando la función filtro toma valores en $\mathbb{R}$ (es decir, cuando el mapper resultante es un grafo). Dicho método se aplica bajo consideraciones iniciales en el conjunto de datos X y se basa en la prueba de que el grafo mapper es una aproximación discreta del grafo de Reeb ([28], [37] y el capítulo 7 de [5] proporcionan un tratamiento adecuado sobre los grafos de Reeb), por lo que los autores verifican su método a través de la convergencia del grafo mapper al grafo de Reeb comparándolos con los diagramas de persistencia extendida y la distancia de cuello de botella (bottleneck distance). Este tópico fue luego abordado nuevamente por los autores para el caso de filtros multidimensionales en [38]. A su vez en [28], cuando los autores introdujeron el "grafo mapper mejorado" analizaron también la estabilidad y convergencia con los grafos de Reeb usando tópicos en teoría de Haces (cosheaf theory) y la distancia de intercalación (interleaving distance) respectivamente, descartando las condiciones iniciales requeridas en [37] para el conjunto de datos X. Por otro lado, Chalapathi, Zhou y Wang [16] también se interesaron en desarrollar una selección de parámetros automática introduciendo un procedimiento estadístico aplicando teoría de la información (information theory) para la selección del parámetro l, el cual representa el número de elementos de la cubierta en la imagen de la función filtro, específicamente para el caso en que la función filtro toma valores reales, es decir, l representa el número de intervalos que cubren a $f(X)$ asumiendo los demás parámetros como fijos. Dicho método propuesto adapta una parte del proceso de agrupación $X-$means [16], generando así cubiertas adaptativas para el algoritmo Mapper partiendo desde un l inicial, sin requerir alguna condición específica para el conjunto de datos X. Además, los autores realizaron diferentes pruebas comparando su método con el descrito en [37], obteniendo mejores resultados (en términos de geometría y clasificación de los datos) para diversos conjuntos de datos, por lo que, a partir de un grafo mapper inicial, se obtiene un mapper refinado que recupera de manera más precisa la forma del conjunto de datos original.

Los siguientes capítulos del presente trabajo muestran tres aplicaciones del algoritmo Mapper a tres problemas específicos de análisis topológico de conjuntos de datos.

2

Aplicación del algoritmo Mapper al estudio de la diabetes

En este capítulo se sigue (no de manera exacta) el proceso utilizado por Singh, Mémoli y Carlsson [1] para la aplicación del algoritmo Mapper a la base de datos del estudio de la diabetes realizado por Miller y Raven [39].

La base de datos 5-dimensional que aparece en [40] fue tomada por el Centro de Investigación Clínica de Stanford y estudiada por Miller y Raven [39] para analizar, mediante técnicas de reducción de dimensionalidad y visualización de los datos utilizando el método de búsqueda de la proyección (projection pursuit method) [6], el estado de salud de 145 individuos entorno a la diabetes, teniendo tres clasificaciones posibles: normal, chemical diabetic y over diabetic. Dichas clasificaciones se tomaron en base a los criterios médicos de la época y fueron agregadas como una sexta columna a modo de verificación y comparación con los resultados obtenidos por el estudio. Las variables (columnas en la base de datos) tenidas en cuenta para el estudio son: peso relativo, glucosa plasmática, área bajo la curva de la glucosa plasmática obtenida por un test de tolerancia de la glucosa por tres horas, área bajo la curva de la insulina obtenida por el mismo test y glucosa plasmática en estado de reposo (SSPG). De este modo, una vez el método fue aplicado, se lograron visualizar las tres clasificaciones en las agrupaciones como “un núcleo y dos alas” (figura 2.1) donde el núcleo contenía en su mayoría individuos “normales” y las alas clasificaban a los individuos según el tipo de diabetes que presentaban. Singh, Mémoli y Carlsson usaron entonces esta base de datos para aplicar el algoritmo Mapper y verificar si el mapper resultante lucía similar a los resultados obtenidos por Miller y Raven y además si éste realizaba la clasificación esperada de los da-

tos. En el presente capítulo se detallan los experimentos realizados al aplicar el algoritmo Mapper a la base de datos mencionada siguiendo el procedimiento que los autores presentan en [1].

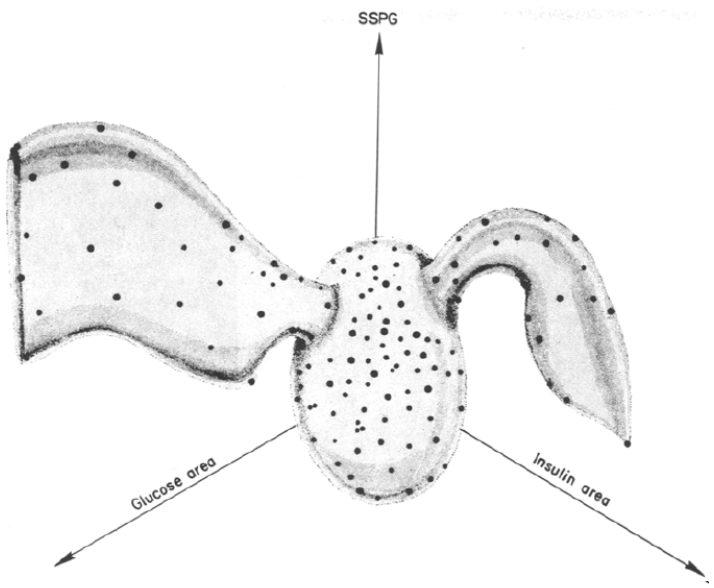


Figura 2.1: Visualización de los datos obtenida con el método aplicado por Miller y Raven en [39] para la base de datos de la diabetes. La terna de variables escogida se muestra como un sistema de coordenadas en la figura.

Una ventaja del algoritmo Mapper sobre el método utilizado por Miller y Raven es el manejo del conjunto de datos. Mientras que en el método que arroja los resultados mostrados en la figura 2.1 se escoge una terna de variables en la base de datos (de las posibles combinaciones de ternas que se puedan obtener), realizando un posterior análisis de las relaciones entre dichas variables escogidas, el algoritmo Mapper automatiza este proceso empleando la base de datos completa y obteniendo resultados similares. Por lo tanto, el conjunto de datos a utilizar para la construcción del mapper resultante consta de un arreglo de tamaño 145×6 guardado en un archivo .csv.

2.1 Normalización de los datos

Como la base de datos presenta valores con rangos ampliamente diferentes (como por ejemplo el peso relativo, que en la base de datos toma valores no mayores a 2 y, por el contrario, el área bajo la curva de la glucosa puede tomar valores mayores a 1000), se realizó como primer paso un proceso de normalización de los datos usando la librería `sklearn`, más específicamente el método `sklearn.preprocessing.MinMaxScaler` [41] con los parámetros que el método trae por defecto, es decir, los valores normalizados obtenidos están entre 0 y 1. El proceso de normalización específico que realiza `MinMaxScaler` es el siguiente:

```
    X_std = (X-Xmin(axis=0))/(X.max(axis=0)-X.min(axis=0))
X_scaled = X_std*(max-min)+min
```

donde `X` es la base de datos, los arreglos `X.min` y `X.max` guardan los mínimos y máximos valores de cada columna de la base de datos respectivamente y los valores `max` y `min` son las cotas del rango en el que se desea normalizar los datos (en este caso, `max=1` y `min=0`). Todas las operaciones se realizan componente a componente. De esta forma, se obtiene un arreglo de tamaño 145×6 con los datos normalizados.

Este no es el único proceso de normalización del conjunto de datos posible para la aplicación del algoritmo Mapper. Existen múltiples métodos y procedimientos como por ejemplo el utilizado en [12] para el conjunto de datos de la propagación del coronavirus. En dichos experimentos también se aplicó un proceso de normalización debido a los diferentes sistemas de medida en los datos recolectados, pero aplicando un procedimiento diferente que consistió en dividir cada componente del vector (fila) entre la norma del vector columna al que pertenece. Por otro lado, en [11] el conjunto de datos también fue modificado inicialmente, pero con un proceso más conveniente para el problema específico que fue estudiado. Al trabajar con un conjunto de datos cuyos elementos contienen información de tejidos enfermos, los autores realizaron una transformación donde cada dato se podía escribir como la suma de dos términos. De esta forma, tomaron sólo uno de los términos para construir una nueva base de datos y realizar los experimentos deseados. Así mismo, este proceso de normalización se adopta en los experimentos realizados para las bases de datos de la calidad del aire en el capítulo 4.

2.2 Filtro: Estimación de densidad usando un kernel Gaussiano

Definimos la función filtro de densidad $f : X \subset \mathbb{R}^5 \to \mathbb{R}$ con un kernel Gaussiano normalizado como

$$f(x) = \frac{1}{n(\sigma\sqrt{2\pi})^d} \sum_{y \in X} e^{-\frac{d(x,y)^2}{2\sigma^2}}, \tag{2.1}$$

donde $x, y \in X$, $n = 145$, $d = 5$ y $d(x,y)$ es la distancia euclidiana. Así, se obtiene un arreglo de tamaño 145×1 con los datos filtrados. Se escogieron diferentes valores de σ en los experimentos, mostrados en la sección 2.4.

2.3 Cubierta en $\mathbb{R}$

Definimos una cubierta en el rango de la función filtro que consta de 4 intervalos con 49.9% de traslape. La cubierta elegida por Singh, Mémoli y Carlsson en [1] para obtener una "alta resolución" en el mapper resultante consta de 4 intervalos con 50% de traslape, sin embargo, a manera de evitar intersecciones triples, en los experimentos presentados en el presente capítulo se tomó el traslape indicado inicialmente. De esta misma forma, consideramos una cubierta en el rango del filtro que consta de 3 intervalos con 49.99% de traslape para obtener el mapper resultante de "baja resolución".

La implementación de la cubierta se realiza en base a los siguientes parámetros y cálculos: sean min y max el mínimo y máximo de los datos filtrados respectivamente, n el número de intervalos en la cubierta y p el porcentaje de traslape. Calculamos el tamaño del intervalo l como

$$l = \frac{max - min}{n - (n-1)p}$$

y calculamos las cotas de cada intervalo $[a_i, b_i]$, $i \in \{0,1,2,3\}$ en la cubierta como

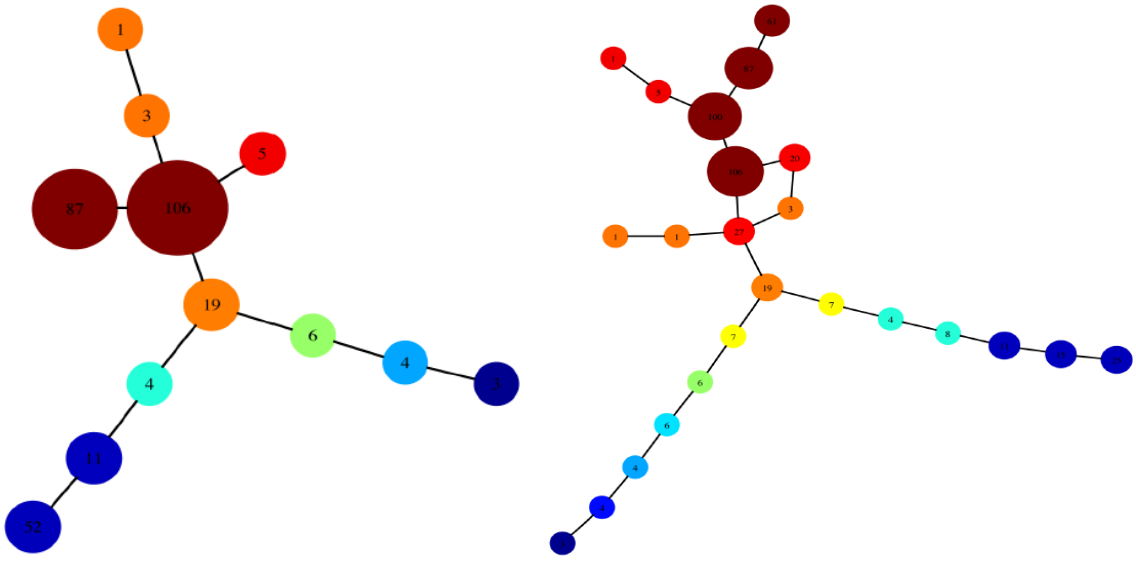
$$a_i = il(1-p) + min$$

$$b_i = a_i + l.$$

De esta forma, obtenemos la cubierta con el número de intervalos y porcentaje de traslape deseado.

Este problema ha sido abordado y citado en múltiples referencias, pues fue uno de los experimentos con los que el algoritmo Mapper se presentó por los autores en [1]. En [37], por ejemplo, los autores ejecutan el ajuste automático de parámetros propuesto a la aplicación

del algoritmo Mapper a la base de datos de la diabetes, obteniendo resultados satisfactorios. Los mapper resultantes obtenidos por Singh, Mémoli y Carlsson se muestran en la figura 2.2.



(a) (b)

Figura 2.2: Mapper obtenidos en [1] de aplicar el algoritmo Mapper a la base de datos de la diabetes.(a) Mapper resultante de tomar una cubierta de tres intervalos con 50 % de traslape. (b) Mapper resultante de tomar una cubierta de cuatro intervalos con 50 % de traslape.

Las “alas” que mencionan Miller y Reaven en [39] se ven representadas en la parte inferior de cada mapper resultante en la figura 2.2 y éstas agrupan y clasifican los individuos según el tipo de diabetes que presentan. El “núcleo” se ve representado por la parte superior de cada mapper resultante, la cual agrupa individuos normales o cercanos a esta clasificación. Por otro lado, los colores de los nodos fueron implementados de acuerdo a los valores de densidad, donde el rojo representa alta densidad y el azul baja densidad. Por último, los nodos con mayor tamaño representan agrupaciones con mayor cantidad de datos.

Los autores en [1] no especifican los parámetros utilizados para el algoritmo Mapper aparte de los necesarios para la implementación de la cubierta. Los experimentos en el presente trabajo para el problema descrito, fueron efectuados utilizando diferentes parámetros hasta encontrar los que realizan una mejor clasificación de los datos y arrojan mappers resultantes similares a los obtenidos por los autores en [1]. Nuestros resultados se muestran a continuación.

2.4 Experimentos

Para los experimentos que se van a describir a continuación, se realizaron distintas pruebas hasta encontrar el conjunto de parámetros adecuados que mostró los resultados deseados. El algoritmo Mapper utilizado para los siguientes experimentos fue programado usando diferentes librerías de `Python` que resultaron útiles en el proceso de normalización, filtro, cubierta y la construcción del mapper resultante, el cual se encuentra en el archivo `Diabetes.py` de la carpeta `Diabetes` en `https://github.com/kbotello129/Mapper`.

2.4.1 **LOW RESOLUTION.** Para el primer experimento realizado de la aplicación del algoritmo Mapper a la base de datos de la diabetes, tomamos el conjunto de datos normalizado tal como se describe en la sección 2.1 y aplicamos el filtro descrito en la sección 2.2 con $\sigma = 0.57$. Además, consideramos una cubierta de $n = 3$ intervalos con $p = 0.4999$ de traslape en el rango del filtro y un radio máximo para el proceso de agrupación de enlace simple de $rc = 0.55$. De esta forma, al aplicar el algoritmo se obtuvo un mapper resultante conexo de 5 nodos y 4 aristas, el cual se muestra en la figura 2.3

Recordemos que cada nodo en el mapper resultante representa una agrupación de datos, por lo que nodos más grandes significan una agrupación con mayor cantidad de elementos del conjunto de datos. Para los experimentos realizados, los colores en los nodos del mapper resultante fueron implementados teniendo en cuenta la sexta columna en el conjunto de datos, es decir la clasificación clínica, donde normal (o tipo 3) se representa con el color rojo, over diabetic (o tipo 1) se representa con el color azul y chemical diabetic (o tipo 2) se representa con el color gris. De esta forma, el mapper resultante mostrado en la figura 2.3 realiza una clasificación similar a la obtenida por Singh, Mémoli y Carlssson (figura 2.2 (a)). El reporte de clasificación del mapper resultante se muestra en la figura 2.4.

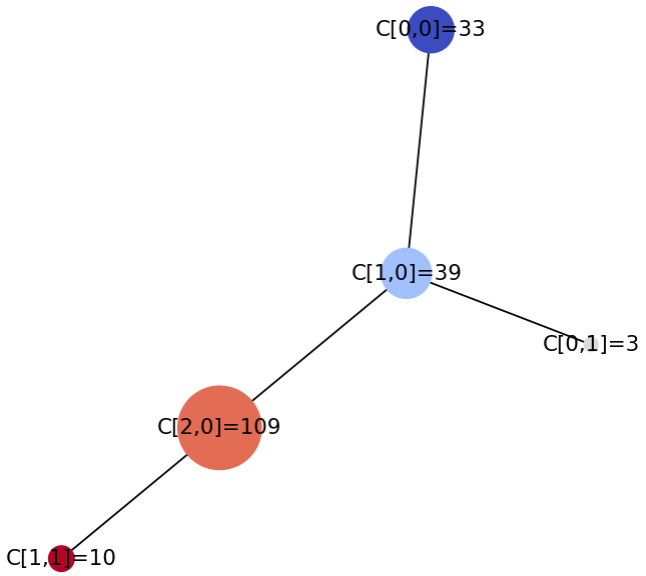


Figura 2.3: Grafo mapper resultante de aplicar el algoritmo Mapper a la base de datos normalizada de la diabetes, usando el filtro del Kernel Gaussiano y una cubierta de 3 intervalos con 49.99% de traslape. Las etiquetas representan (de izquierda a derecha) el nombre de cada nodo, su componente conexa y la cantidad de elementos en el cluster que representan. Este mapper está disponible en el archivo `low_re.png` en el repositorio mencionado al inicio de esta sección.

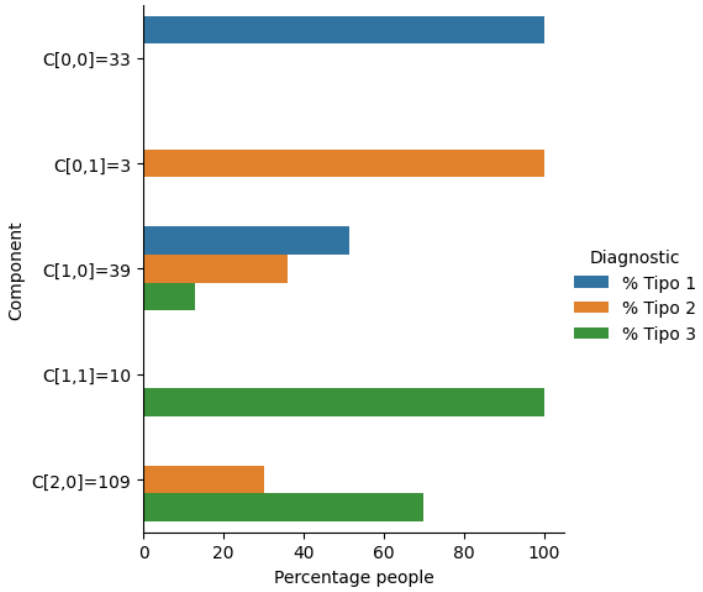


Figura 2.4: Reporte de clasificación del mapper resultante mostrado en la figura 2.3. Para el caso del nodo $C[0,0]$, por ejemplo, el 100% de los elementos del conjunto de datos pertenecientes a este cluster son de individuos cuya clasificación clínica es over diabetic (tipo 1). Este reporte está disponible en el archivo `low_re_report.png` en el repositorio mencionado al inicio de esta sección.

2.4.2 **HIGH RESOLUTION.** Para el siguiente experimento realizado de la aplicación del algoritmo Mapper a la base de datos de la diabetes, tomamos nuevamente el conjunto de datos normalizado tal como se describe en la sección 2.1 y aplicamos el filtro descrito en la sección 2.2 con $\sigma = 0.58$. Además, consideramos una cubierta de $n = 4$ intervalos con $p = 0.4999$ de traslape en el rango del filtro y un radio máximo para el proceso de agrupación de enlace simple de $rc = 0.58$. De esta forma, al aplicar el algoritmo se obtuvo un mapper resultante conexo de 6 nodos y 5 aristas, el cual se muestra en la figura 2.5

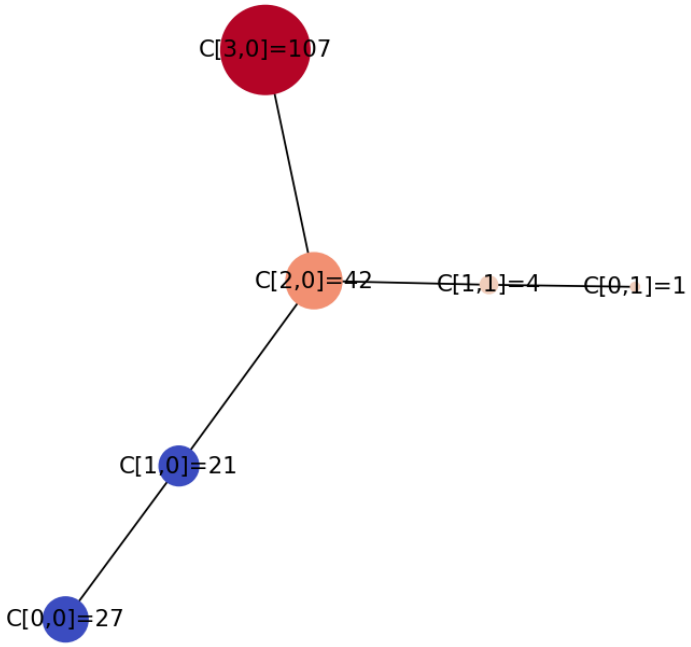


Figura 2.5: Grafo mapper resultante de aplicar el algoritmo Mapper a la base de datos normalizada de la diabetes, usando el filtro del Kernel Gaussiano y una cubierta de 4 intervalos con 49.99% de traslape. Las etiquetas representan (de izquierda a derecha) el nombre de cada nodo, su componente conexa y la cantidad de elementos en el cluster que representan. Este mapper está disponible en el archivo `high_re.png` en el repositorio mencionado al inicio de esta sección.

El mapper resultante mostrado en la figura 2.5 realiza una clasificación similar a la obtenida por Singh, Mémoli y Carlssson (figura 2.2 (b)). El reporte de clasificación del mapper resultante se muestra en la figura 2.6. En este caso, el cambio en la cubierta produce un mapper resultante con más nodos pero una mejor clasificación, pues si se observan las “alas” de nuestro mapper comparadas con las obtenidas en el experimento anterior, los nodos que logran aislar por completo los individuos con clasificación tipo 1 y tipo 2 contienen agrupa-

ciones mayores de elementos del conjunto de datos.

Así, con la tupla de parámetros, el filtro y el proceso de normalización escogidos se obtienen los resultados esperados para el experimento presentado por los autores en [1].

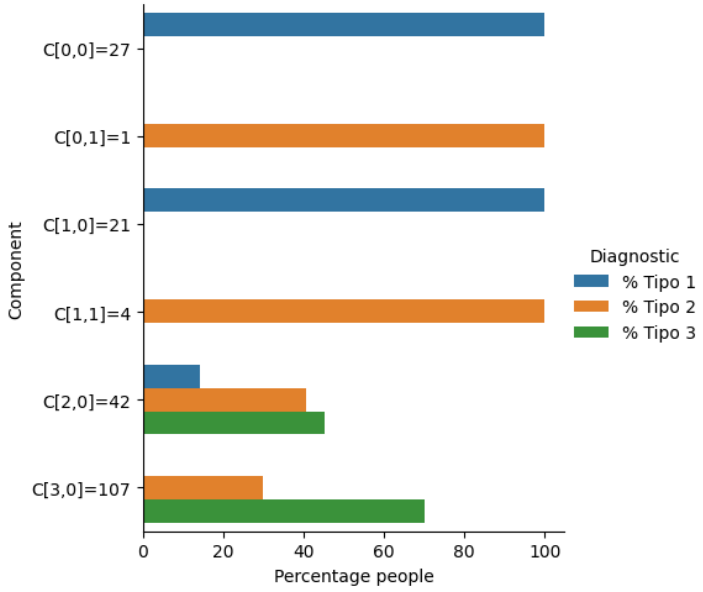


Figura 2.6: Reporte de clasificación del mapper resultante mostrado en la figura 2.5. Este reporte está disponible en el archivo `high_re_report.png` en el repositorio mencionado al inicio de esta sección.

3

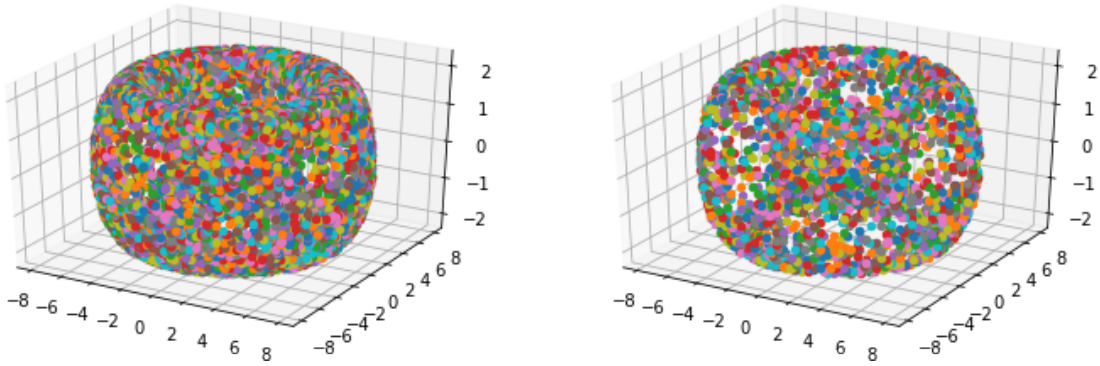
Aplicación del algoritmo Mapper a la base de datos del toro

En este capítulo se aplica el algoritmo mapper a diferentes bases de datos sobre la superficie del toro.

Para la aplicación del algoritmo Mapper al conjunto de datos del toro, se tuvieron en cuenta varias consideraciones. Primero, se generó una base de datos aleatoria sobre la superficie del toro en tres dimensiones con la librería `tadasets` [42]

```
data_torus = tadasets.torus(n=7000, c=5.7, a=2.1),
```

donde el parámetro n es la cantidad de datos y los parámetros c y a son las medidas del toro (distancia del centro del toro al centro del tubo del toro y radio del tubo del toro respectivamente). Así, se obtiene un arreglo de tamaño $n \times 3$ que posteriormente se guardó en un archivo .csv.



(a) Conjunto de datos generado con $n = 7000, c = 5.7$ y $a = 2.1$.

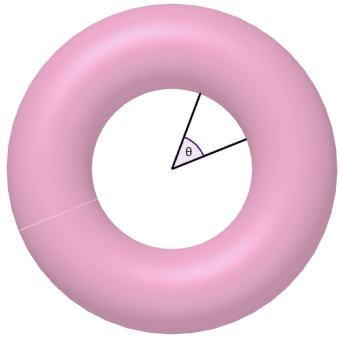
(b) Conjunto de datos generado con $n = 3000, c = 5.7$ y $a = 2.1$.

Figura 3.1: Conjuntos de datos sobre la superficie del toro generados con la librería `tadasets`.

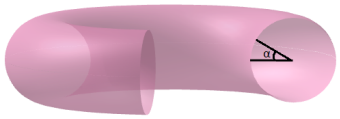
Por otro lado, se generó también un conjunto de datos construido con la siguiente información:

- a y c representan las medidas del toro tal y como se describieron anteriormente.
- $x = dcos(\theta)$, $y = dsen(\theta)$ y $z = \pm\sqrt{a^2 - (c - \sqrt{x^2 + y^2})^2}$ son las coordenadas de los puntos pertenecientes al conjunto de datos.
- θ es el ángulo que describe el recorrido "horizontal" de los puntos, es decir, el recorrido que realizan los puntos alrededor del orificio central del toro.
- α es el ángulo que describe el recorrido "vertical" de los puntos, es decir, el recorrido que realizan los puntos alrededor del centro del tubo del toro.
- d es la distancia del centro del toro a la proyección en el plano xy de los puntos generados en cada iteración, dada por $d = c - acos(\alpha)$.

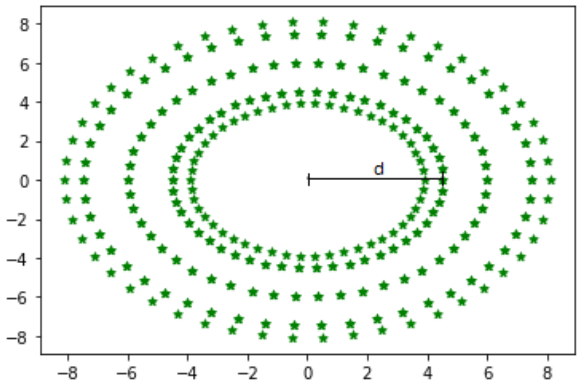
Así, se obtiene un arreglo de tamaño $n \times 3$, donde $n = \frac{2\pi}{\alpha}\frac{2\pi}{\theta}$. Este conjunto de datos fue construido con la intención de que los puntos estén distribuidos uniformemente sobre la superficie del toro, tal como se indica en [1].



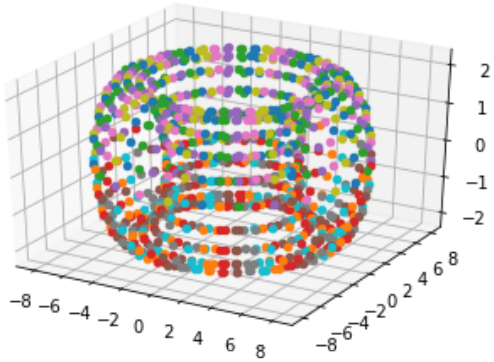
(a) Representación gráfica del ángulo θ.



(b) Representación gráfica del ángulo α.



(c) Representación gráfica de d en la segunda iteración.



(d) Conjunto de datos construido con $a = 2.1, c = 6, \theta = \frac{\pi}{25}$ y $\alpha = \frac{\pi}{10}$.

Figura 3.2: Representaciones gráficas de los parámetros θ, α y d del conjunto de datos construido. (d) muestra un conjunto de datos de 1000 elementos con los parámetros señalados.

3.1 Filtros: Proyección y Laplaciano

Para la aplicación del algoritmo Mapper a la base de datos del toro, se implementaron filtros 2-dimensionales.

3.1.1 FILTRO DE PROYECCION.

Sea $f : X \subset \mathbb{R}^3 \to \mathbb{R}^2$ tal que para $(x, y, z) \in X$, $f(x, y, z) = (x, y)$. Para el conjunto de datos del toro, se tomó a f como función filtro, obteniendo así un arreglo de tamaño $n \times 2$ con los datos filtrados.

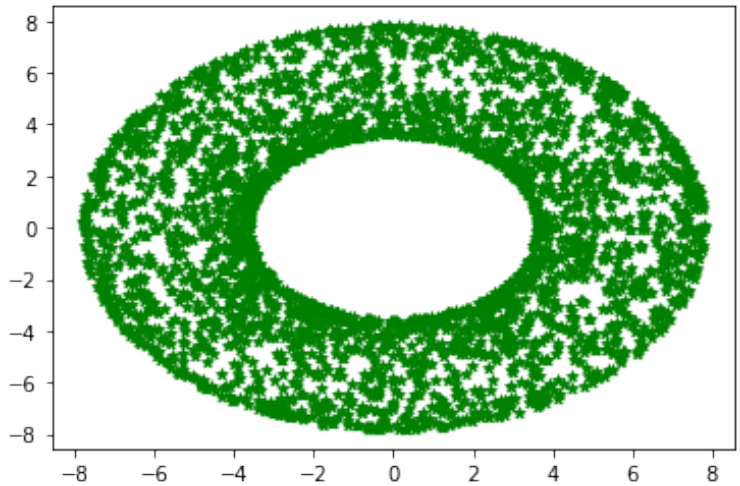


Figura 3.3: Representación gráfica de los datos obtenidos al aplicar el filtro de proyección a la primera y segunda coordenada a una base de datos aleatoria sobre la superficie del toro, generada con $n = 3000, c = 5.7$ y $a = 2.1$.

3.1.2 **FILTRO DEL LAPLACIANO.** Para la implementación del filtro del Laplaciano ([1] sección 4.3) se construyó un grafo completo donde cada elemento del conjunto de datos representa un vértice en el grafo y cada arista tiene asignado un peso descrito por

$$w(v_i, v_j) = e^{\frac{-d(v_i,v_j)^2}{\varepsilon}}, \tag{3.1}$$

con v_i, v_j vértices en el grafo, $d(v_i, v_j)$ la distancia euclidiana de v_i a v_j y ε un parámetro de densidad. De esta forma, los pesos muestran nociones de cercanía entre los vértices y el grafo (con los pesos) representa la geometría local conocida de la base de datos [30]. Así, se construye la matriz Laplaciana normalizada

$$L_{ij} = \frac{w(v_i, v_j)}{\sqrt{\sum_k w(v_i, v_k)}\sqrt{\sum_k w(v_j, v_k)}}. \tag{3.2}$$

Luego, el filtro se define como una función f con

$$\begin{aligned} f : X \subset \mathbb{R}^3 &\longrightarrow \mathbb{R}^2 \\ v_i &\longmapsto [w_{1i} \;\; w_{2i}], \end{aligned}$$

donde las asignaciones para cada elemento del conjunto de datos se describen como

$$\begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix} \longmapsto \begin{bmatrix} w_{11} & w_{21} \\ w_{12} & w_{22} \\ \vdots & \vdots \\ w_{1n} & w_{2n} \end{bmatrix}, \tag{3.3}$$

tales que $(w_{11}, w_{12}, ..., w_{1n})$ y $(w_{21}, w_{22}, ..., w_{2n})$ son vectores propios de la matriz L_{ij}.

Singh, Mémoli y Carlsson [1] mencionan que dicho conjunto de vectores propios de la matriz L_{ij} guarda información geométrica del conjunto de datos. Esto se explica de manera detallada en [8] y [30]. Además, al aplicar el algoritmo Mapper a la base de datos de toro, los autores en [1] escogieron "las dos primeras eigenfunciones no triviales del Laplaciano" como su función filtro. Por lo que, para escoger los dos vectores propios (eigenfunciones) a utilizar, consideremos primero un ejemplo más intuitivo 1-dimensional de la aplicación del filtro del laplaciano.

Ejemplo 3.1. Sea $X = \{1,2,3,4,5,6,7,8\}$ un conjunto de datos sobre un círculo. Se construye entonces el grafo completo con pesos asignados de manera intuitiva, donde puntos más cercanos tienen mayor peso, como se muestra en la figura 3.4.

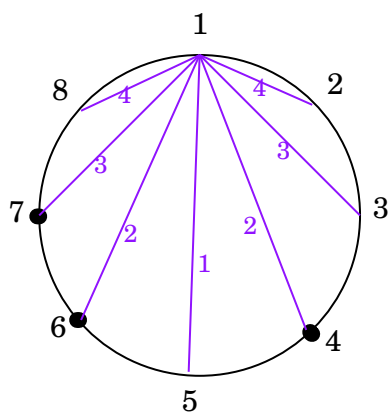


Figura 3.4: Conjunto de datos X con las aristas y pesos asignados para el vértice 1. De manera similar se asignan las aristas y pesos de los vértices restantes.

Así, la matriz Laplaciana L construida con los valores L_{ij} de la ecuación 3.2 (sin normalizar), y además teniendo en cuenta que para el valor del peso de un vértice a sí mismo se

considera un valor mayor a cualquier otro peso en el grafo, es

$$L = \begin{bmatrix} 5 & 4 & 3 & 2 & 1 & 2 & 3 & 4 \\ 4 & 5 & 4 & 3 & 2 & 1 & 2 & 3 \\ 3 & 4 & 5 & 4 & 3 & 2 & 1 & 2 \\ 2 & 3 & 4 & 5 & 4 & 3 & 2 & 1 \\ 1 & 2 & 3 & 4 & 5 & 4 & 3 & 2 \\ 2 & 1 & 2 & 3 & 4 & 5 & 4 & 3 \\ 3 & 2 & 1 & 2 & 3 & 4 & 5 & 4 \\ 4 & 3 & 2 & 1 & 2 & 3 & 4 & 5 \end{bmatrix}$$

Luego, al calcular los valores propios de la matriz L se obtienen, ordenados de mayor a menor

$$24, 4+2\sqrt{2}, 4+2\sqrt{2}, 4-2\sqrt{2}, 4-2\sqrt{2}, 0, 0, 0,$$

con vectores propios asociados

$$\begin{aligned}
f_1 &= (1,1,1,1,1,1,1,1), \\
f_2 &= (\sqrt{2},1,0,-1,-\sqrt{2},-1,0,1), \\
f_3 &= (-1,-\sqrt{2},-1,0,1,\sqrt{2},1,0), \\
f_4 &= (-\sqrt{2},1,0,-1,\sqrt{2},-1,0,1), \\
f_5 &= (-1,\sqrt{2},-1,0,1,-\sqrt{2},1,0), \\
f_6 &= (-1,0,0,1,-1,0,0,1), \\
f_7 &= (-1,0,1,0,-1,0,1,0), \\
f_8 &= (-1,1,0,0,-1,1,0,0)
\end{aligned}$$

respectivamente. f_1 es entonces la primera eigenfunción trivial que colapsa todos los puntos del conjunto de datos a un único número real. Las eigenfunciones f_2 y f_3 representan dos funciones altura que se ven como se muestra en las figuras 3.5 y 3.6. Por lo tanto, las dos primeras eigenfunciones no triviales del Laplaciano para este ejemplo se obtuvieron de escoger dos vectores propios asociados al mayor valor propio de multiplicidad mayor a 1. Como el presente ejemplo es 1−dimensional, sólo se toma como función filtro a una de las eigenfunciones escogidas.

Nota. Las eigenfunciones f_4, f_5, f_6, f_7 y f_8, muestran una filtración del conjunto de datos

X que no es similar a la que muestran las eigenfunciones f_2 y f_3, pues la estructura circular del conjunto de datos es deformada. Por lo tanto, resultan más convenientes para la aplicación del filtro (y posteriormente, la construcción del mapper resultante) las eigenfunciones escogidas.

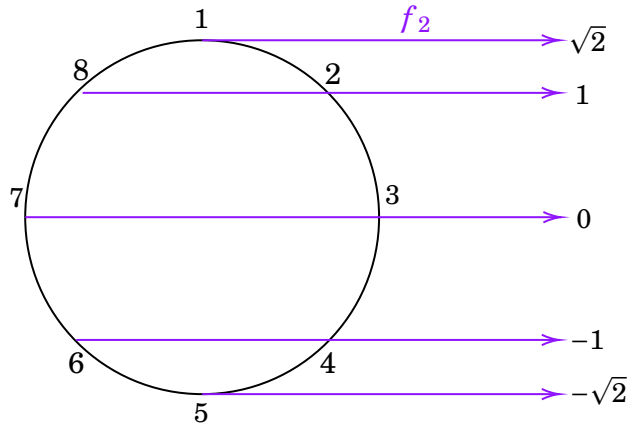


Figura 3.5: Función altura obtenida al aplicar el filtro del Laplaciano al conjunto de datos X usando la segunda eigenfunción de la matriz L.

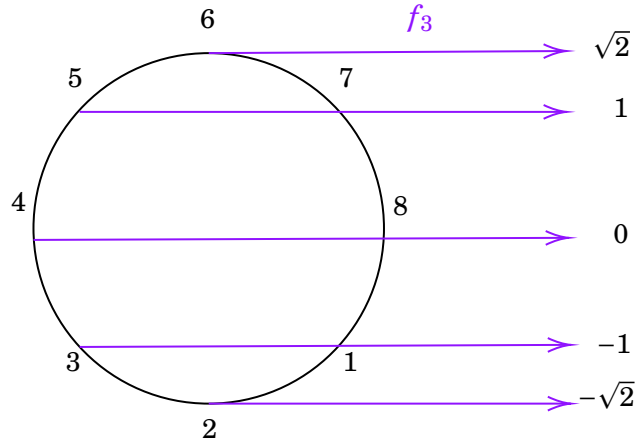


Figura 3.6: Función altura obtenida al aplicar el filtro del Laplaciano al conjunto de datos X usando la tercera eigenfunción de la matriz L.

Ejemplo 3.2. Consideremos ahora un ejemplo similar al anterior pero menos intuitivo, con

$$X = \left\{(0,1), \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right), (1,0), \left(\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}}\right), (0,-1), \left(-\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}}\right), (-1,0), \left(-\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right)\right\}.$$

Para construir la matriz L usamos entonces 3.1 y 3.2 tomando $\varepsilon = 1$. De esta forma se obtiene (de manera similar al ejemplo anterior) que el mayor valor propio es 0.40516561 de multipli-

cidad 1 cuya eigenfunción es trivial. El siguiente valor propio es 0.28274789 de multiplicidad 2 con eigenfunciones que representan funciones altura, tal como se muestra en las figuras 3.7 y 3.8.

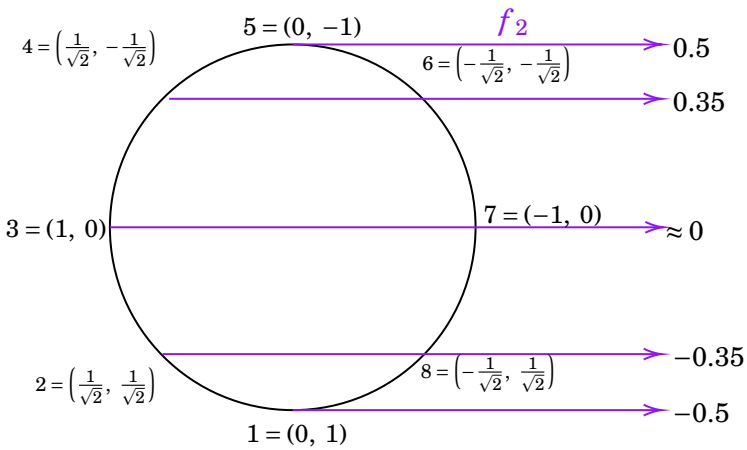


Figura 3.7: Función altura obtenida al aplicar el filtro del Laplaciano al conjunto de datos X usando la segunda eigenfunción de la matriz L.

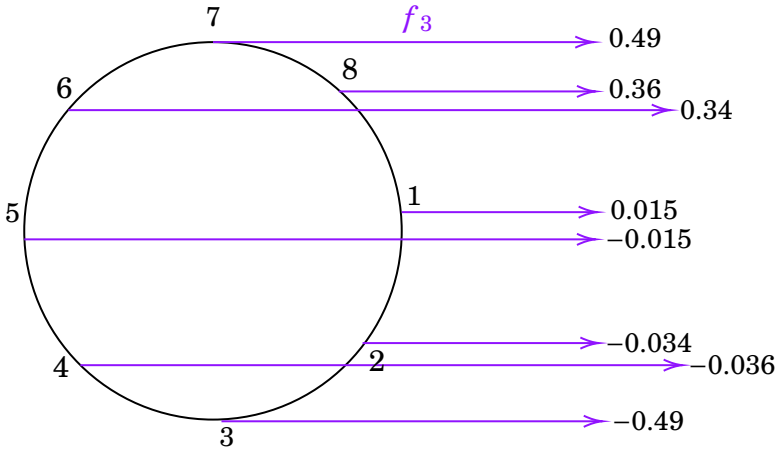


Figura 3.8: Función altura obtenida al aplicar el filtro del Laplaciano al conjunto de datos X usando la tercera eigenfunción de la matriz L.

Nota. Se realizó nuevamente el experimento tomando $\varepsilon = 2$ y $\varepsilon = 0.7$, obteniendo resultados similares en cuanto a las eigenfunciones convenientes a escoger como filtro.

Ejemplo 3.3. (2-dimensional.) Sea X un conjunto de coordenadas en $\mathbb{R}^3$ sobre la superficie del toro construído como se describe a inicios del presente capítulo, con $a = 2.1$, $c = 6$, $\theta = \frac{\pi}{2}$ y $\alpha = \frac{\pi}{2}$, para un total de $n = 16$ coordenadas sobre la superficie del toro en tres dimensiones.

Nuevamente, usamos 3.1 y 3.2 para construir L y calculamos sus valores propios. En todos los casos estudiados, el mayor valor propio tiene multiplicidad mayor a 1. Sin embargo, al realizar las combinaciones posibles de parejas de eigenfunciones asociadas a dicho valor para el filtro, tal como se describe en 3.3, no se logró recuperar una proyección adecuada de los datos en $\mathbb{R}^2$. Por lo tanto, realizando nuevamente las combinaciones posibles de las eigenfunciones asociadas al segundo mayor valor propio de la matriz L, se lograron obtener mejores proyecciones de los datos en $\mathbb{R}^2$. Así, se obtuvieron los resultados mostrados en la figura 3.9.

Por lo tanto, para los experimentos de la aplicación del algoritmo Mapper a la base de datos del toro, se tomarán como eigenfunciones a dos vectores propios asociados al segundo mayor valor propio de la matriz Laplaciana, si y sólo si dicho valor propio tiene multiplicidad mayor o igual a 2, en caso contrario, se tomará el siguiente valor propio que cumpla con la condición mencionada. En caso de que el valor propio escogido tenga multiplicidad estrictamente mayor a 2, se realizarán las combinaciones posibles de parejas de eigenfunciones asociadas a éste hasta obtener una filtración de los datos que arroje mejores resultados a nivel gráfico, es decir, que los datos filtrados muestren información topológica del conjunto de datos en $\mathbb{R}^2$ (similar a como se muestra en la figura 3.3), o en cuanto al análisis cualitativo de los datos en el mapper resultante.

Los autores en [8] demostraron la eficiencia del uso de las dos primeras eigenfunciones no triviales para realizar el análisis topológico de las bases de datos utilizadas en sus experimentos, siendo éstas eigenfunciones las asociadas a los menores valores propios positivos, cuando presentaron el algoritmo para reducir la dimensión de los datos usando la matriz Laplaciana. Sin embargo, en los experimentos realizados en el presente trabajo, los valores propios obtenidos fueron sumamente cercanos a cero, por lo que la escogencia del valor propio que sugieren los autores se torna menos precisa. El uso de diferentes eigenfunciones y variaciones en las matrices laplacianas para varios experimentos de análisis de conjuntos de datos se muestra en [31], donde además es enunciado un Lema sobre la relación entre los eigenvectores asociados a los mayores y menores valores propios al usar la matriz Laplaciana *normalizada*.

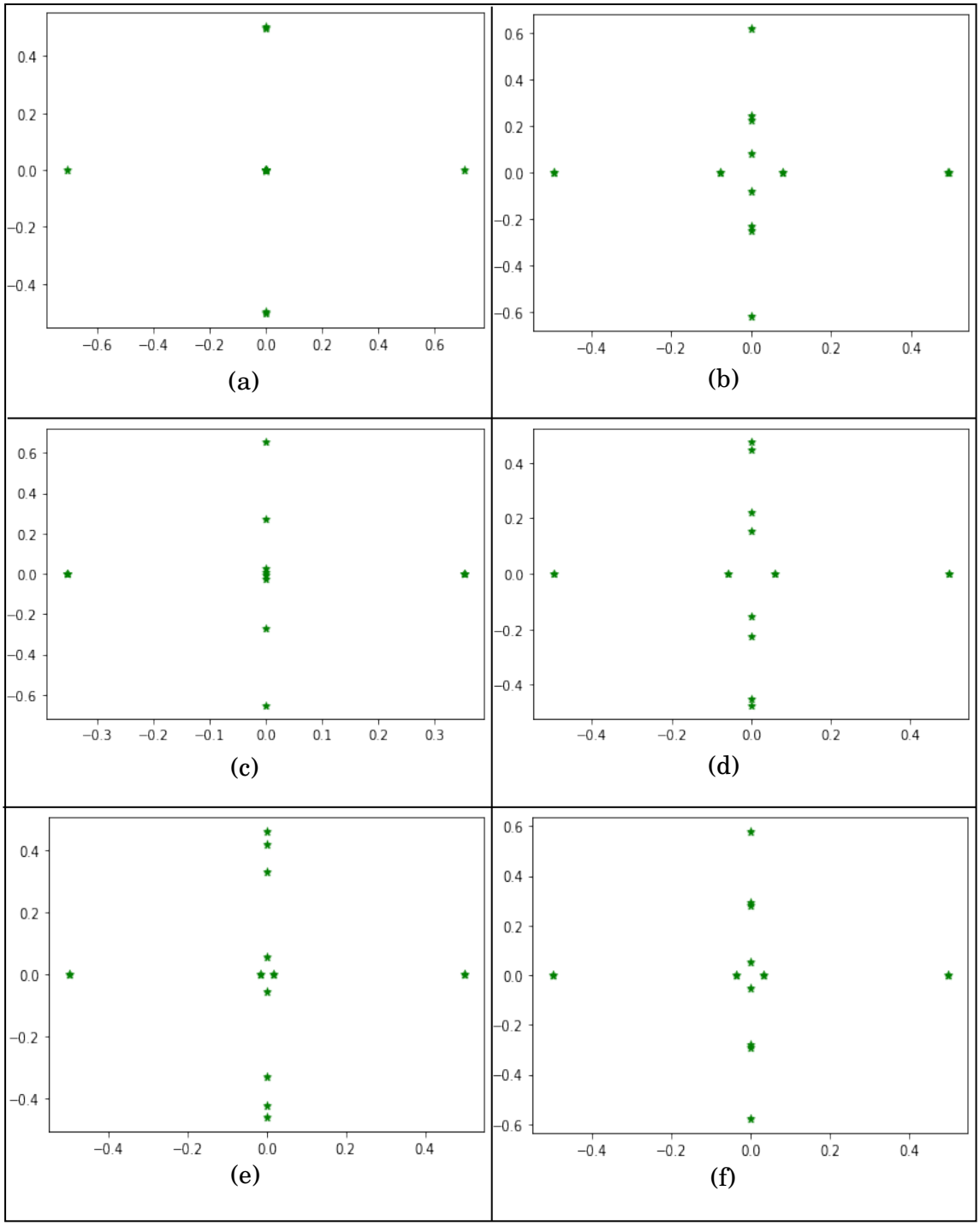


Figura 3.9: Representación gráfica de los datos filtrados aplicando el filtro del Laplaciano al conjunto de datos X. En todos los casos, el valor propio escogido tiene multiplicidad 8. (a) $\varepsilon = 0.7$. (b) $\varepsilon = 1$. (c) $\varepsilon = 1.25$. (d) $\varepsilon = 1.5$. (e) $\varepsilon = 1.75$. (f) $\varepsilon = 2$.

3.2 Cubierta en $\mathbb{R}^2$

Debido a que el filtro a utilizar en los experimentos de la aplicación del algoritmo Mapper a la base de datos del toro es 2-dimensional, la cubierta debe ser a su vez una cubierta de una región en $\mathbb{R}^2$. El mapper resultante que se espera obtener, es un complejo simplicial con vértices, aristas y triángulos rellenos (es decir, un mapper con simplejos máximo de dimensión 2), por lo tanto, entre los elementos de la cubierta pueden presentarse máximo intersecciones triples. De manera intuitiva, dichos elementos podrían ser rectángulos o cuadrados, sin embargo, esta opción presenta dos inconvenientes: al modificar los tamaños de los elementos de la cubierta aumenta la posibilidad de obtener intersecciones cuádruples, por lo que se debe recurrir a corrimientos y otras modificaciones. Por otro lado, que el rango de los datos filtrados esté cubierto de cuadrados o rectángulos limita las opciones de obtener un mapper resultante que preserve la topología del toro, esto debido a que al construir una triangulación de dicha superficie se busca recuperar las dos circunferencias del toro, la central y la del tubo, y esto último se logra al calcular la imagen inversa de los elementos de la cubierta que yacen dentro de su proyección por completo.

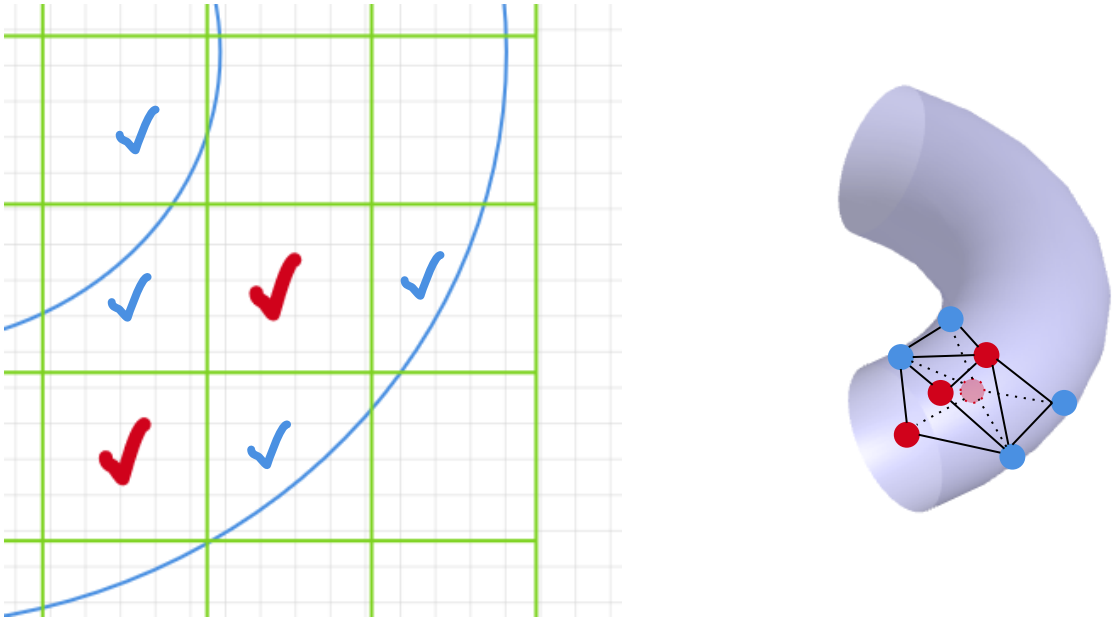
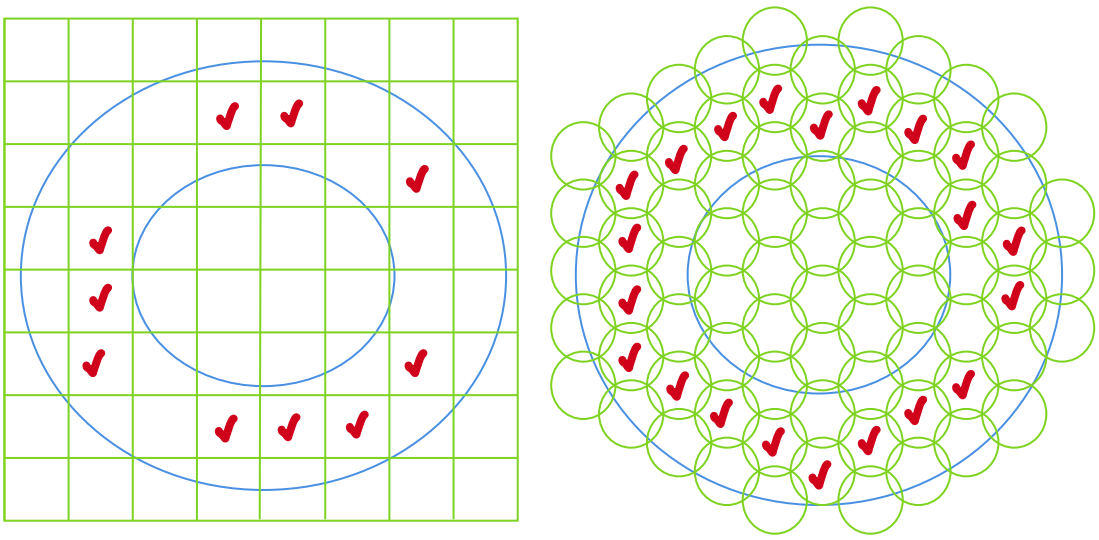


Figura 3.10: A la izquierda, una proyección en $\mathbb{R}^2$ de una parte del toro cubierta por cuadrados. A la derecha, el mapper resultante de los elementos de la cubierta señalados en rojo y azul, suponiendo que se dan tales intersecciones representadas por aristas.

Con un parámetro de agrupación adecuado, los vértices de la figura 3.10 se generan a

partir de las componentes conexas de la imagen inversa de cada elemento de la cubierta señalado, siendo los vértices rojos el resultado de los cuadrados en la cubierta completamente dentro de la región proyectada, con dos componentes conexas en su imagen inversa (la parte superior e inferior del tubo del toro), y los vértices azules el resultado de los cuadrados cuya imagen inversa abarca una región conexa del toro. Por lo tanto, para recuperar las dos circunferencias del toro, es necesario que en el recorrido de los datos filtrados existan elementos de la cubierta adyacentes que estén completamente dentro de la región proyectada. Esto se logra de manera más efectiva cubriendo la región filtrada con formas diferentes a cuadrados o rectángulos.



(a) (b)

Figura 3.11: (a) Proyección del toro en dos dimensiones cubierta por cuadrados. Las regiones donde las señalizaciones rojas no son adyacentes representan extrangulaciones en el mapper resultante alterando la topología del toro, pues las imágenes inversas de los cuadrados pertenecientes a dichas regiones contienen una sola componente conexa de la superficie del toro, esto produce que la circunferencia del tubo del toro no se recupere en su totalidad. Para corregir este comportamiento, sería necesario disminuir considerablemente el tamaño de los cuadrados (aumentando el costo computacional de la implementación) o modificando las medidas del toro (es decir, generando una base de datos nueva). La figura no muestra intersecciones convenientes de los cuadrados y tampoco se evitaron las intersecciones cuádruples, sin embargo, realizando los corrimientos necesarios el resultado es similar. (b) Proyección descrita en (a) pero esta vez cubierta por círculos de diámetro igual al lado de los cuadrados. En este caso, los círculos adyacentes contenidos en la región proyectada dan un giro casi completo. Por lo tanto, las correcciones a hacer en la cubierta para lograr “el giro completo” que recupere el tubo del toro son menores y no aumentan considerablemente el costo computacional del algoritmo.

Notemos además que, al cubrir el rango del filtro con círculos (o equivalentemente, con hexágonos), sólo basta con ajustar el tamaño de éstos para controlar el traslape de los elementos de la cubierta y evitar intersecciones cuádruples. De manera complementaria, en el ejemplo 3.3 de [1] se ilustra el caso de una cubierta para dos regiones rectangulares usando rectángulos y hexágonos.

Para la implementación de la cubierta del rango del filtro aplicado al conjunto de datos del toro, se usarán *círculos de radio r*. Dados $(w_{1i}, w_{2i}), i \in \{1, ..., n\}$ pertenecientes los datos filtrados, la región a cubrir será delimitada por

$$x_{min} := \min_i \ w_{1i}, \ y_{min} := \min_i \ w_{2i},$$
$$x_{max} := \max_i \ w_{1i}, \ y_{max} := \max_i \ w_{2i}.$$

Los círculos serán ubicados a partir del primero, el cual tendrá su centro en (x_{min}, y_{min}), y a partir de una distancia $\sigma = rsin\left(\frac{\pi}{3}\right)$.

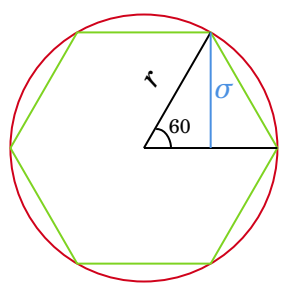


Figura 3.12: Representación gráfica de σ.

Así, realizando una teselación de los hexágonos inscritos en los círculos se obtiene la cubierta del rango del filtro, tal como se ilustra para una muestra de seis círculos en la figura 3.13. Notemos que, hasta este punto las intersecciones triples se dan en un solo punto, lo cual no es conveniente si se desean recuperar simplejos de dimensión 2 en el mapper resultante, por lo tanto, para la implementación de la cubierta se agregará un parámetro adicional R que aumente el radio de los círculos, sin cambiar la ubicación del centro de éstos. De esta forma tenemos que los centros de los elementos de la cubierta son ubicados como se describe en la figura 3.13, pero el radio de éstos es R tal que $r < R < \frac{3}{2}r$, evitando con esto llegar a tener intersecciones cuádruples.

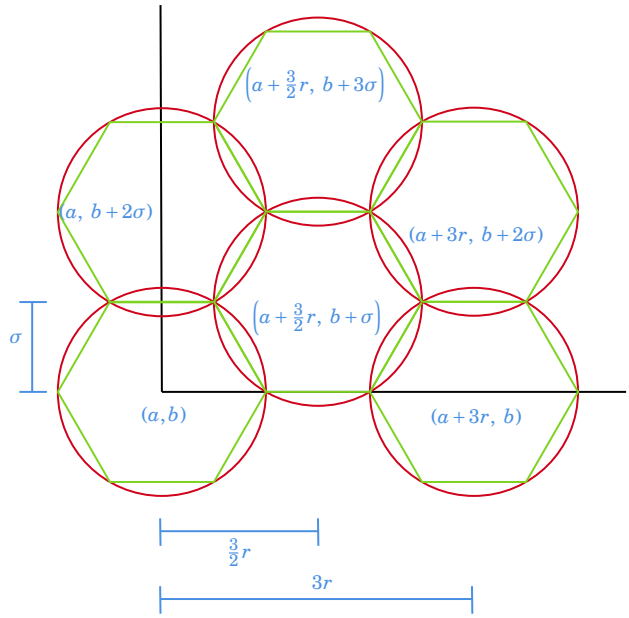


Figura 3.13: Seis círculos de la cubierta ubicados a partir del primero con su centro en coordenadas (a,b), donde $a = x_{min}$ y $b = y_{min}$.

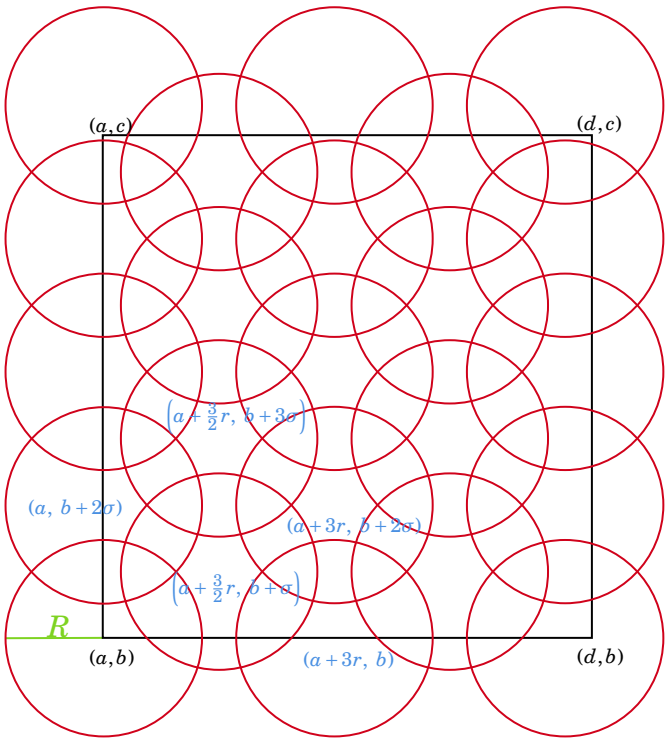


Figura 3.14: Cubierta de la región delimitada por (a,b), (a,c), (d,c) y (d,b), donde $a = x_{min}$, $b = y_{min}$, $c = y_{max}$, $d = x_{max}$, con círculos de radio R (sin embargo, con centros ubicados a partir de círculos de radio r).

3.3 Experimentos

Para los experimentos que se van a describir a continuación, se realizaron distintas pruebas hasta encontrar el conjunto de parámetros adecuados que mostró los resultados deseados. Para estos experimentos, se adaptó el algoritmo Mapper utilizado en los experimentos del capítulo 2 de tal manera que construya el mapper resultante deseado con simplejos de dimensión a lo más 2. La implementación de este algoritmo que será utilizado en las secciones 3.3.1 y 3.3.2 se encuentra en los archivos `TorusProjection.py` y `TorusLaplacian.py` respectivamente de la carpeta `Torus` en `https://github.com/kbotello129/Mapper`.

3.3.1 BASE DE DATOS ALEATORIA: FILTRO DE PROYECCION.

Para el primer experimento realizado de la aplicación del algoritmo Mapper a la base de datos del toro se tomó un conjunto de datos aleatorio tridimensional tal como se describe al inicio del presente capítulo, con $n = 4500$, $c = 5.7$ y $a = 2.1$. Además, para el proceso de agrupación, se tomó un radio máximo de $rc = 1$.

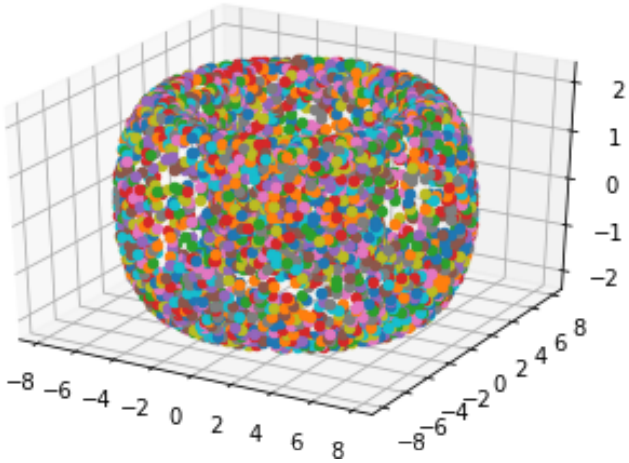


Figura 3.15: Base de datos aleatoria sobre la superficie del toro en tres dimensiones, generada con la librería `tadasets`, con $n = 4500$, $c = 5.7$ y $a = 2.1$.

Sea $X \subset \mathbb{R}^3$ el conjunto de datos generado en la figura 3.15. Posteriormente, se define la función filtro de proyección como se describe en 3.1.1, es decir, la base de datos es proyectada a la primera y segunda componente. Así, se obtiene un arreglo de tamaño 4500×2 con los datos filtrados.

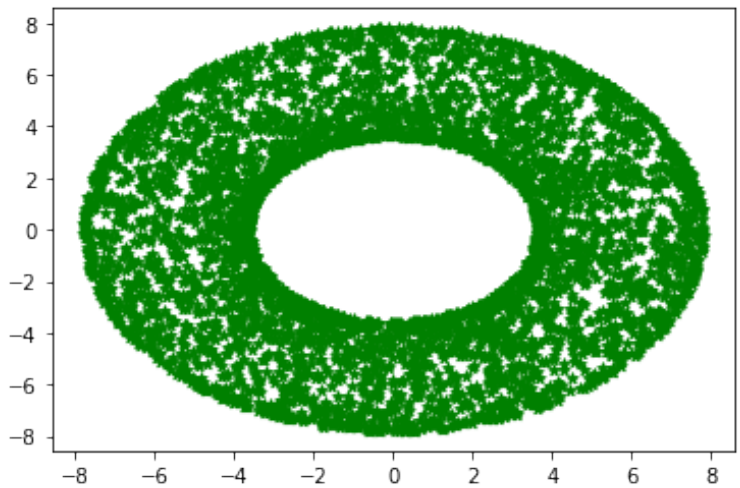


Figura 3.16: Representación gráfica del rango del filtro de proyección a las primeras dos componentes, aplicado al conjunto de datos X.

Se aplicó entonces la cubierta descrita en 3.2 a los datos filtrados, con $r = 0.8$ y $R = 1.17$. Note que, dichas longitudes para los radios fueron escogidas para lograr lo que se muestra en la figura 3.11 (b), teniendo en cuenta cómo se ven los datos proyectados en la figura 3.16 y así recuperar la topología del toro en el mapper resultante.

De esta forma, al ejecutar el algoritmo éste arrojó un mapper resultante con 172 nodos (simplejos de dimensión 0), 515 aristas (simplejos de dimensión 1) y 343 triángulos rellenos (simplejos de dimensión 2. En este caso, todos los triángulos presentes en el mapper están rellenos), el cual se muestra en la figura 3.17.

Para comprobar que, efectivamente el mapper obtenido es el correcto, es decir, que recupera la topología del toro, se realizaron tres verificaciones:

1. La característica de Euler es $vértices - aristas + caras = 172 - 515 + 343 = 0$.

2. La nulidad de la matriz N tiene cardinalidad 1, con $i \in \{1, ..., k\}$, $j \in \{1, ..., l\}$, siendo k y l las cantidades de aristas y triángulos en el mapper resultante respectivamente, y

$$N_{ij} = \begin{cases} 1 & si \quad \text{el triángulo } j \text{ contiene a la arista } i, \\ \\ 0 & \qquad\qquad\qquad \text{de otra manera.} \end{cases} \tag{3.4}$$

Comentario. En términos de Homología simplicial (el capítulo 2 de [5] y [43] brinda un tratamiento adecuado en complejos simpliciales y grupos de Homología), tenemos

que si K es un complejo simplicial, el complejo de cadenas de K se define como

$$C(K): \ldots \xrightarrow{\partial_3} C_2(K) \xrightarrow{\partial_2} C_1(K) \xrightarrow{\partial_1} C_0(K) \xrightarrow{\partial_0} 0,$$

donde cada $C_i(K)$ es un espacio vectorial cuya base son los simplejos de dimensión i en K y donde se cumple que $C_i(K) = 0$ si $dimK < i$. Adicionalmente, estos morfismos cumplen que $\partial_{n-1} \circ \partial_n = 0$ para cada n en el complejo de cadenas, esto es, $Im(\partial_n) \subseteq Ker(\partial_{n-1})$. Luego, el $n-1$-grupo de homología de K se denota como $H_{n-1}(K) = Ker(\partial_{n-1})/Im(\partial_n)$. Tomando coeficientes en $\mathbb{Z}_2$, para el caso del toro $T = S^1 \times S^1$ tenemos que

$$H_i(T) = \begin{cases} \mathbb{Z}_2 & si \quad i = 0,2, \\ \mathbb{Z}_2 \times \mathbb{Z}_2 & si \quad i = 1, \\ 0 & si \quad i \geq 3. \end{cases}$$

Por lo tanto, el complejo simplicial que recupere las características topológicas del toro debe cumplir que $dim(H_0(K)) = 1 = dim(H_2(K))$. Consideremos entonces el mapper resultante K obtenido en el presente experimento y su complejo de cadenas

$$0 \xrightarrow{\partial_3} C_2(K) \xrightarrow{\partial_2} C_1(K) \xrightarrow{\partial_1} C_0(K) \xrightarrow{\partial_0} 0$$

con $C_i(K) = 0,\ i \geq 3$ pues K contiene simplejos de dimensión a lo más 2. Claramente $dim(H_0(K)) = 1$, pues K es conexo y además $H_0(K) = Ker(\partial_0)/Im(\partial_1)$, donde $Ker(\partial_0) = C_0(K) = Im(\partial_1)$. Por otro lado, $H_2(K) = Ker(\partial_2)/Im(\partial_3) = Ker(\partial_2)$ pues $Im(\partial_3) = 0$. N es entonces la matriz asociada al morfismo ∂_2, por lo que $dim(Ker(\partial_2))$ se obtiene al calcular la cardinalidad de la nulidad de N. En conclusión, $dim(H_2(K)) = 1$.

3. Algoritmo con *Teoría de Morse discreta.* La teoría de Morse discreta surgió a principios de este siglo y se ha convertido en una herramienta computacional muy importante en el estudio de objetos geométricos con orígenes tanto teóricos como aplicados. La descripción de dicha teoría escapa a los fines de este trabajo. Sin embargo fue usada con el apoyo de Emilio J. González Rivero a fin de construir un campo gradiente discreto en los mappers resultantes obtenidos en esta sección. El campo gradiente tiene una celda crítica en dimensión cero, dos celdas críticas en dimensión uno y una celda crítica en dimensión dos, así como diferenciales de flujo triviales, lo cual no sólo es compatible con la descripción homológica anterior, sino que dice que el tipo de homotopía de nuestro mapper resultante es en efecto consistente con el del toro.

Así, se comprueba que el mapper obtenido cumple con los resultados esperados.

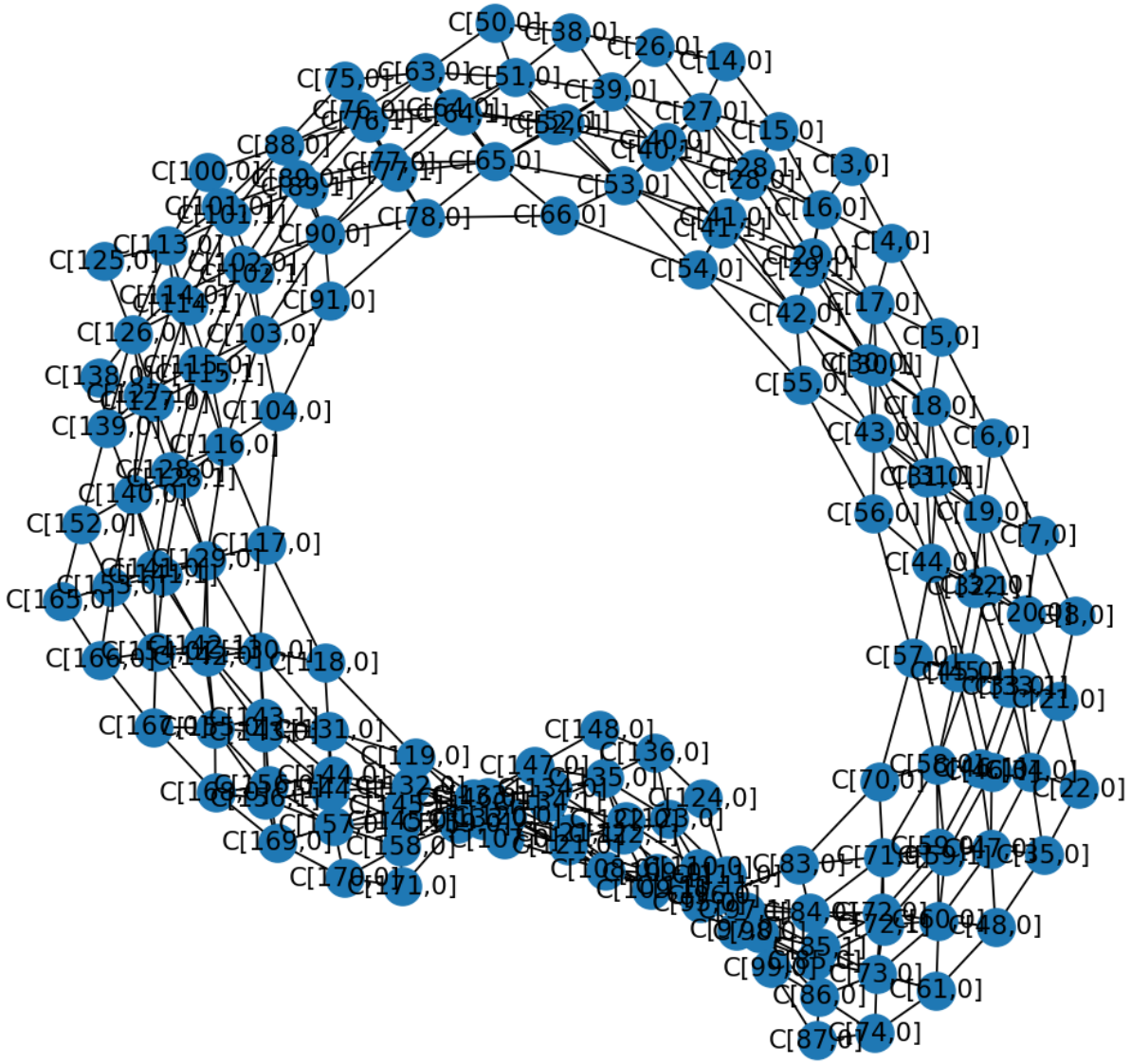


Figura 3.17: Mapper resultante de aplicar el algoritmo Mapper a la base de datos X, usando el filtro de proyección a las dos primeras componentes. Las estiquetas en cada nodo representan el nombre de cada cluster (valor de la izquierda) y su componente conexa (valor de la derecha). La sección 1.2.3 muestra una explicación más detallada de este proceso. Este mapper está disponible en el archivo `torus_proyec.png` en el repositorio mencionado al inicio de esta sección.

3.3.2 **BASE DE DATOS CONSTRUIDA: FILTRO DEL LAPLACIANO.** Para el siguiente experimento realizado, se tomó una base de datos construida de $n = 1000$ coordenadas sobre la superficie del toro, con $a = 2.1$, $c = 6$, $\theta = \frac{\pi}{25}$ y $\alpha = \frac{\pi}{10}$. Dicho conjunto de coordenadas obtenidas fue posteriormente encajado en 30 dimensiones y rotado a un ángulo aleatorio. De esta forma, el conjunto de datos contiene puntos sobre la superficie del toro en 30 dimensiones rotado a un ángulo aleatorio. Para el proceso de agrupación, se escogió un radio máximo de $rc = 1.5$.

El filtro utilizado para este experimento fue el filtro del Laplaciano (3.1.2), con parámetro $\varepsilon = 4$ para el cálculo de los pesos en la construcción del grafo. Las pruebas realizadas mostraron que, al aumentar el valor de ε, el orificio central que muestra la representación gráfica de los datos filtrados (figura 3.18) se hace más grande, por lo que su valor fue escogido para así recuperar dicha característica topológica al ejecutar el algoritmo. De este modo, se obtiene un arreglo de tamaño 1000×2 con los datos filtrados.

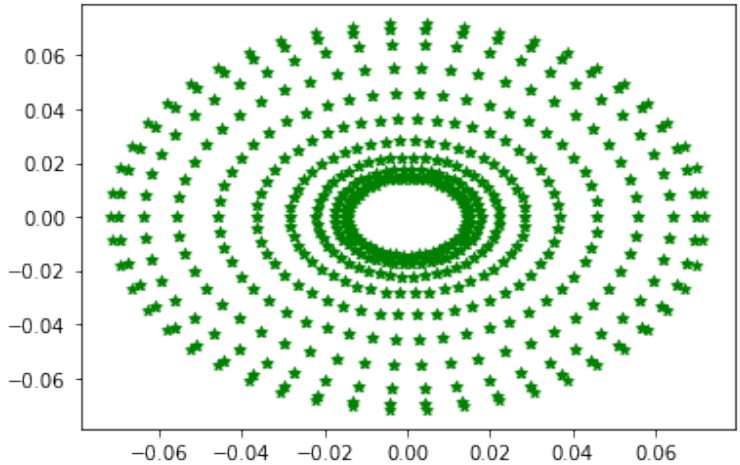


Figura 3.18: Representación gráfica del rango del filtro del Laplaciano, aplicado a la base de datos construida.

Se aplicó entonces la cubierta descrita en 3.2 a los datos filtrados, con $r = 0.01$ y $R = 0.015$. De esta forma, al ejecutar el algoritmo éste arrojó un mapper resultante con 113 nodos, 333 aristas y 220 triángulos rellenos, el cual se muestra en la figura 3.19.

En este caso, sólo uno de los triángulos presentes en el mapper no está relleno, específicamente el triángulo formado por los nodos $C[52,0]$, $C[51,0]$ y $C[42,0]$, es decir, dicha terna de clusters no presenta una intersección triple y por lo tanto, es ésta la sección del mapper resultante la que recupera el orificio central del toro. Nuevamente, se comprobó que el map-

per resultante cumple con los resultado esperados calculando la característica de Euler, la nulidad y aplicando el Algoritmo con *Teoría de Morse discreta*.

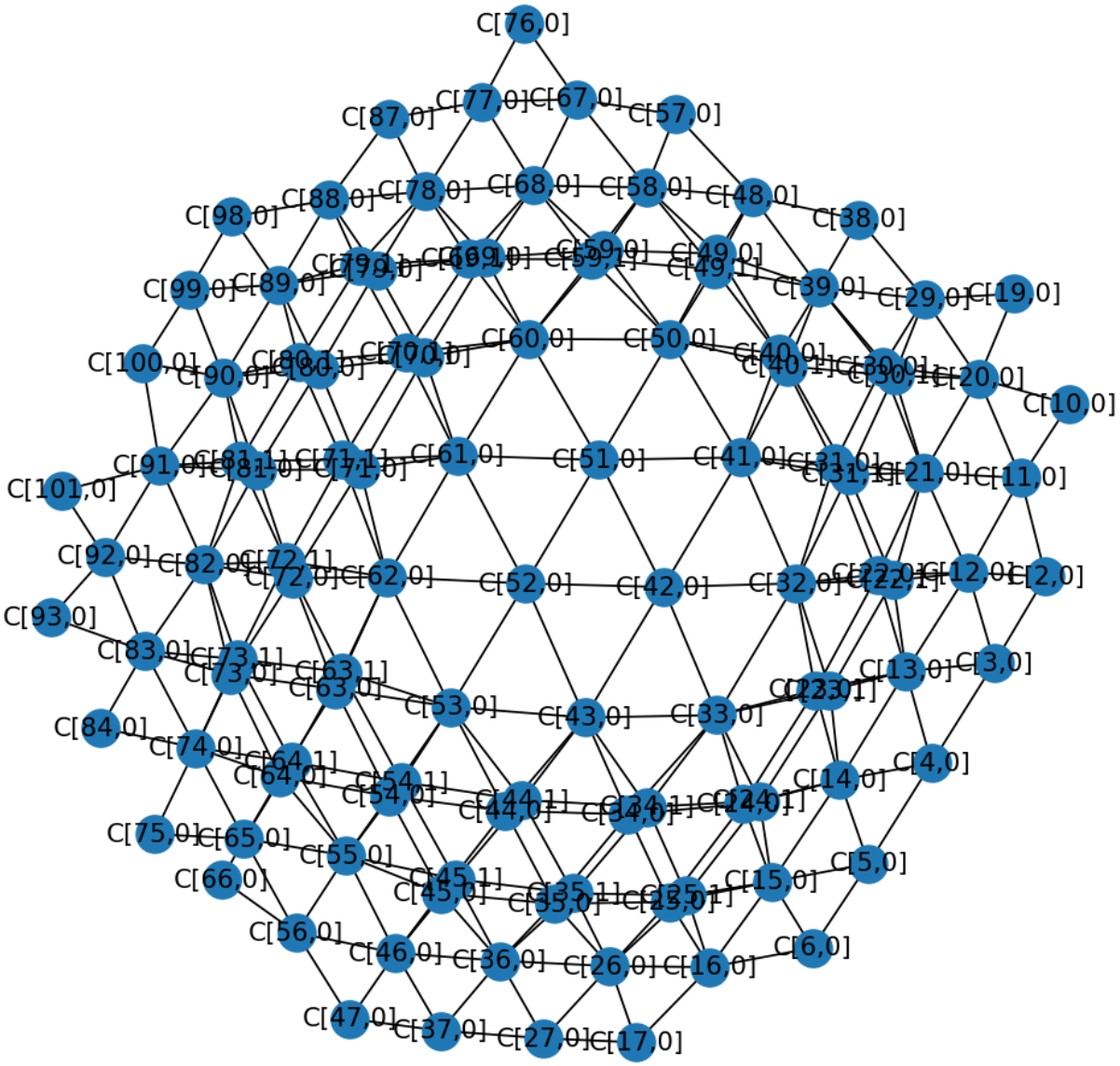


Figura 3.19: Mapper resultante de aplicar el algoritmo Mapper a la base de datos construida con los parámetros escogidos, usando el filtro del Laplaciano. Este mapper está disponible en el archivo `torus_lapla.png` en el repositorio mencionado al inicio de esta sección.

4

Base de datos calidad del aire

En este capítulo se aplica el algoritmo Mapper a la base de datos de la calidad del aire en Ciudad de México durante el año 2018. Este estudio es original y fue realizado en colaboración con Fernando Moreno Gómez.

La base de datos 4−dimensional de la calidad del aire fue obtenida a partir de los datos que muestra la página de Dirección de Monitoreo Atmosférico en la Ciudad de México, disponible en `http://www.aire.cdmx.gob.mx/`, donde se realiza un registro diario de varios agentes contaminantes y valores meteorológicos en distintas zonas de la ciudad. La página muestra las estaciones de monitoreo en el mapa de la ciudad y sus respectivos estados actuales en términos de calidad del aire. Las clasificaciones de índice de aire y salud posibles son "Buena", "Aceptable", "Mala", "Muy mala", "Extremadamente mala" y "outliers" que representan valores anormales y considerablemente diferentes de uno o más agentes contaminantes con respecto a los demás registrados. Los intervalos que determinan la clasificación de cada contaminante y la clasificación general, las unidades de medida, consecuencias en la salud, cálculos para cada índice de la calidad del aire entre otra información relevante se encuentran en la norma con los lineamientos para la obtención y comunicación del Índice de Calidad del Aire y Riesgos a la Salud, y la norma que establece los requisitos para elaborar el Índice de Calidad del Aire en la Ciudad de México [1]. A partir de este punto, se denotarán a las estaciones de monitoreo como *estaciones* y a los agentes contaminantes y meteorológicos como *parámetros*.

La base de datos fue construida tomando los datos registrados de cinco estaciones durante el año 2018: TLA (Tlalnepantla), XAL (Xalostoc), MGH (Miguel Hidalgo), CUA (Cuajimal-

[1] `http://www.aire.cdmx.gob.mx/default.php?opc=%27ZaBhnmI=&dc=%27Yw==`

pa) y MER (Merced). Dichas estaciones fueron escogidas teniendo en cuenta su estabilidad (presencia a lo largo de los años), una menor cantidad de valores nulos, su ubicación y por lo tanto relevancia a la hora de medir niveles de contaminación, y los parámetros que registran. La figura 4.1 muestra la ubicación de las estaciones escogidas. Algunas de las razones por las cuales se puede obtener un valor nulo en los registros es debido a fallas en alguna estación o mantenimiento de éstas. Más información sobre el comportamiento de las estaciones y de los datos registrados se encuentra en [25]. Adicionalmente, se tuvieron en cuenta cuatro parámetros: dióxido de nitrógeno (NO2), ozono (O3), partículas PM10 y la velocidad del viento (WSP). En términos de contaminación y calidad del aire estos parámetros son relevantes, ya sea por su impacto en los niveles de otros parámetros (como es el caso de la velocidad del viento, donde a menor velocidad del viento se suelen presentar mayores niveles de contaminación), por la geografía de la ciudad o su impacto en la salud. Además, las estaciones y parámetros escogidos fueron utilizados en estudios anteriores y arrojaron resultados satisfactorios en los experimentos mostrados en el presente capítulo.

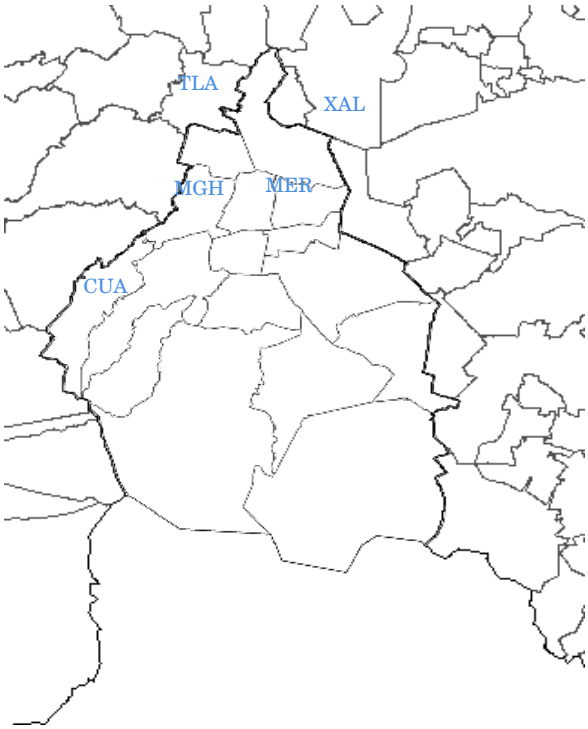


Figura 4.1: Ubicación de las estaciones de monitoreo escogidas para obtener el conjunto de datos de la calidad del aire. La información actualizada de los parámetros que registra cada estación se encuentra en `http://www.aire.cdmx.gob.mx/default.php?opc=%27ZaBhnmM=%27`.

Por último, no se tuvieron en cuenta los registros del periodo de tiempo de las 10 PM a las 6 AM pues es éste el intervalo de tiempo que presenta más valores nulos en general debido a mantenimiento en las estaciones o no representan horas críticas de contaminación debido al poco tráfico en la ciudad. De esta forma, consideremos dos conjuntos de datos a usar para aplicar el algoritmo Mapper: un arreglo de tamaño 6989×4 con los registros de las cinco estaciones y los cuatro parámetros escogidos en el intervalo de tiempo de las 6 AM a las 10 AM durante el año 2018, más una quinta columna que contiene la clasificación numérica (índice de aire y salud) del registro, siendo 1 Buena, 2 Aceptable, 3 Mala, 4 Muy mala, 5 Extremadamente mala y 6 outlier. Este intervalo de tiempo se considera el más crítico en la ciudad en términos de niveles de contaminación. Además, consideremos el arreglo de tamaño 23548×4 con los registros de las cinco estaciones y los cuatro parámetros escogidos en el intervalo de tiempo de las 6 AM a las 10 PM durante el año 2018, más una quinta columna que contiene la clasificación numérica ya mencionada. Dichos conjuntos de datos fueron posteriormente guardados en un archivo `.csv` cada uno. Denotaremos al primer conjunto de datos mencionado como la *base de datos crítica* y al segundo conjunto de datos mencionado como la *base de datos general*.

Para los experimentos del presente capítulo, se aplicará el algoritmo Mapper utilizando la librería `Keppler Mapper` [18] [44], pues su implementación acepta sin inconveniente el método de agrupación, el filtro y la cubierta a utilizar, además de estar optimizada para reducir costo computacional y tiempo de ejecución para bases de datos de gran tamaño. Adicionalmente, nos permite mostrar en el presente trabajo su funcionamiento, utilidad y aportar más ejemplos de su aplicación.

4.1 Normalización de los datos

Nuevamente, como ocurrió en los experimentos del capítulo 2, los conjuntos de datos presentan valores con rangos diferentes, por lo que se realizó como primer paso un proceso de normalización de manera análoga a como se explica en la sección 2.1 para los conjuntos de datos a utilizar en los experimentos del presente capítulo. De esta forma, se obtienen un arreglo de tamaño 6989×4 con los datos normalizados para la base de datos crítica y un arreglo de tamaño 23548×4 con los datos normalizados para la base de datos general.

Este método específico de normalización fue escogido para la aplicación del algoritmo Mapper a las bases de datos de la calidad del aire debido a que arrojó mejores resultados en conjunto con el método de agrupación escogido, el DBSCAN (sección 1.2.3). Otros métodos de

normalización de la librería `sklearn` como el `StandardScaler`[2] o el `QuantileTransformer`[3] presentaron una pérdida de datos importante en la construcción del mapper resultante, pues clasificaban una mayor cantidad de datos como puntos aislados (outliers) y el `Keppler Mapper` no los toma en cuenta para construir el mapper resultante. De esta forma, comparando los procesos de normalización mencionados (cuadro 4.1), el `MinMaxScaler` presentó una menor cantidad de puntos aislados y por lo tanto se obtuvo una menor pérdida de datos en el mapper resultante. En [45] se hace una comparativa de diferentes métodos de normalización en términos de la cantidad de puntos aislados obtenidos.

Base de datos crítica		
`StandardScaler`	`MinMaxScaler`	`QuantileTransformer`
6811	19	131
Base de datos general		
`StandardScaler`	`MinMaxScaler`	`QuantileTransformer`
20005	14	26

Cuadro 4.1: Cantidad de puntos aislados clasificados por el algoritmo de agrupación DBSCAN para las bases de datos normalizadas con cada método mencionado. Para ambas bases de datos a utilizar en los experimentos, normalizar los datos con el método `MinMaxScaler` arrojó una menor cantidad de puntos aislados al realizar el proceso de agrupación.

Nota. En la construcción del mapper resultante, la librería `Keppler Mapper` extrae del algoritmo de agrupación una cantidad mínima de datos para tener en cuenta un cluster y crear un nodo. Este proceso conlleva a una pérdida de datos para agrupaciones menores o iguales a dicha cantidad. Para el caso del DBSCAN, el valor de `min_samples` por default es 5, por lo tanto, al construir el mapper resultante no se tienen en cuenta agrupaciones de datos menores a 5 elementos en la preimagen de los elementos de la cubierta para la creación de un nodo.

[2] `https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html`

[3] `https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.QuantileTransformer.html`

4.2 Filtro: Estimación de densidad usando un kernel Gaussiano

Tal como se sugiere en la sección 1.2.1, para las bases de datos a utilizar resulta conveniente utilizar un filtro de densidad. Por lo tanto, se utilizó la misma función filtro del Kernel Gaussiano implementada para los experimentos realizados en el capítulo 2. Como la dimensión de los conjuntos de datos a utilizar es diferente en este caso, la función filtro queda definida como $f : X \subset \mathbb{R}^4 \to \mathbb{R}$ donde $f(x)$ se describe como muestra la ecuación 2.1, con x, y elementos del conjunto de datos, $d = 4$, $d(x, y)$ la distancia euclidiana y $n = 6989$ para los experimentos utilizando la base de datos crítica o $n = 23548$ para los experimentos utilizando la base de datos general. De esta forma, se obtienen un arreglo de tamaño 6989×1 para la base de datos crítica y un arreglo de tamaño 23548×1 para la base de datos general con los datos filtrados. Los valores de σ escogidos para cada experimento se muestran más adelante en la sección 4.5.

Tal como mencionan los autores al introducir el algoritmo Mapper en [1], cualquier función filtro que refleje propiedades del conjunto de datos puede ser utilizada. En este caso, la escogencia de la función filtro del Kernel Gaussiano arrojó resultados interesantes por lo que se mantuvo para los experimentos que se mostrarán en el presente capítulo, abriendo la posibilidad de futuros estudios similares que arrojen a su vez resultados interesantes utilizando otras funciones para filtrar los datos.

4.3 Cubierta en $\mathbb{R}$

Definimos una cubierta 1−dimensional para el rango de la función filtro usando la librería `Keppler Mapper` [18], más específicamente la clase `kmapper.Cover` [4] la cual recibe tres parámetros: número de intervalos (o de n cubos para cubiertas multidimensionales), porcentaje de traslape y límites del rango a cubrir. El porcentaje de traslape es calculado teniendo en cuenta la fórmula mostrada en [37]

$$0 < traslape = \frac{cube_i \cap cube_{i+1}}{cube_i} < 0.5.$$

Para los experimentos sobre los conjuntos de datos de la calidad del aire, asignaremos los valores de `n_cubes` y `perc_overlap` según corresponda en cada experimento, con `limits=None`. De esta forma, obtenemos la cubierta con el número de intervalos y porcentaje de traslape deseado, la cual se enviará como parámetro en el método que construye el mapper resultante.

[4] `https://kepler-mapper.scikit-tda.org/en/latest/reference/stubs/kmapper.Cover.html`

4.4 Agrupación y mapper

El algoritmo de agrupación que se aplicó a los conjuntos de datos para los experimentos fue el DBSCAN [35], más específicamente el implementado por la librería `sklearn` [22] de Python, explicado en la sección 1.2.3. En este caso, el único parámetro a variar es `eps` según convenga en cada experimento.

El mapper resultante es construido utilizando el método `kmapper.KepplerMapper.map`[5], el cual recibe para los experimentos los parámetros

- `lens`, el arreglo con los datos filtrados.
- `X`, el conjunto de datos original. En este caso se inserta la base de datos normalizada.
- `clusterer`, el algoritmo de agrupación. En este caso se inserta la clase `sklearn.cluster.DBSCAN` con el parámetro `eps` escogido.
- `cover`, la cubierta. En este caso, se inserta la clase `kmapper.Cover` con los parámetros de `n_cubes` y `perc_overlap` escogidos.

El método retorna el complejo simplicial (en este caso el grafo mapper resultante) como un diccionario que contiene el nombre de cada nodo y la lista de elementos en el conjunto de datos pertenecientes a dicho cluster. Finalmente, el mapper resultante se visualiza utilizando la misma clase y el método `visualize`, el cual recibe para los experimentos los parámetros

- `graph`, el diccionario obtenido por el método `map`.
- `color_values`, un arreglo 1−dimensional con los colores a asignar a los nodos, donde cada elemento representa un color. Para los experimentos del presente capítulo, se inserta la última columna del conjunto de datos que contiene las clasificaciones numéricas de cada elemento.
- `color_function_name`, una cadena que describe la asignación de los colores en `color_values`.
- `path_html`, una cadena con el nombre del archivo .html que muestra la visualización del mapper resultante.
- `title`, una cadena con el nombre del mapper resultante.

[5] https://kepler-mapper.scikit-tda.org/en/latest/reference/stubs/kmapper.KeplerMapper.html

- `custom_tooltip`, un arreglo 1−dimensional que identifica a los elementos de cada cluster y los muestra al hacer click en cada nodo. En este caso, se inserta la última columna del conjunto de datos que contiene las clasificaciones numéricas de cada elemento. De esta forma, al seleccionar un nodo y hacer click en "cluster details" se muestran las clasificaciones numéricas de cada elemento perteneciente a dicho cluster. Esto permite justificar el color obtenido en el nodo y visualizar la clasificación predominante en éste.

El método retorna el archivo .html que redirige a la visualización del mapper resultante y su información detallada.

4.5 Experimentos

Para los experimentos que se van a describir a continuación, se realizaron distintas pruebas hasta encontrar el conjunto de parámetros que arrojó los resultados más interesantes. La implementación del proceso de normalización, filtro, cubierta, la aplicación de la librería `Keppler Mapper` para la construcción del mapper resultante y todos los resultados obtenidos se encuentran en la carpeta `Air` del repositorio `https://github.com/kbotello129/Mapper`.

4.5.1 BASE DE DATOS CRITICA.

Para los experimentos utilizando la base de datos crítica, consideremos el arreglo de tamaño 6989×4 con los datos normalizados de ésta. Filtramos los datos bajo la función descrita en la sección 4.2 con $\sigma = 0.2$ y definimos una cubierta de 20 intervalos con 15% de traslape para el rango de la función filtro aplicada, tal como se describe en la sección 4.3. Así, aplicamos el algoritmo Mapper siguiendo lo mencionado en la sección 4.4, donde tomamos $\texttt{eps} = 0.129$ para el algoritmo de agrupación escogido. Adicionalmente, visualizamos el mapper resultante obteniendo el mostrado en la figura 4.2.

Comenzamos el análisis e interpretación del mapper resultante por los colores obtenidos. Los colores asignados a cada nodo van desde los más oscuros (morado) que representan a los registros buenos o de clasificación 1, hasta los más claros (amarillo) que representan los outliers o de clasificación 6. Es decir, el color de cada nodo ilustra la clasificación numérica de la mayoría de datos presentes en el cluster. Por ejemplo, para el caso de los nodos más oscuros, la mayoría de datos (o todos) de dicho cluster son registros con índice de aire y salud BUENA (figura 4.3), en los nodos de tono un poco más claro como el azul se presenta una cantidad mayor de datos con clasificación 3 o índice de aire y salud MALA (figura 4.4) y en

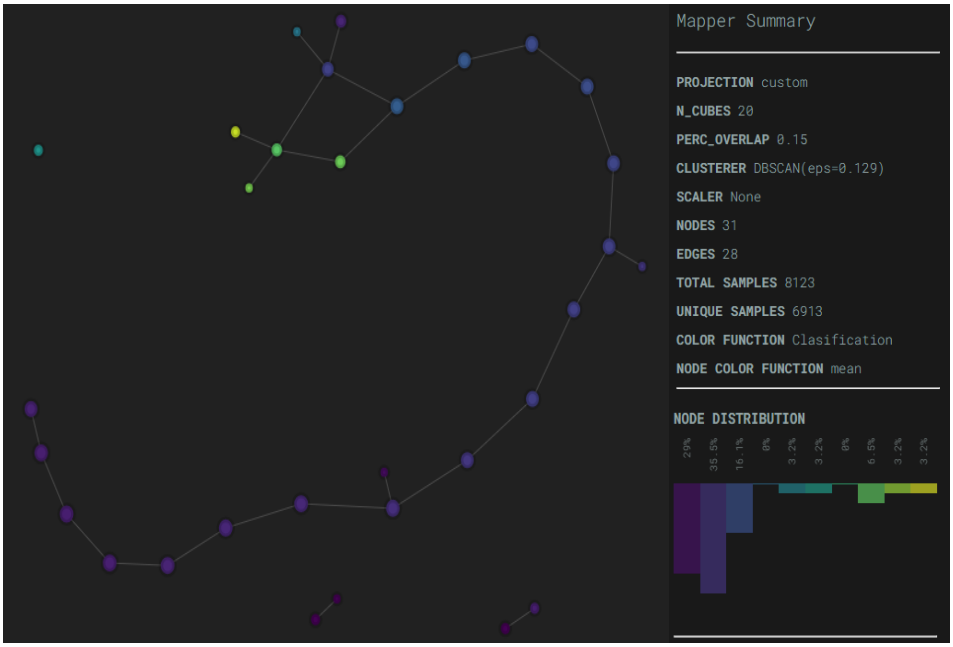


Figura 4.2: Mapper resultante de aplicar el algoritmo Mapper a la base de datos crítica de la calidad del aire con la tupla de parámetros escogida, aplicando el filtro de la densidad gaussiana. El archivo `Mapper_Aire_Critica_cla.html` de este mapper resultante se encuentra en el repositorio referenciado al incio de la sección.

los nodos de tono más claro como el amarillo o el verde se presenta una cantidad mayor de datos con clasificación 5 o 6, es decir índice de aire y salud EXTREMADAMENTE MALA u OUTLIER (figuras 4.5 y 4.6 respectivamente).

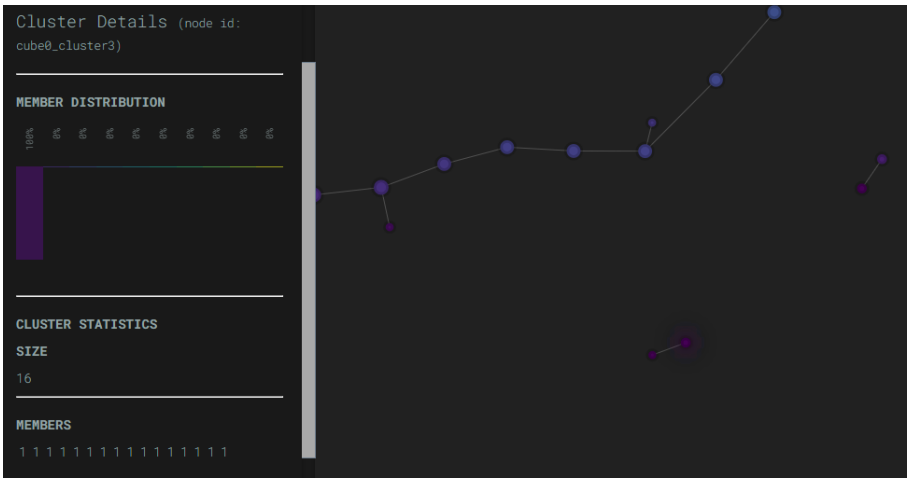


Figura 4.3: Detalles mostrados del nodo seleccionado. Para este caso, el cluster contiene 16 elementos del conjunto de datos los cuales en su totalidad son registros con clasificación 1.

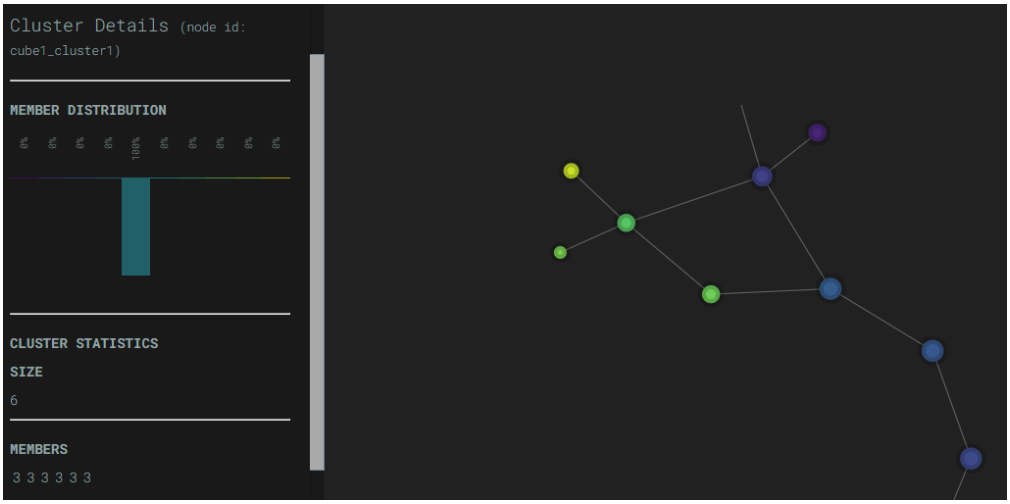


Figura 4.4: Detalles mostrados del nodo seleccionado. Para este caso, el cluster contiene 6 elementos del conjunto de datos los cuales en su totalidad son registros con clasificación 3.

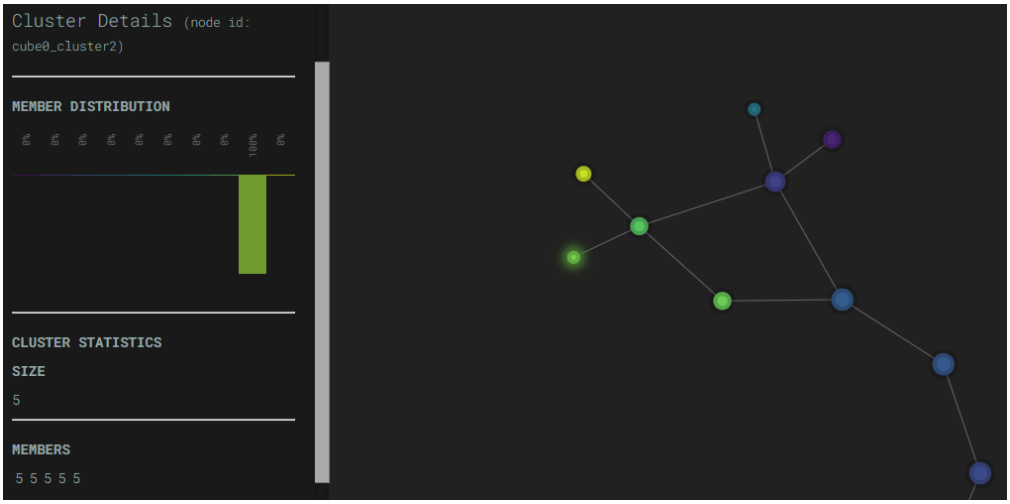


Figura 4.5: Detalles mostrados del nodo seleccionado. Para este caso, el cluster contiene 5 elementos del conjunto de datos los cuales en su totalidad son registros con clasificación 5.

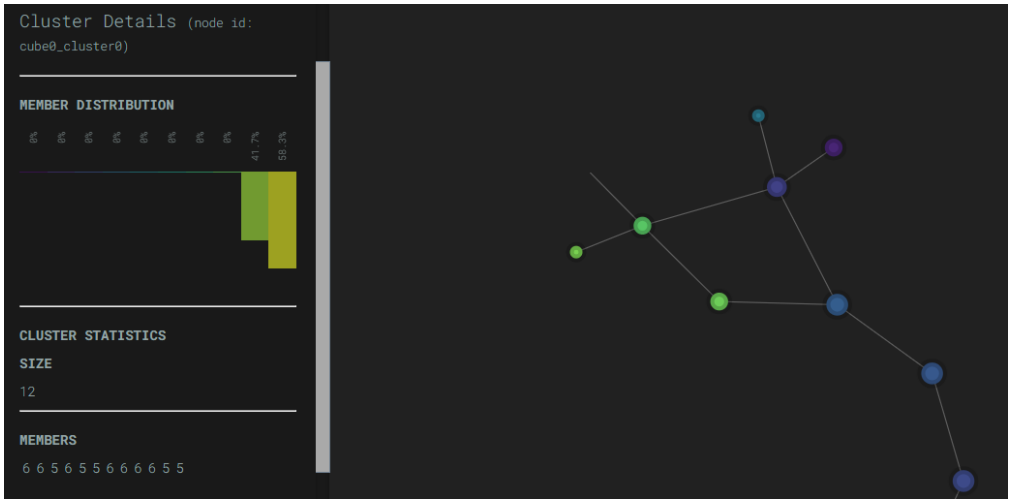


Figura 4.6: Detalles mostrados del nodo seleccionado. Para este caso, el cluster contiene 12 elementos del conjunto de datos los cuales en su mayoría son registros con clasificación 6. Los outliers del conjunto de datos se dan debido a niveles considerablemente más grandes de PM10 y NO2 con respecto a los demás registrados, es por esto que en el cluster se agrupan también datos de clasificación 5.

Al hacer un recorrido por la rama más larga del mapper resultante, observamos que los nodos contienen datos con clasificación 1 y 2 y conforme nos acercamos a donde nace dicha sección del mapper, disminuye el porcentaje de los datos con dichas clasificaciones y aumenta el porcentaje de los datos de clasificación 3, 4 y 5, como se muestra para algunos nodos en la figura 4.7.

Por lo tanto, el mapper resultante en este experimento visualiza y secciona esta clasificación numérica de índice de aire y salud, debido a que las agrupaciones de datos e intersecciones entre agrupaciones se dan por niveles cercanos de uno o más parámetros, o por niveles similares de densidad bajo la función filtro. En la figura 4.8 se muestra el registro por nodo de algunas intersecciones presentes en el mapper resultante. El registro completo por nodos se encuentra en la carpeta del repositorio referenciado al incio de la presente sección. Además, el mapper resultante con los colores asignados de acuerdo a los valores de densidad, es decir asignando a `color_values` el arreglo con los datos filtrados, se muestra en la figura 4.9.

Observamos además que las partes disconexas del mapper resultante se obtienen de preimágenes de elementos de la cubierta con componentes conexas donde predominan clasificaciones diferentes, la figura 4.10 ilustra este comportamiento. Por último, en términos de tiempo se realizó un análisis por nodo y, aunque en algunas secciones del mapper resultante

se agrupan registros de fechas cercanas o de una misma temporada, este comportamiento no se repite en todo el grafo, pues en la rama más larga no sólo se tienen datos con clasificaciones variadas, sino también de todas las horas y temporadas del año.

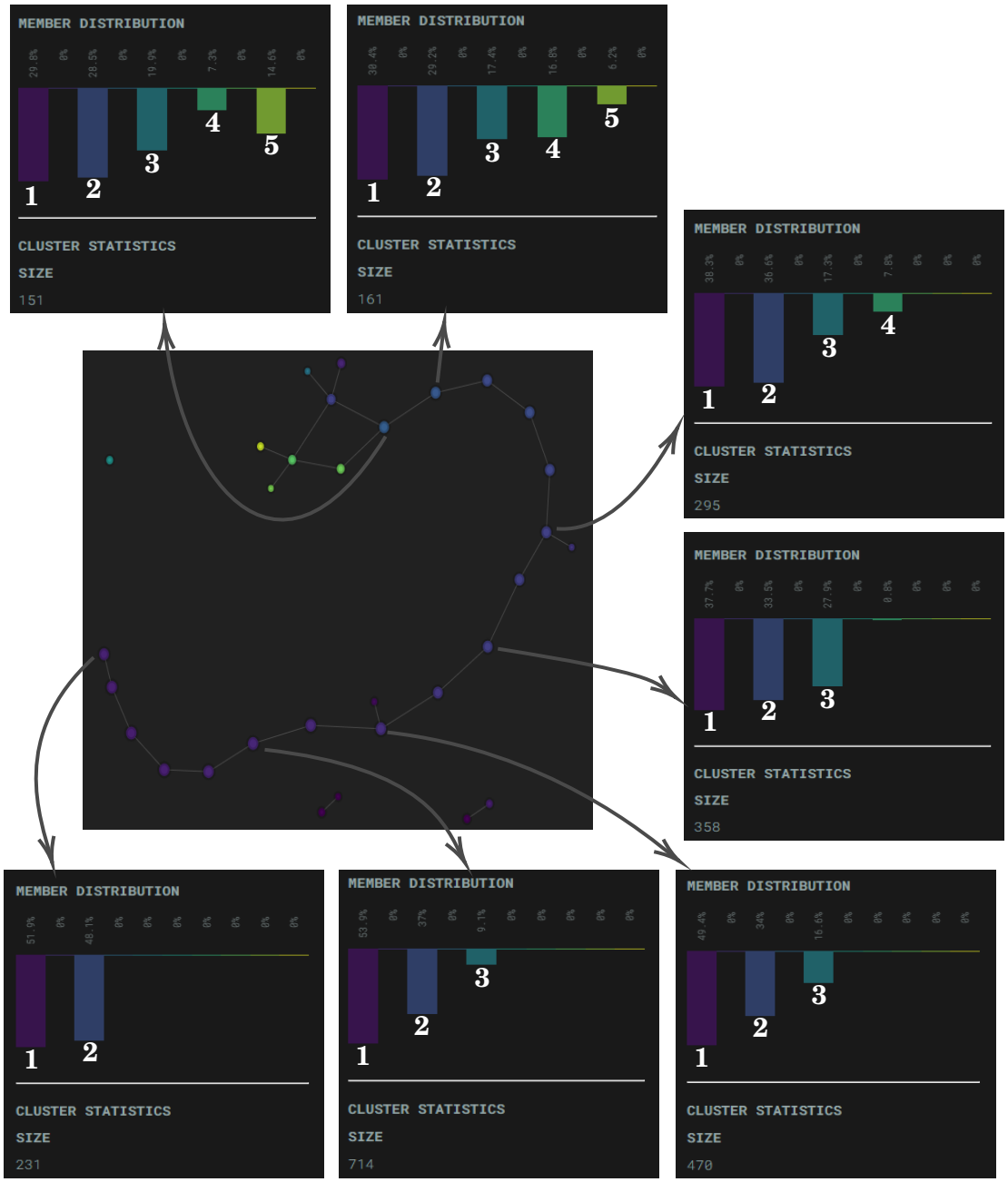


Figura 4.7: Distribución de los datos agrupados en cada cluster señalado y su tamaño. El número en cada barra representa la clasificación numérica de los datos representados por ésta.

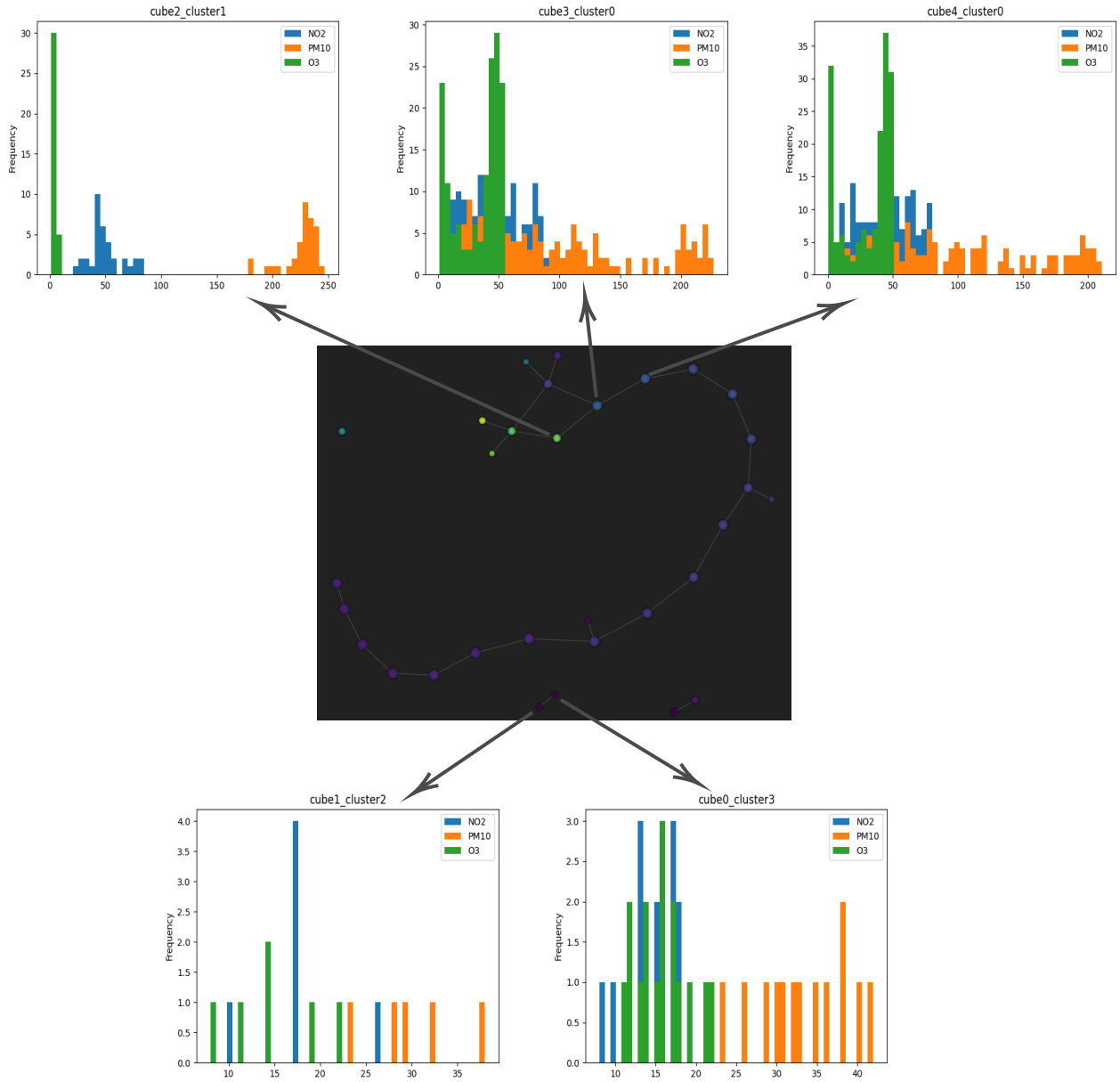


Figura 4.8: Registro de contaminantes para cada nodo señalado. El eje horizontal mide los niveles registrados de cada parámetro y el eje vertical la frecuencia de dichos niveles en el cluster. Para el caso del nodo cube2_cluster1 y cube3_cluster0 (parte superior), ambas gráficas muestran niveles cercanos a 200 de PM10, niveles cercanos a 50 de NO2 y niveles menores a 50 de O3, de donde surge la intersección entre estas dos agrupaciones de datos. Para el caso del nodo cube3_cluster0 y cube4_cluster0 estos registros comunes se hacen más notorios. En la parte inferior, para el caso de la intersección entre el nodo cube1_cluster2 y cube0_cluster3, se observan niveles similares de los tres parámetros medidos.

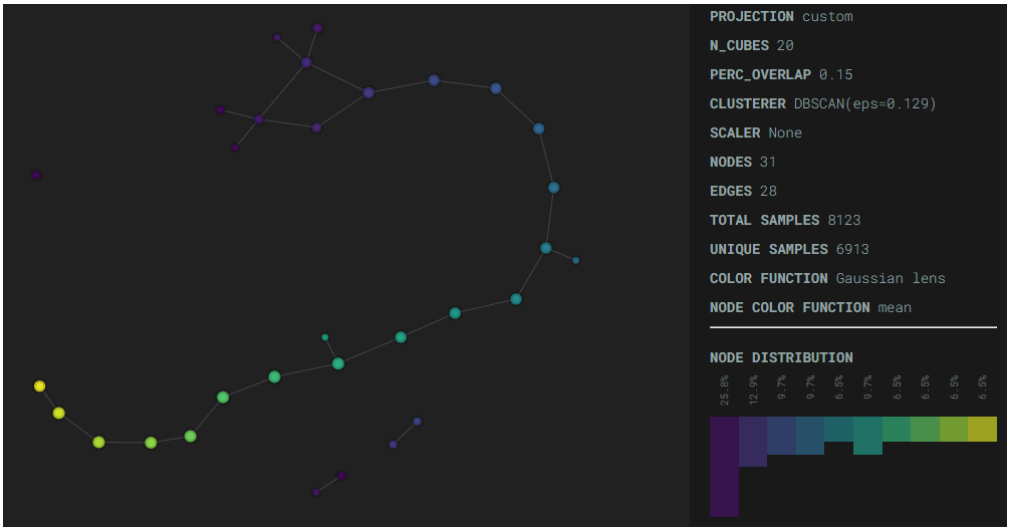


Figura 4.9: Mapper resultante con colores asignados de acuerdo a la función filtro aplicada. En la rama más larga del mapper que contiene nodos con clasificaciones variadas y son coloreados con tonos muy similares en la figura 4.2, acá muestran que su distribución y ubicación en el mapper resultante es debido a sus valores de densidad bajo la función filtro del Kernel gaussiano.

Figura 4.10: Los nodos señalados resultan de las preimágenes de los elementos de la cubierta cube0, cube1, cube3 y cube4. La preimagen de cube0, por ejemplo, tiene 4 componentes conexas que resultan en los nodos cube0_cluster0, cube0_cluster1, cube0_cluster2 y cube0_cluster3 los cuales (por su color) contienen agrupaciones de datos en donde predominan clasificaciones diferentes.

Base de datos crítica por estación. Como se mencionó al inicio del capítulo, la base de datos crítica contiene registros de 5 estaciones de monitoreo. Consideremos ahora 5 subbases de datos de la base de datos crítica con los registros de cada estación por separado y apliquemos el algoritmo Mapper utilizando los mismos parámetros. De esta forma, obtenemos un mapper resultante por estación, que en conjunto con el mapper resultante anterior permite complementar el análisis y obtener conclusiones más precisas con base a los resultados obtenidos.

Nota. El archivo .html con los siguientes mappers resultantes y su información se encuentran en la carpeta del repositorio mencionada al inicio de la presente sección.

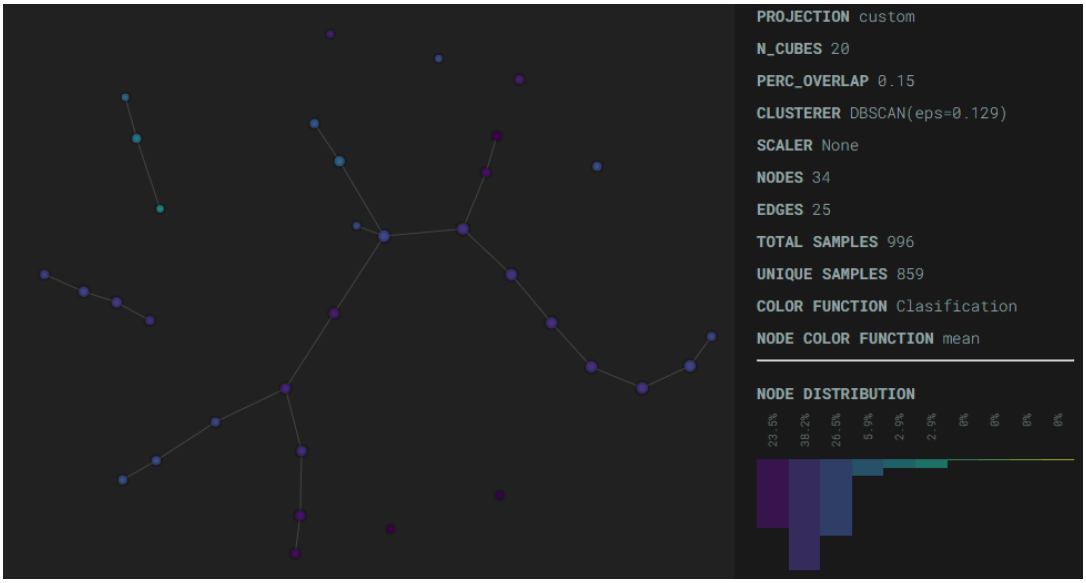


Figura 4.11: Mapper resultante de la subbase de datos de la estación de Tlalnepantla (TLA). Los nodos de color más claro que indican un índice mayor muestran una clasificación máxima de 3 (mala).

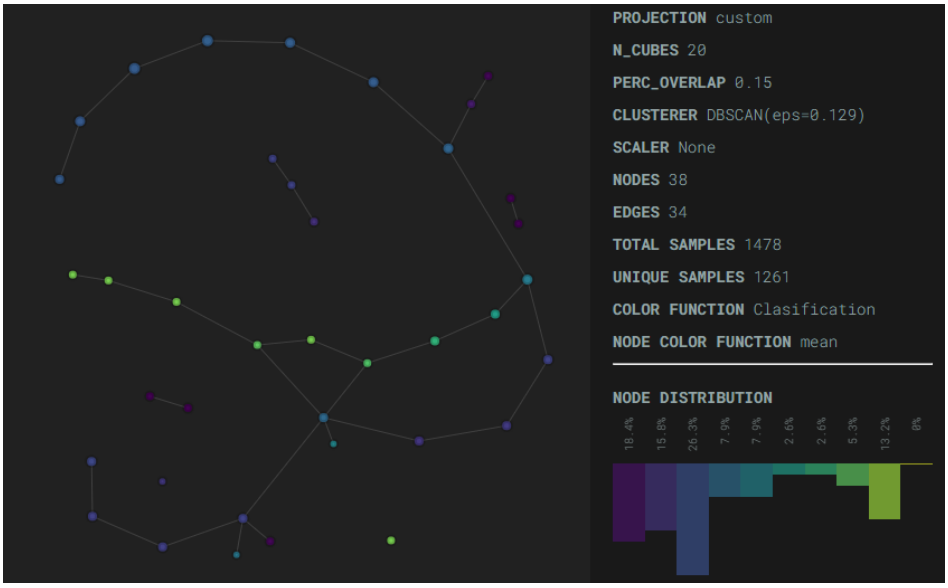


Figura 4.12: Mapper resultante de la subbase de datos de la estación de Xalostoc (XAL). Los nodos de color más claro que indican un índice mayor muestran una clasificación máxima de 5 (extremadamente mala). Esta estación contiene los 10 registros de la base de datos crítica con clasificación 6, sin embargo, para la tupla de parámetros utilizada el algoritmo no los agrupó en un solo cluster y por lo tanto no fueron tenidos en cuenta para construir el mapper resultante al no cumplir la cantidad mínima requerida por cluster.

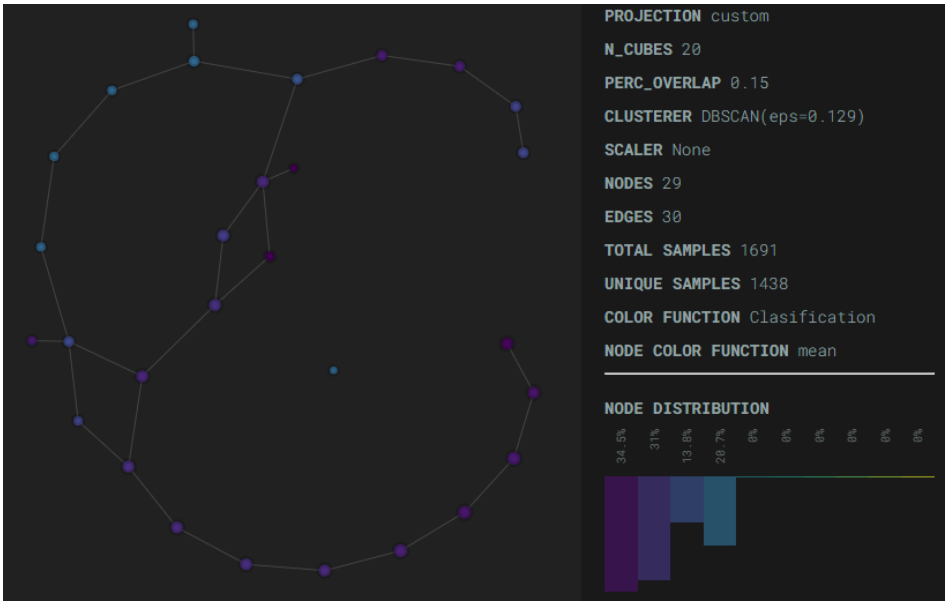


Figura 4.13: Mapper resultante de la subbase de datos de la estación de Miguel Hidalgo (MGH). Los nodos de color más claro que indican un índice mayor muestran una clasificación máxima de 2 (aceptable).

Figura 4.14: Mapper resultante de la subbase de datos de la estación de Cuajimalpa (CUA). Los nodos de color más claro que indican un índice mayor muestran una clasificación máxima de 2 (aceptable).

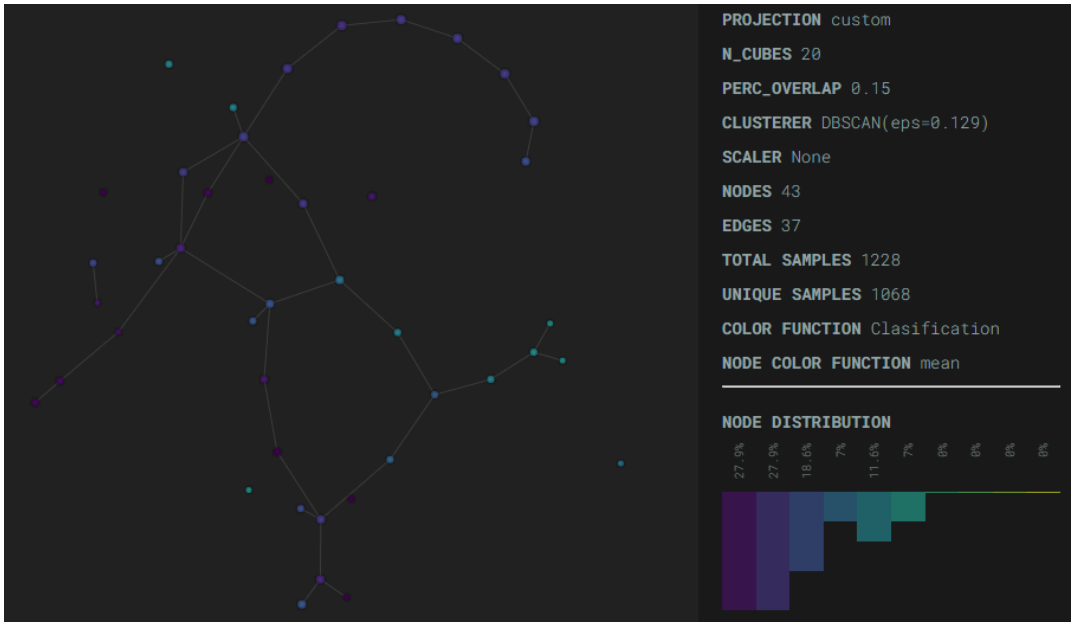


Figura 4.15: Mapper resultante de la subbase de datos de la estación de Merced (MER). Los nodos de color más claro que indican un índice mayor muestran una clasificación máxima de 3 (mala).

Al realizar las pruebas con los mismos parámetros y obtener el mapper resultante por es-

tación, el algoritmo Mapper realiza un buen trabajo de clasificación en algunas secciones de los índices de aire y salud, algunos con resultados más favorables que otros en términos de calidad del aire. En el caso de las estaciones MGH y CUA, los resultados obtenidos sugieren que durante el año 2018 en el intervalo de tiempo de las 6 AM a las 10 AM, las locaciones donde se encuentran ubicadas estas estaciones tuvieron una mejor calidad del aire en términos de los parámetros estudiados, por lo que estaciones como XAL, TLA o MER podrían requerir más atención y análisis pues arrojaron resultados más críticos.

Estos resultados obtenidos pueden deberse a múltiples factores como la ubicación, cantidad de tráfico cerca de las estaciones o de árboles en la zona, situaciones atípicas como incendios, entre otros. Poner en contexto o realizar un análisis detallado por estación, por nodos en términos de tiempo o de parámetros complementa, justifica y le da sentido al mapper obtenido en cada experimento.

4.5.2 **BASE DE DATOS GENERAL.** Para los experimentos utilizando la base de datos general, procedemos de manera análoga al experimento anterior considerando el arreglo de tamaño 23548×4 con los datos normalizados de ésta. Filtramos los datos bajo la función descrita en la sección 4.2 con $\sigma = 0.2$ y definimos una cubierta de 30 intervalos con 15% de traslape para el rango de la función filtro aplicada, tal como se describe en la sección 4.3. Así, aplicamos el algoritmo Mapper siguiendo lo mencionado en la sección 4.4, donde tomamos `eps=0.12` para el algoritmo de agrupación escogido. Adicionalmente, visualizamos el mapper resultante obteniendo el mostrado en la figura 4.16.

Para este experimento, se ajustaron ligeramente los parámetros buscando una mejor clasificación en el mapper resultante. Al ser un conjunto de datos significativamente más grande, se aumentaron el número de intervalos en la cubierta y se disminuyó el parámetro del DBSCAN. Para el caso de σ, valores mayores mostraron mejores clasificaciones de los nodos de color claro, es decir los que contienen datos con índices de aire y salud mayor, sin embargo no fue favorable en la clasificación de los datos con índice de aire y salud menor. Por otro lado, valores menores de σ tuvieron el efecto contrario con una clasificación mejor para los datos con índice de aire y salud menor pero desfavorable para los datos con índices entre 4 y 6. Por este motivo, dicho parámetro no fue modificado para el experimento mostrado.

Como se observa en la figura 4.17, el mapper resultante clasifica de manera exitosa datos con clasificación numérica de 3 en la rama que contiene al nodo cube4_cluster1 y agrupa en la rama que contiene a los nodos cube0_cluster0 y cube2_cluster0 datos con clasificación 3, 4,

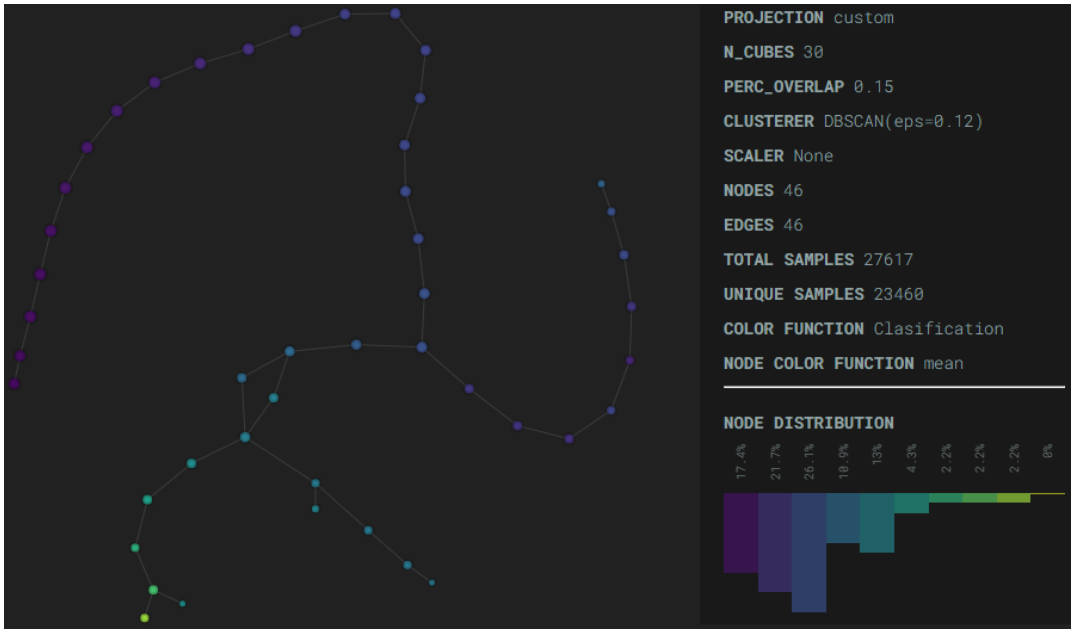


Figura 4.16: Mapper resultante de aplicar el algoritmo Mapper a la base de datos general de la calidad del aire con los parámetros escogidos, aplicando el filtro de la densidad gaussiana. El archivo `Mapper_Aire_General_cla.html` de este mapper resultante se encuentra en el repositorio referenciado al incio de la sección.

5 y 6. Esta sección del mapper es entonces la que visualiza datos recolectados donde la calidad del aire es mala, muy mala, extremadamente mala o presenta valores atípicos. Por otro lado, la rama superior que contiene al nodo cube29_cluster0 visualiza y agrupa datos con una clasificación numérica menor. En el caso de los nodos cube6_custer1 y cube6_cluster2 señalados, el algoritmo Mapper identifica y separa en dos nodos las componentes conexas de la preimagen del intervalo cube6, en donde predominan clasificaciones numéricas diferentes. Las secciones restantes agrupan clasificaciones numéricas variadas, aunque también interesantes pues en todos los experimentos realizados, incluso en agrupaciones con índices de aire y salud variados éstas no presentaron inconsistencias o agrupaciones “anormales” (como por ejemplo, un cluster de datos con clasificación 1 y 6 únicamente).

Por lo tanto, el mapper resultante de este experimento logra de manera análoga lo que se observó en el experimento anterior, con un conjunto de datos más grande y con una tupla de parámetros sin mayores variaciones. De manera similar, el mapper resultante no muestra una clasificación clara o a resaltar en términos de tiempo por lo que las agrupaciones de datos e intersecciones se dan por niveles cercanos de uno o más parámetros y principalmente por valores similares de densidad. La figura 4.18 muestra el mapper resultante con la asignación

de colores de acuerdo a la función filtro aplicada. El registro completo por nodos se encuentra en la carpeta del repositorio referenciado al inicio de la presente sección.

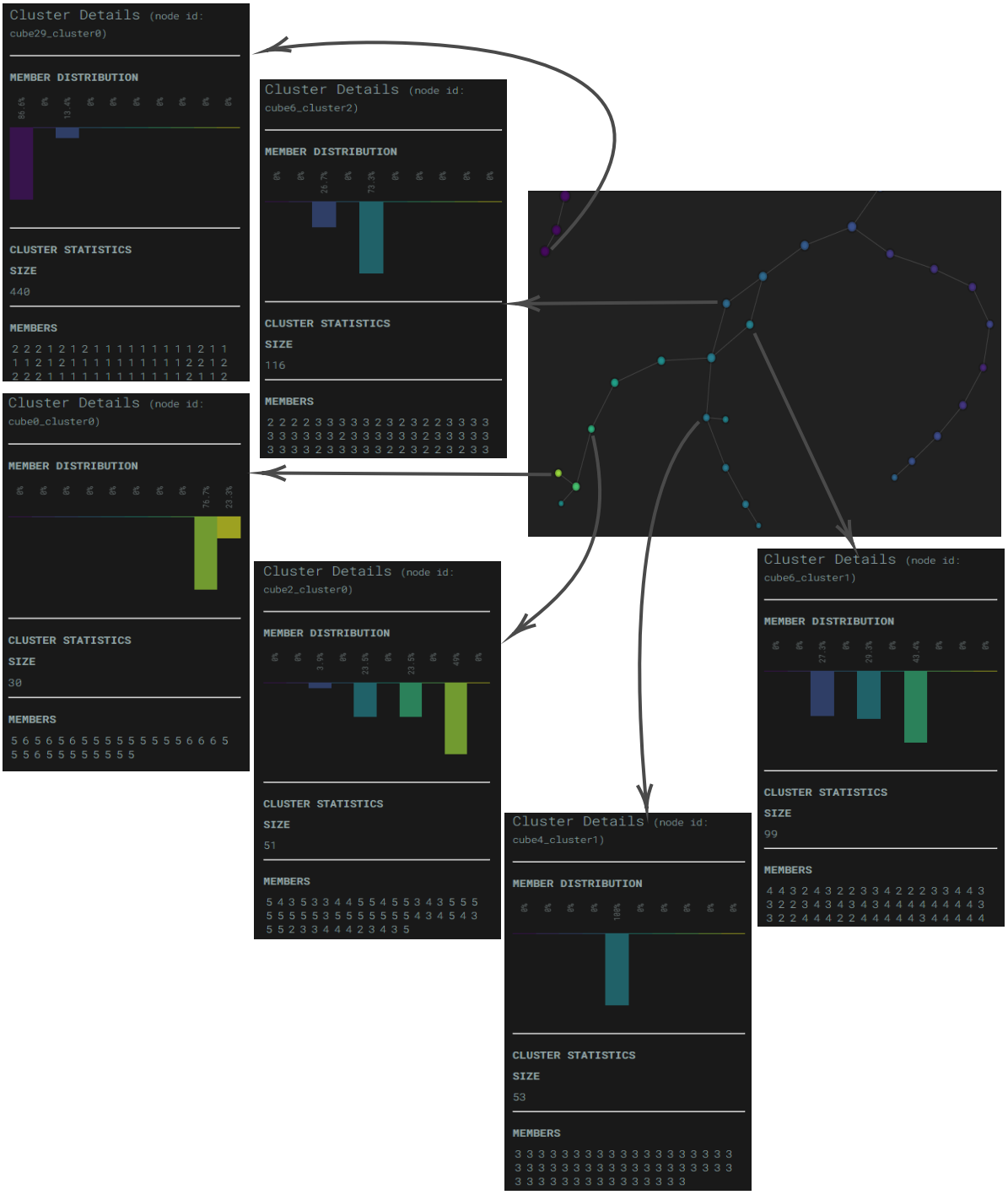


Figura 4.17: Información detallada de los nodos señalados del mapper resultante.

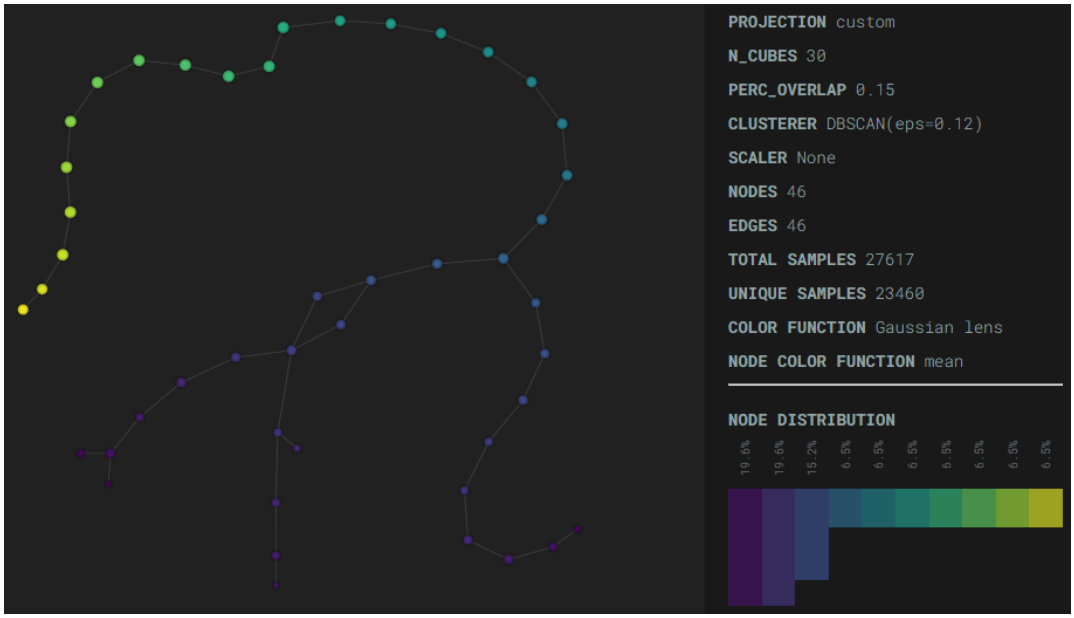


Figura 4.18: Mapper resultante con colores asignados de acuerdo a los valores de densidad bajo la función filtro del Kernel Gaussiano. Esta figura nos muestra de manera clara que el Mapper realiza una clasificación de acuerdo a estos valores y, aunque valores similares de densidad no implican una clasificación numérica equivalente o cercana, en conjunto con los procesos realizados por el algoritmo Mapper se logra además una clasificación y visualización de los datos más interesante como la obtenida en la figura 4.16.

Base de datos general por estación. Consideremos ahora las 5 subbases de datos de la base de datos general con los registros de cada estación por separado para obtener los mappers resultantes por estación, utilizando la misma tupla de parámetros.

Nota. El archivo .html con los mappers resultantes y su información se encuentran en la carpeta del repositorio mencionada al inicio de la presente sección.

Al realizar las pruebas con los mismos parámetros y obtener el mapper resultante por estación, observamos que de manera esperada, una mayor recolección de datos muestra índices máximos de aire y salud mayores por estación. Los resultados sugieren que, como se observó en el experimento anterior, la estación de Xalostoc presentó niveles de contaminación del aire alarmantes en comparación a las demás estudiadas. Adicionalmente, los mappers resultantes de las estaciones TLA, MGH y CUA presentan clasificaciones numéricas máximas mayores a las obtenidas en los experimentos por estación de la base de datos crítica, por lo que se infiere que la mayoría de los registros donde se presenta una menor calidad del aire en dichas locaciones se dan en el intervalo de las 10 AM a las 10 PM.

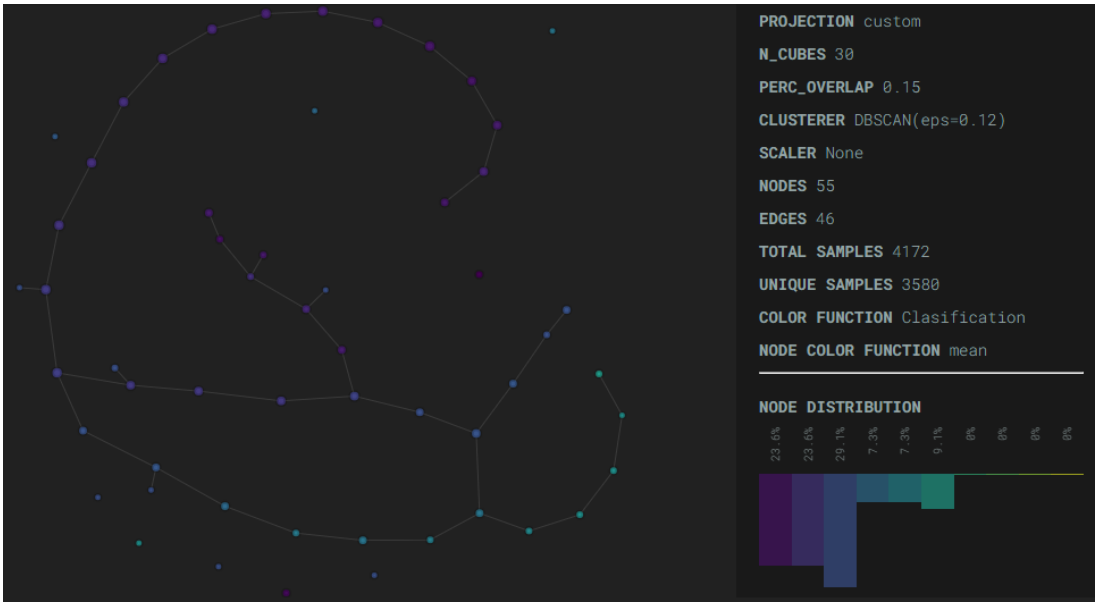


Figura 4.19: Mapper resultante de la subbase de datos de la estación de Tlalnepantla (TLA). Los nodos de color más claro que indican un índice mayor muestran una clasificación máxima de 4 (muy mala).

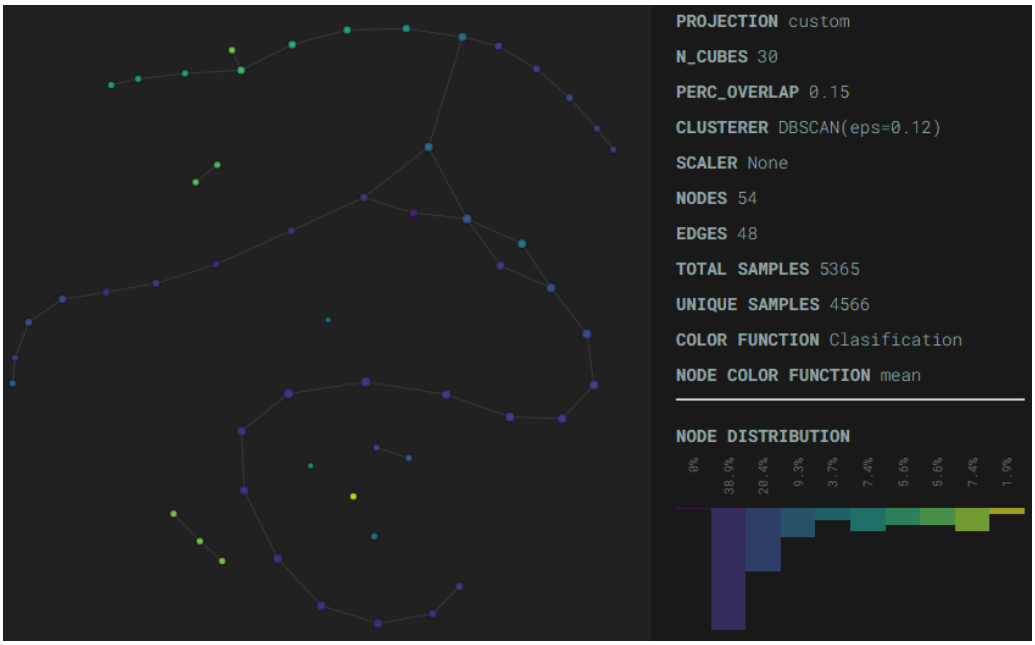


Figura 4.20: Mapper resultante de la subbase de datos de la estación de Xalostoc (XAL). Los nodos de color más claro que indican un índice mayor muestran una clasificación máxima de 6 (outliers).

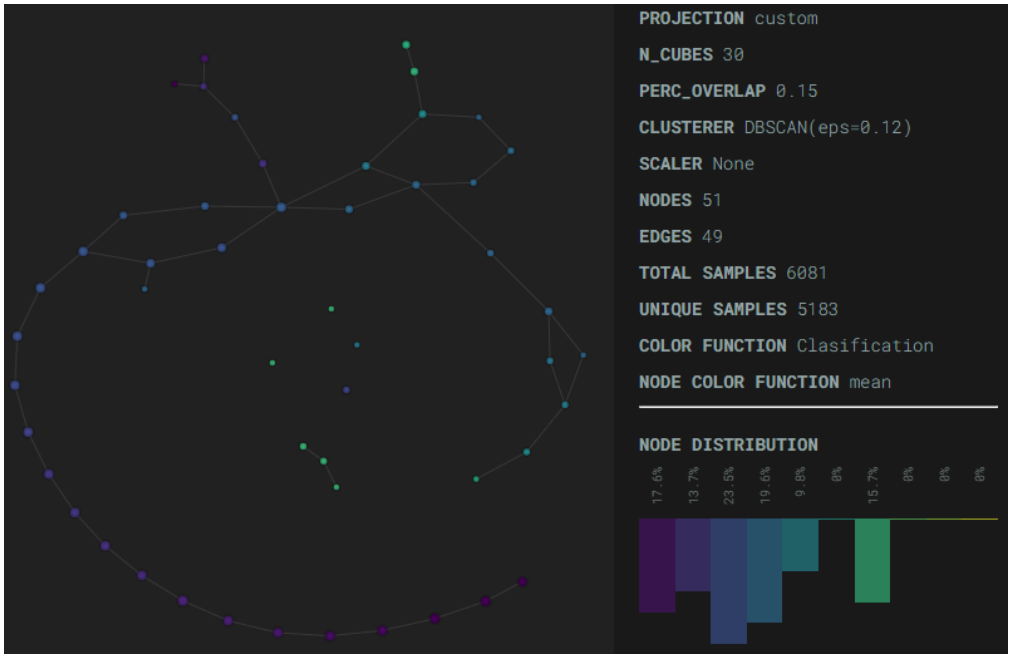


Figura 4.21: Mapper resultante de la subbase de datos de la estación de Miguel Hidalgo (MGH). Los nodos de color más claro que indican un índice mayor muestran una clasificación máxima de 3 (mala).

Figura 4.22: Mapper resultante de la subbase de datos de la estación de Cuajimalpa (CUA). Los nodos de color más claro que indican un índice mayor muestran una clasificación máxima de 3 (mala).

Figura 4.23: Mapper resultante de la subbase de datos de la estación de Merced (MER). Los nodos de color más claro que indican un índice mayor muestran una clasificación máxima de 3 (mala).

4.6 Resultados obtenidos

Como se observó en los experimentos realizados, el mapper resultante nos muestra una distribución de los niveles de calidad del aire de las estaciones estudiadas durante el año escogido. Éste logra visualizar y separar por ramas o secciones agrupaciones de datos donde predomina alguna clasificación numérica. El mapper resultante brinda un panorama sobre la calidad del aire en las estaciones estudiadas que permite tener una idea sobre el comportamiento de los agentes contaminantes en la ciudad durante el año estudiado. Un análisis por nodos o por secciones nos ayuda a interpretar y reconocer los resultados pues nos muestra qué parámetros influyen en la clasificación numérica predominante y a qué época del año o intervalo de tiempo pertenecen. Este análisis cobra más sentido en los mappers resultantes por estación, pues además de lo mencionado brindan una ubicación específica de los datos. Consideremos por ejemplo a la estación de Xalostoc (XAL) la cual en los experimentos arrojó los resultados más críticos en términos de calidad del aire y analicemos el nodo que contiene las clasificaciones numéricas más altas, mostrado en la figura 4.24. La gráfica que muestra el registro de contaminantes para dicho nodo (figura 4.25) evidencia que los datos presentan una clasificación numérica de 5 o 6 principalmente por niveles muy altos de

PM10, algunos superando una medida de 300. Además, las gráficas que muestran el registro de los meses y las horas en las que fueron tomados los datos presentes en el nodo señalado (figuras 4.26 y 4.27 respectivamente) evidencian que estos niveles de contaminación en el aire se obtuvieron durante la época de invierno en horas de la mañana (7 y 8 AM), donde se suele presentar mayor cantidad de tráfico por hora pico. Notemos además que esta estación se encuentra ubicada cerca de una avenida de alto flujo vehicular y en una zona industrial, por lo que descartando situaciones atípicas, estos resultados cobran sentido y esta estación (por su ubicación) está expuesta a presentar una calidad del aire deficiente principalmente a ciertas horas o en ciertas épocas.

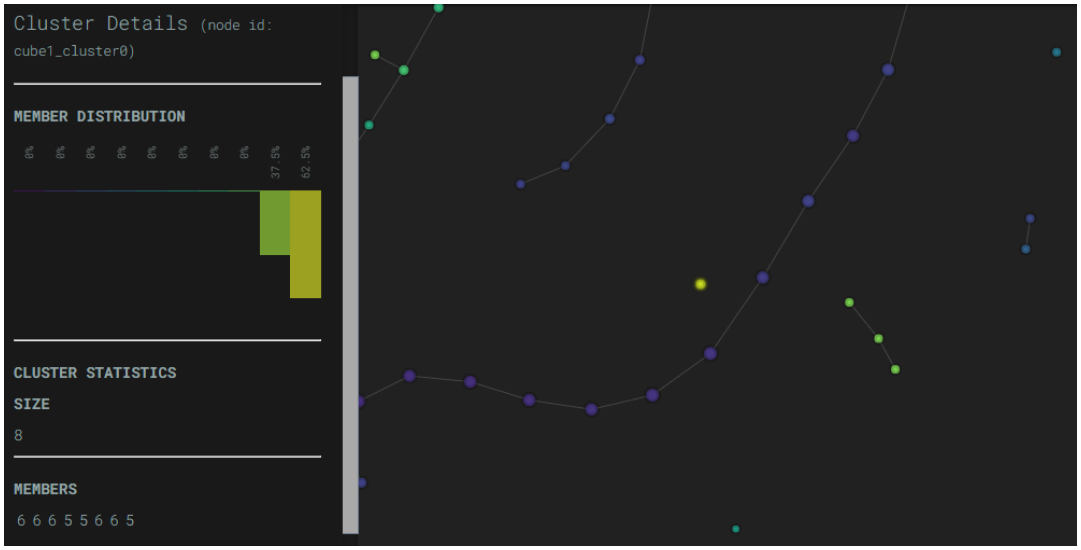


Figura 4.24: Detalles del nodo cube1_cluster0 del mapper resultante de la subbase de datos de la estación de Xalostoc.

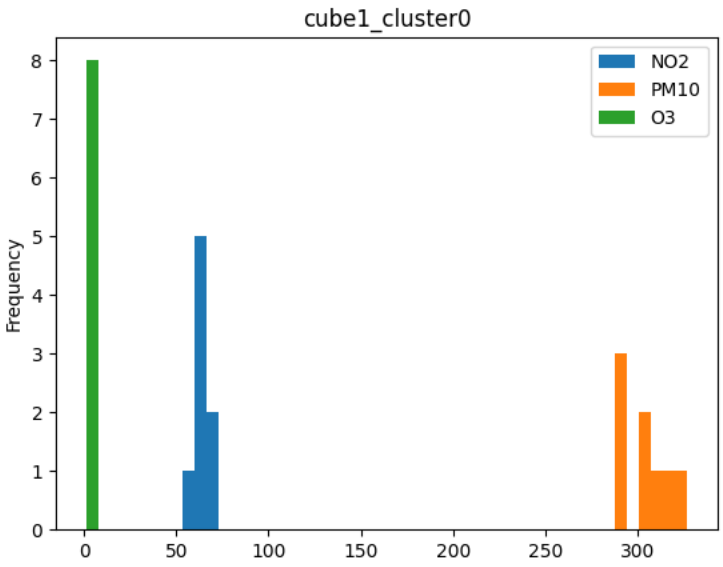


Figura 4.25: Registro de contaminantes del nodo cube1_cluster0 del mapper resultante de la subbase de datos de la estación XAL. El eje horizontal mide el nivel de contaminación registrado y el eje vertical la cantidad de datos que presentaron tal nivel.

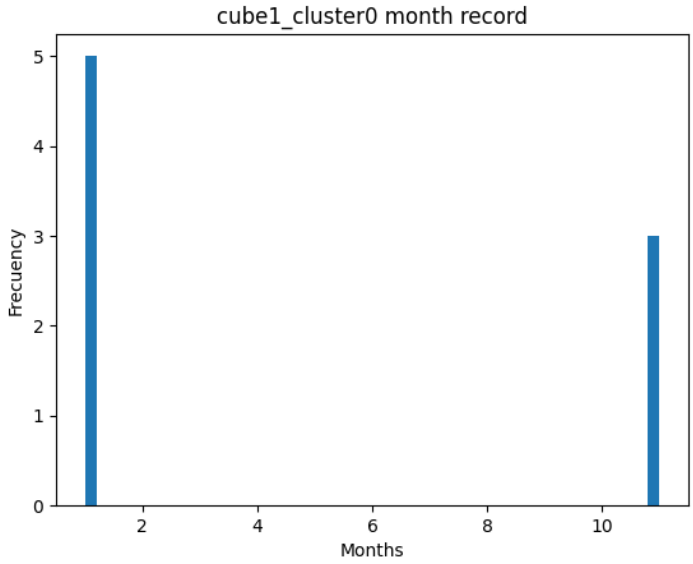


Figura 4.26: Registro de los meses en los que fueron tomados los datos presentes en el nodo cube1_cluster0 del mapper resultante de la subbase de datos de la estación de Xalostoc. Como se observa, los 8 datos agrupados en este cluster fueron tomados durante los meses de Noviembre y Enero.

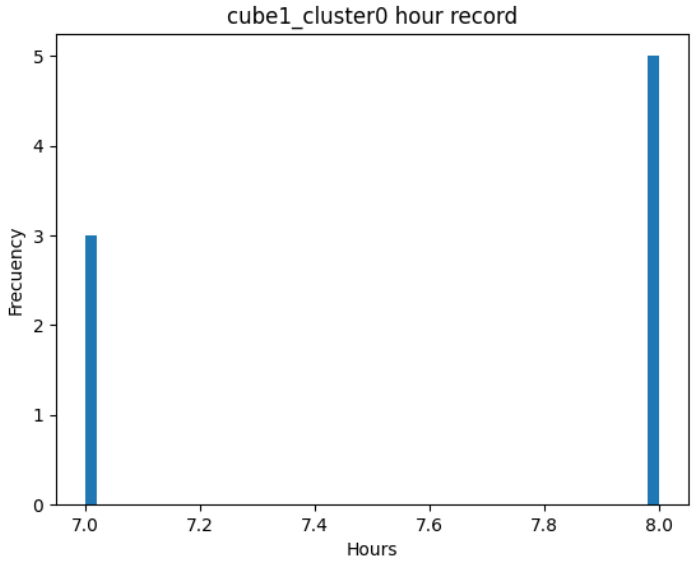


Figura 4.27: Registro de las horas en las que fueron tomados los datos presentes en el nodo cube1_cluster0 del mapper resultante de la subbase de datos de la estación de Xalostoc. Como se observa, los datos agrupados en este cluster fueron tomados a las 7 u 8 de la mañana.

Al realizar un estudio similar al que se detalló en el presente capítulo con bases de datos de otros años, con otros parámetros u otras estaciones, el Mapper puede arrojar resultados sumamente útiles, visualizando el comportamiento de conjuntos con millones de datos en un grafo (o complejo simplicial) más amigable e intuitivo para la toma de decisiones en términos de planeación de estrategias que permitan aminorar el impacto en la salud, la prevención de estos niveles en el futuro u otros aspectos a tener en cuenta con los resultados obtenidos. La figura 4.28 muestra una prueba adicional realizada para una base de datos de un año posterior y una estación adicional. Un ajuste en la tupla de parámetros podría mejorar este resultado y realizar una mejor clasificación, así como un ajuste en el conjunto de datos que involucre en sus columnas información como el tiempo o características geográficas de las estaciones podría implicar a su vez otro tipo de clasificación y resultados útiles e interesantes en el mapper resultante. Este problema crea un abanico de posibilidades y queda abierto para futuros estudios y experimentos.

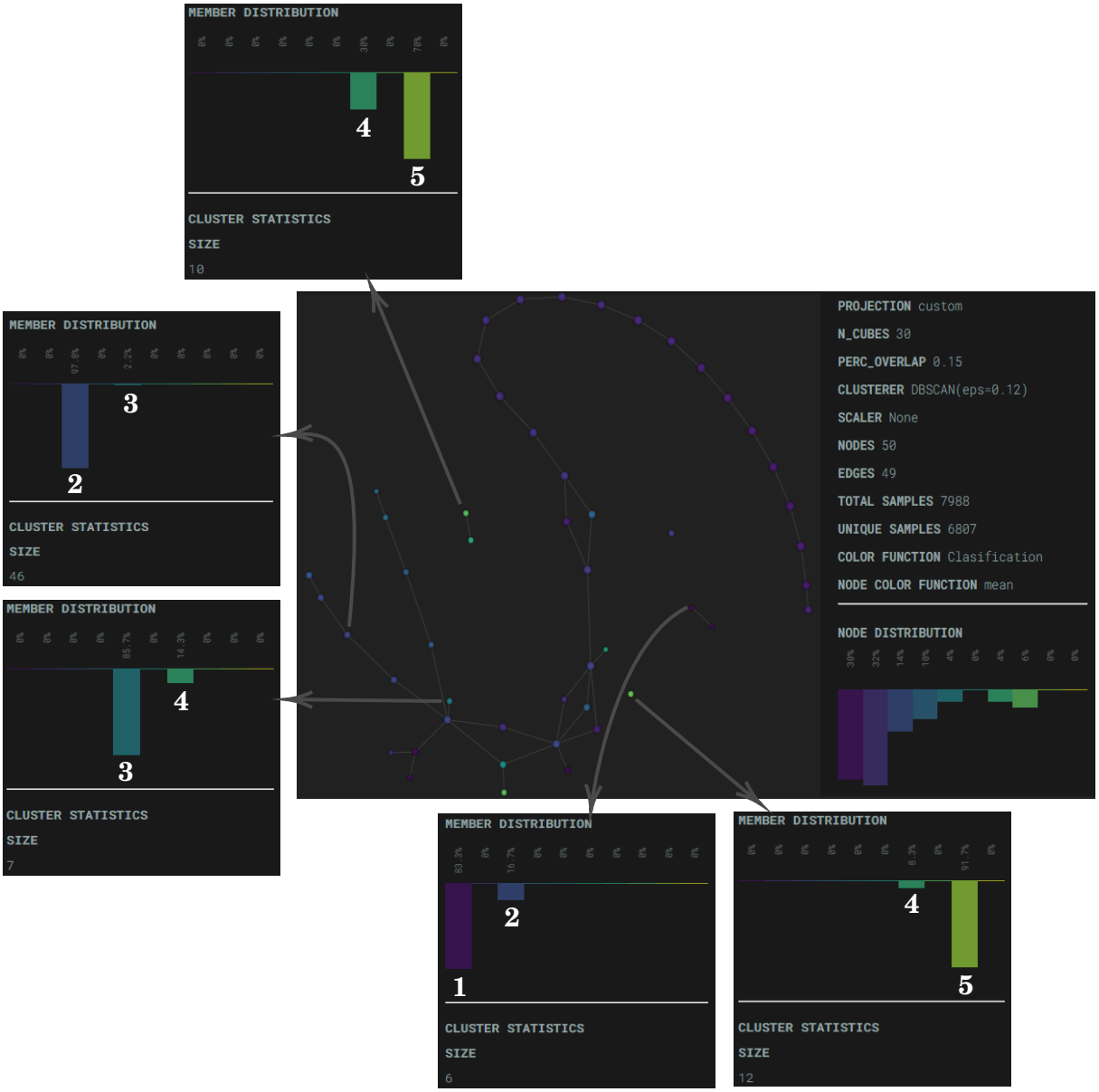


Figura 4.28: Se realizó una prueba adicional para una base de datos del año 2019, con datos de las estaciones CUA, XAL, TLA, GAM (Gustavo A. Madero), MER y MGH en el intervalo de las 6 AM y las 10 AM. Se consideraron los mismos parámetros de contaminación y la misma tupla de parámetros en el algoritmo utilizada para la base de datos general. Este es el mapper resultante y algunos nodos donde el algoritmo logró una clasificación numérica predominante.

5

CONCLUSIONES

En el desarrollo de este trabajo se planteó como objetivo principal el conocimiento y la implementación de una de las técnicas del análisis topológico de datos, el algoritmo Mapper, mediante la repetición de experimentos realizados en su presentación y la búsqueda de resultados interesantes para conjuntos de datos propuestos en este mismo trabajo. Adoptando conceptos y resultados de la topología algebraica y el análisis topológico de datos, se explicó y tomó sentido el algoritmo Mapper. Haciendo uso del lenguaje de programación Python, el cual ofrece librerías y clases útiles para su implementación, en las que se encuentran algoritmos de agrupación como el DBSCAN, normalizadores como el `MinMaxScaler` y una librería con documentación completa como `Keppler Mapper`, se realizaron tres aplicaciones del algoritmo Mapper a diferentes conjuntos de datos.

Siguiendo lo propuesto por Singh, Mémoli y Carlsson en [1], se realizaron dos experimentos para el conjunto de datos de la diabetes con adaptaciones propuestas en este trabajo para el proceso de normalización, la escogencia de algunos parámetros y la implementación del algoritmo Mapper el cual fue programado para estos experimentos, obteniendo así resultados favorables en los mappers resultantes "low resolution" y "high resolution" al compararlos con los mostrados por los autores. De manera similar y adaptando el algoritmo programado para aceptar una cubierta 2−dimensional, implementado de tal manera que construya mappers resultantes con simplejos máximo de dimensión 2 a manera de mejorar los resultados en términos de dimensión y visualización, se realizaron dos experimentos para conjuntos de datos sobre la superficie del toro basados en uno realizado por los autores en [1]. Se generaron dos conjuntos de datos (uno de manera aleatoria y otro siguiendo el propuesto por los autores) para dos experimentos, el primero aplicado al conjunto de datos aleatorio utilizando un filtro más intuitivo de proyección, donde se obtuvo un mapper resultante favorable

pues este conservó la topología del toro. El segundo experimento se aplicó a un conjunto de datos con características sugeridas por los autores, al igual que la función filtro utilizada, el cual también arrojó resultados favorables. Finalmente, se realizaron cuatro experimentos propuestos en este trabajo sobre dos conjuntos de datos de la calidad del aire en la Ciudad de México utilizando la librería `Keppler Mapper`. De esta forma, el algoritmo Mapper mostró una clasificación de los datos en términos de índices de aire y salud proporcionando una visualización de estos índices en el mapper resultante general y por estación. Se realizó además un reporte por nodos de contaminantes y tiempo para posibilitar el análisis del mapper obtenido.

Como trabajo futuro se puede dar continuidad a los experimentos mostrados del conjunto de datos del toro, utilizando la base de datos aleatoria y el filtro del Laplaciano, el cual requiere una nueva sintonización de parámetros no lograda para este trabajo. Para esto, estudiar y aplicar los métodos de selección de parámetros propuestos en [38] podría arrojar resultados interesantes que además pueden implementarse en los demás experimentos de este trabajo y futuros experimentos tanto 1−dimensionales [37], [28], [16] como multidimensionales [38]. Esto ahorraría tiempo entre pruebas y proporcionaría una sintonización de parámetros automática. Por último, los experimentos mostrados sobre la base de datos de la calidad del aire plantean la posibilidad de futuros experimentos sobre diferentes bases de datos, con otros parámetros, otras estaciones, otros años u otras características relevantes, además de la implementación de otras funciones filtro 1−dimensionales o multidimensionales. En general, para futuras investigaciones se puede considerar la modificación en alguno (o algunos) de los procesos relevantes del algoritmo Mapper (Normalización, agrupación, filtro, cubierta) que resulten favorables en términos de visualización y clasificación de los datos, que a su vez sean útiles para el análisis y toma de decisiones con base a éstos.

BIBLIOGRAFÍA

[1] G. Singh, F. Mémoli, G. E. Carlsson, *et al.*, "Topological methods for the analysis of high dimensional data sets and 3d object recognition.," *PBG@ Eurographics*, vol. 2, pp. 091–100, 2007.

[2] Edelsbrunner, Letscher, and Zomorodian, "Topological persistence and simplification," *Discrete & Computational Geometry*, vol. 28, pp. 511–533, 2002.

[3] A. Zomorodian and G. Carlsson, "Computing persistent homology," *Discrete and Computational Geometry*, vol. 33, 2005.

[4] F. Chazal and B. Michel, "An introduction to topological data analysis: fundamental and practical aspects for data scientists," *Frontiers in artificial intelligence*, vol. 4, p. 108, 2021.

[5] T. K. Dey and Y. Wang, *Computational topology for data analysis*. Cambridge University Press, 2022.

[6] P. J. Huber, "Projection pursuit," *The annals of Statistics*, pp. 435–475, 1985.

[7] B. W. Silverman, *Density estimation for statistics and data analysis*, vol. 26. CRC press, 1986.

[8] M. Belkin and P. Niyogi, "Laplacian eigenmaps for dimensionality reduction and data representation," *Neural computation*, vol. 15, no. 6, pp. 1373–1396, 2003.

[9] H. Abdi, "Metric multidimensional scaling (mds): analyzing distance matrices," *Encyclopedia of measurement and statistics*, pp. 1–13, 2007.

[10] J. B. Tenenbaum, V. d. Silva, and J. C. Langford, "A global geometric framework for nonlinear dimensionality reduction," *science*, vol. 290, no. 5500, pp. 2319–2323, 2000.

[11] M. Nicolau, A. J. Levine, and G. Carlsson, "Topology based data analysis identifies a subgroup of breast cancers with a unique mutational profile and excellent survival," *Proceedings of the National Academy of Sciences*, vol. 108, no. 17, pp. 7265–7270, 2011.

[12] Y. Chen and I. Volić, "Topological data analysis model for the spread of the coronavirus," *Plos one*, vol. 16, no. 8, p. e0255584, 2021.

[13] Y. Yao, J. Sun, X. Huang, G. R. Bowman, G. Singh, M. Lesnick, L. J. Guibas, V. S. Pande, and G. Carlsson, "Topological methods for exploring low-density states in biomolecular folding pathways," *The Journal of chemical physics*, vol. 130, no. 14, 2009.

[14] A. Robles, M. Hajij, and P. Rosen, "The shape of an image: A study of mapper on images," *arXiv preprint arXiv:1710.09008*, 2017.

[15] M. Alagappan, "From 5 to 13: Redefining the positions in basketball," in *2012 MIT Sloan Sports Analytics Conference. http://www. sloansportsconference. com*, 2012.

[16] N. Chalapathi, Y. Zhou, and B. Wang, "Adaptive covers for mapper graphs using information criteria," in *2021 IEEE International Conference on Big Data (Big Data)*, pp. 3789–3800, IEEE, 2021.

[17] G. Carlsson, "Topology and data," *Bulletin of the American Mathematical Society*, vol. 46, no. 2, pp. 255–308, 2009.

[18] H. J. van Veen, N. Saul, D. Eargle, and S. W. Mangham, "Keppler mapper: A flexible python implementation of the mapper algorithm.," *Journal of Open Source Software*, vol. 4, no. 42, p. 1315, 2019.

[19] G. Tauzin, U. Lupo, L. Tunstall, J. B. Pérez, M. Caorsi, A. M. Medina-Mardones, A. Dassatti, and K. Hess, "giotto-tda: A topological data analysis toolkit for machine learning and data exploration," *The Journal of Machine Learning Research*, vol. 22, no. 1, pp. 1834–1839, 2021.

[20] D. Müllner and A. Babu, "Python mapper: An open-source toolchain for data exploration, analysis and visualization." `http://danifold.net/mapper`, 2013.

[21] Y. Zhou, N. Chalapathi, A. Rathore, Y. Zhao, and B. Wang, "Mapper interactive: A scalable, extendable, and interactive toolbox for the visual exploration of high-dimensional data," in *2021 IEEE 14th Pacific Visualization Symposium (PacificVis)*, pp. 101–110, IEEE, 2021.

[22] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[23] J. D. Hunter, "Matplotlib: A 2d graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[24] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using networkx," in *Proceedings of the 7th Python in Science Conference* (G. Varoquaux, T. Vaught, and J. Millman, eds.), (Pasadena, CA USA), pp. 11 – 15, 2008.

[25] F. Moreno, "Técnicas de inteligencia computacional para el análisis de datos de la calidad del aire en la ciudad de méxico," Master's thesis, Instituto Politécnico Internacional, Escuela Superior de Física y Matemáticas, 2022.

[26] E. Munch, "A user's guide to topological data analysis," *Journal of Learning Analytics*, vol. 4, no. 2, pp. 47–61, 2017.

[27] P. Dłotko, "Ball mapper: A shape summary for topological data analysis," *arXiv preprint arXiv:1901.07410*, 2019.

[28] A. Brown, O. Bobrowski, E. Munch, and B. Wang, "Probabilistic convergence and stability of random mapper graphs," *Journal of Applied and Computational Topology*, vol. 5, no. 1, pp. 99–140, 2021.

[29] M. K. Chung, "3. the gaussian kernel." `https://pages.stat.wisc.edu/~mchung/teaching/MIA/reading/diffusion.gaussian.kernel.pdf.pdf`, 2007.

[30] S. Lafon and A. B. Lee, "Diffusion maps and coarse-graining: A unified framework for dimensionality reduction, graph partitioning, and data set parameterization," *IEEE transactions on pattern analysis and machine intelligence*, vol. 28, no. 9, pp. 1393–1403, 2006.

[31] Y. Weiss, "Segmentation using eigenvectors: a unifying view," in *Proceedings of the seventh IEEE international conference on computer vision*, vol. 2, pp. 975–982, IEEE, 1999.

[32] H. Badri, M. El Hassouni, and D. Aboutajdine, "Kernel-based laplacian smoothing method for 3d mesh denoising," in *Image and Signal Processing: 5th International Conference, ICISP 2012, Agadir, Morocco, June 28-30, 2012. Proceedings 5*, pp. 77–84, Springer, 2012.

[33] H. J. van Veen, N. Saul, D. Eargle, and S. W. Mangham, "kmapper.cover." `https://kepler-mapper.scikit-tda.org/en/latest/reference/stubs/kmapper.Cover.html`, 2019-2021.

[34] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Dbscan." `https://scikit-learn.org/stable/modules/clustering.html#dbscan`, 2007 - 2023.

[35] M. Ester, H.-P. Kriegel, J. Sander, X. Xu, *et al.*, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *kdd*, vol. 96, pp. 226–231, 1996.

[36] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "sklearn.cluster.dbscan." `https://scikit-learn.org/stable/modules/generated/sklearn.cluster.DBSCAN.html`, 2007 - 2023.

[37] M. Carriere, B. Michel, and S. Oudot, "Statistical analysis and parameter selection for mapper," *The Journal of Machine Learning Research*, vol. 19, no. 1, pp. 478–516, 2018.

[38] M. Carrière and B. Michel, "Statistical analysis of mapper for stochastic and multivariate filters," *Journal of Applied and Computational Topology*, vol. 6, no. 3, pp. 331–369, 2022.

[39] G. M. Reaven and R. G. Miller, "An attempt to define the nature of chemical diabetes using a multidimensional analysis," *Diabetologia*, vol. 16, pp. 17–24, 1979.

[40] D. F. D. F. Andrews, *Data : a collection of problems from many fields for the student and research worker / D. F. Andrews ; A. M. Herzberg.* Springer series in statistics, Springer-Verlag, 1985.

[41] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "sklearn.preprocessing.minmaxscaler." `https:`

//scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html, 2007 - 2023.

[42] N. Saul, "Tadasets 0.1.0 documentation." https://tadasets.scikit-tda.org/en/latest/index.html#, 2018.

[43] A. Hatcher, *Algebraic topology*. Cambridge: Cambridge University Press, 2002.

[44] H. J. van Veen, N. Saul, D. Eargle, and S. W. Mangham, "Keppler Mapper: A flexible Python implementation of the Mapper algorithm." https://doi.org/10.5281/zenodo.4077395, Oct. 2020.

[45] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Compare the effect of different scalers on data with outliers." https://scikit-learn.org/stable/auto_examples/preprocessing/plot_all_scaling.html#compare-the-effect-of-different-scalers-on-data-with-outliers, 2007 - 2023.

[46] J. A. Hartigan, *Clustering Algorithms*. John Wiley and Sons, 1975.

[47] N. Saul and C. Tralie, "Scikit-tda: Topological data analysis for python." https://doi.org/10.5281/zenodo.2533369, 2019.